CST306 : Algorithm analysis and design

Chapter 1
Foundations of algorithm analysis

Narasimhan T.

1.1 Algorithm

An **algorithm** is a finite sequence of instructions to solve a problem in a finite amount of time. It is a step by step procedure which, given an input from some set of possible inputs, enables us to obtain an output if such an output exists, or else obtain nothing at all if there is no output for that particular input. We say the algorithm transforms the input into output through a systematic execution of the instructions. Any algorithm must satisfy the following criteria :

- Input : To be supplied by the user.
- Output : To be generated by the algorithm.
- Definiteness : Each instruction must be clear and unambiguous.
- Finiteness : Algorithms must terminate after a finite number of steps.
- Effectiveness : Instructions in the algorithm must involve only basic arithmetic/logical operations.

An **instance of a problem** refers to the input needed to compute a solution to the problem. **Program** is the implementation of algorithm in some programming language. A **data structure** is a homogeneous collection of data items organized in memory in such a way that the insertion and deletion operations follow certain policies and rules. It is a systematic way of storing, organizing and accessing the data needed to solve a problem. The data structure for solving a problem should be carefully chosen because it directly affects the performance of the algorithm.

1.2 Analysis of algorithms

When we have a problem to solve, there may be several suitable algorithms available. We would obviously like to choose the best or the efficient one. This raises the question of how to decide which

of the several algorithms is preferable. An algorithm which has less running time and which takes less memory would be considered as efficient. Thus running time and space (computer memory) are two parameters that determine the efficiency of an algorithm (Although there could be many other parameters, we concentrate only on these two). We are interested in characterizing an algorithm's efficiency as a function of the amount of data it is supposed to process (i.e. input size). For small inputs, the algorithm may take less time to execute. As and when the input size increases, the execution time also grows generally. So it is traditional to study how the behaviour of an algorithm varies as the input size is increased and this forms the underlying principle of algorithm analysis.

Typically we analyse an algorithm to determine its time complexity and space complexity. **Time complexity** is expressing the running time of the algorithm as a function of the input size. **Space complexity** is the expression of amount of computer memory required during the algorithm execution, as a function of the input size. Time and space are both important, but we are usually more interested in time efficiency rather than in space efficiency. (These days, computer memories are very cheaper. So we really don't bother even if the algorithm takes some decent amount of space. Nevertheless the space complexity has indeed got a role to play while designing algorithms for hand-held (or similar) devices, where memory is constrained.)

1.2.1 RAM model of computation

To precisely analyze an algorithm, we need a mathematical model of computation. A model of computation is the definition of the set of allowable operations used in computation and their respective costs. We have to use the model so that we do not go too deep (into the machine instructions) as we want to have a machine independent analysis. Some examples of models include Turing machines, finite state machines, recursive functions, lambda calculus etc. For our discussions, we use the Random Access Machine (RAM) model. The assumptions made in the RAM model are :

- The model employs a single processor for its operations.
- Instructions are executed sequentially (and not concurrently).
- The model has instructions commonly found in real computers:
  - arithmetic (add, subtract, multiply, divide)
  - data movement (load, store, copy)
  - control (conditional and unconditional branch, subroutine call)
- Each such instruction takes a constant time.
- Loops and subroutines are not considered simple operations. Instead, they are composed of many single-step operations. The time it takes to run through a loop or execute a subroutine depends upon the number of loop iterations or the specific nature of the subroutine.
- Each memory access takes exactly one time step, and we have as much memory as we need. The RAM model takes no notice of whether an item is in cache or on the disk, which simplifies the analysis.
- The data types are float and int.

Under the RAM model, we measure the run time of an algorithm by counting up the number of steps it takes on a given problem instance. By assuming that our RAM executes a given number of steps per second, the operation count converts easily to the actual run time. The RAM model may sound

too simple. Perhaps there are situations in which the above assumptions fail. Yet the RAM proves an excellent model for understanding how an algorithm will perform on a real computer. It strikes a fine balance by capturing the essential behaviour of computers while being simple to work with. It is quite difficult to design an algorithm such that the RAM model gives you substantially misleading results. The robustness of the RAM enables us to analyze algorithms in a machine-independent way.

1.2.2 Time complexity

There are two techniques to analyse the running time of an algorithm. In the first technique called experimental or empirical analysis, we implement the algorithm in some programming language, run the program with various input sets and obtain the actual running time for each run of the program. But this approach of algorithm analysis has some limitations as listed below:

- It is necessary to implement the algorithm and run it in order to determine its running time. The implementation is very time consuming and largely depends on the programmer's skills.
- Experiments can be done only on a limited set of inputs. It is possible that the program runs quite fast on the input sets that we chose, but the same algorithm could take a longer time for some other inputs which we have not considered.
- If we want to compare two algorithms, the implemented programs need to be run in the same hardware and software environments (precisely, the same platform) which may not be feasible always.

In theoretical or *apriori* approach to analyse an algorithm, we mathematically determine the running time of an algorithm. The theoretical analysis:

- can be performed by using a high-level description of the algorithm (called pseudo-code) without actually implementing it.
- takes into account all possible inputs.
- allows one to evaluate the efficiency of an algorithm in a way that is independent of any platform.

This approach saves both the time that would have been spent needlessly programming an inefficient algorithm and the machine time that would have been wasted testing it.

1.2.2.1 Pseudo-code

Pseudo-code is a compact and informal high level description of an algorithm. It uses short, English-like phrases to describe the outline of an algorithm. Pseudo-code is not a computer program, but is more structured than English prose and less detailed than a program. It is a mixture of informal English statements and high-level programming constructs that describe the main ideas behind the generic implementation of an algorithm or of a data structure. These constructs include the following:

- Procedure declarations: The pseudo-codes for algorithms are presented as procedures or functions. The statement ABC(*param1*,*param2*,...,*paramn*) declares a new procedure with name ABC and with *param1*, *param2* ..., *paramn* as its parameters.

- Expressions: Standard mathematical symbols are used to form expressions. We use '=' as the assignment operator in assignment statements and '==' as the equality relation in Boolean expressions.
- Decision structures: **if** <condition> true-actions **else** false-actions. Here true-actions indicate what actions are to be performed if the condition is true. Similarly false-actions indicate what actions are to be performed if the condition is false. (All if statements need not have else part).
- **while** loop: **while** <condition> actions.
- **repeat** loop: **repeat** actions **until** condition.
- **for** loop: **for** <loop-variable-initialization> **to** <final-value-of-loop-variable> actions.

  Eg: **for** $k = 0$ **to** $n-1$
  $sum = sum + k$

  It is also possible to have a **for** loop that decrements the loop variable in each iteration.

  Eg: **for** $j = n$ **downto** 1
  $sum = sum + j$

  When the loop counter changes by an amount greater than 1, the amount of change follows the keyword **by**.

  Eg: **for** $k = 0$ **to** $n-1$ **by** 2 (k is increased by 2 every iteration)
- Array indexing: $A[i]$ represents the ith element in an array A. In a similar way, $B[i,j]$ denotes the jth element in the ith row of a two dimensional array B.

  The notation '..' is used to indicate a range of values within an array. Thus, $A[1..j]$ indicates the subarray of A consisting of the j elements $A[1]$,$A[2]$,$\ldots\ldots$,$A[j]$.
- Procedure calls and returns: We call a procedure with its name and specifying the arguments. When a procedure wants to return any value, we use **return** keyword.
- The symbol '// ' indicates that the remainder of the line is a comment.
- Handling errors: The keyword **error** indicates that an error has occurred. The calling procedure is responsible for handling the error, and so we do not specify what action to take.

Pseudo-code is independent of any programming language. It typically omits details that are not essential for understanding the algorithm such as variable declarations, semicolons and so on. Pseudo-code cannot be compiled nor executed, and does not follow any syntax rules.

To make ideas more concrete, let us write an algorithm in pseudo-code notation to find the sum of elements in an array A. The procedure ARRAY-SUM(A,n) takes the array A and its size n as parameters. The variable sum is initialized to 0. Then we access each array element using a **for** loop and add it to sum. Finally we return the value of the computed sum.

```
ARRAY-SUM(A,n)
1  sum = 0
2  for i = 1 to n
3        sum = sum + A[i]
4  return sum
```

1.2.2.2 Frequency count analysis

Suppose there is an assignment statement $x = x+y$ in our algorithm and we want to find its execution time. For this we need to determine two things: the time a single execution would take and the number of times it is executed (frequency). Then the total time taken would be just the product of time for single execution and frequency. Since the execution time is machine dependent, we neglect it and concentrate on the frequency. Thus for finding the time complexity of an algorithm, we rely on **frequency count** (aka order of magnitude) which is a count of how many times a statement is executed. If we find the frequency count for each statement and add them up, we get the frequency count for the entire algorithm which is a reasonable estimate of its time complexity.

By inspecting the pseudocode, we can determine its order of magnitude. We then express the time complexity as a function of the input size (this function represents the order of magnitude of the algorithm). The apriori analysis of algorithms is concerned primarily with order of magnitude determination. Time complexity is usually denoted by $T(n)$ where n represents the input size. Let us look at some examples.

Example 1.1.

NO-LOOP()	Frequency count
1 $x = x+y$	1

Line 1 is executed only once. Thus $T(n) = 1$.

Example 1.2.

CONSTANT-LOOP(c)	Frequency count
1 **for** $i = 1$ **to** c	$c+1$
2 $x = x+y$	c

The loop runs c times (i varies from 1 to c). During the $(c+1)^{th}$ iteration, $i = c+1 > c$. Hence loop will not be executed. Thus line 1 will be executed $c+1$ times in total (c times for true case and a check is performed for the loop exit (the false case)). We say c *true calls* and one *false call* are made to the statement. The second statement gets executed c times as it is inside the loop. Thus total frequency count would be $2c+1$, i.e. $T(n) = 2c+1$.

Example 1.3.

SIMPLE-LOOP(n)	Frequency count
1 **for** $i = 1$ **to** n	$n+1$
2 $x = x+y$	n

Line 1 is executed $n+1$ times. Line 2 is executed n times. Thus total frequency count would be $(n+1)+n \Longrightarrow T(n) = 2n+1$.

Note that the $T(n)$ obtained here is very much similar to the one obtained for the preceding example problem. Nevertheless they aren't the same, since with the preceding problem, c is a constant and hence $T(n) = 2c+1$ is independent of the input size n.

Example 1.4.

ANOTHER-SIMPLE-LOOP(n)	Frequency count
1 **for** $i = 1$ **to** n **by** i	$\lg n + 2$
2 $x = x + y$	$\lg n + 1$

- i varies as $1, 2, 4, 8, \cdots n$. Without loss of generality, we assume that n is a power of 2. i.e., $n = 2^k, k \geq 0$. For each value of i, we have one true call to line 1.
- The series $1, 2, 4, 8, \cdots n$ forms a GP with common ratio 2 and with m^{th} term as n.

  $$1.2^{m-1} = n$$
  $$\implies m - 1 = \lg n \quad \text{where lg denotes } \log_2$$
  $$\implies m = \lg n + 1$$

  Thus there are $\lg n + 1$ values of i and line 1 executes $\lg n + 2$ times. ($\lg n + 1$ true calls and one false call)
- Line 2 executes once for every true call to line 1 and thus executes $\lg n + 1$ times.

Thus $T(n) = 2\lg n + 3$

Example 1.5.

SEQUENTIAL-LOOP(m, n)	Frequency count
1 **for** $i = 1$ **to** m	$m + 1$
2 $x = x + y$	m
3 **for** $j = 1$ **to** n	$n + 1$
4 $y = y - 1$	n

Line 1 is executed $m + 1$ times. Line 2 is executed m times. Lines 3 and 4 get executed $n + 1$ and n times respectively. Thus $T(m, n) = 2m + 2n + 2$. Here we denote the time complexity as $T(m, n)$ as there are two input sizes m and n.

Example 1.6.

NESTED-LOOP(n)	Frequency count
1 **for** $i = 1$ **to** n	$n + 1$
2 **for** $j = 1$ **to** n	$n(n + 1)$
3 $x = x + y$	n^2

- Line 1 executes $n + 1$ times.
- For a single iteration of the outer loop, line 2 executes $n + 1$ times. Since the outer loop runs n times, line 2 executes $n(n + 1)$ times in total.
- A single iteration of the inner loop causes line 3 to execute n times. The outer loop causes the inner loop to run n times. Thus line 3 will get executed n^2 times in total.

Therefore $T(n) = (n+1) + n(n+1) + n^2 = 2n^2 + 2n + 1$.

Example 1.7.

ANOTHER-NESTED-LOOP(n)	Frequency count
1 **for** $i = 1$ **to** n	$n+1$
2 **for** $j = i+1$ **to** n	$\frac{n(n+1)}{2}$
3 $x = x+1$	$\frac{n(n-1)}{2}$

- Line 1 executes $n+1$ times.
- j varies from $(i+1)$ to n. When $i == 1, j$ varies from 2 to n. Then line 2 gets executed n times ($(n-1)$ true calls and one false call). When $i == 2, j$ varies from 3 to n and line 2 gets executed $(n-1)$ times. When $i == n$, it will execute just once. Thus line 2 executes $n + (n-1) + ... + 1 = \frac{n(n+1)}{2}$ times.
- Line 3 is executed once for every true call to line 2 but not for any false call. Out of the $\frac{n(n+1)}{2}$ executions of line 2, n calls are false. (For each value of i, there will be a false call to line 2). Thus line 3 executes $\frac{n(n+1)}{2} - n = \frac{n(n-1)}{2}$ times.

Thus $T(n) = (n+1) + \frac{n(n+1)}{2} + \frac{n(n-1)}{2} = n^2 + n + 1$

Example 1.8.

YET-ANOTHER-NESTED-LOOP(n)	Frequency count
1 **for** $i = 1$ **to** n	$n+1$
2 **for** $j = 1$ **to** i	$\frac{n(n+3)}{2}$
3 $x = x+1$	$\frac{n(n+1)}{2}$

- Line 1 executes $n+1$ times.
- j varies from 1 to i. When $i == 1, j$ varies from 1 to 1. Then line 2 gets executed 2 times (one true call and one false call). When $i == 2, j$ varies from 1 to 2 and line 2 gets executed thrice. When $i == n$, it will execute $n+1$ times. Thus line 2 executes $2 + 3 + ... + n + (n+1) = \frac{n(n+3)}{2}$ times.
- Line 3 is executed once for every true call to line 2 but not for any false call. Out of the $\frac{n(n+3)}{2}$ executions of line 2, n calls are false. (For each value of i, there will be a false call to line 2). Thus line 3 executes $\frac{n(n+3)}{2} - n = \frac{n(n+1)}{2}$ times.

Thus $T(n) = (n+1) + \frac{n(n+3)}{2} + \frac{n(n+1)}{2} = n^2 + 3n + 1$

Example 1.9.

NTH-FIBONACCI(n)	Frequency count
1 **if** $n == 0 \,\|\|\, n == 1$	1
2 **return** n	1
3 **else**	0
4 $prev1 = 0$	1
5 $prev2 = 1$	1
6 **for** $i = 2$ **to** n	n
7 $next = prev1 + prev2$	$n-1$
8 $prev1 = prev2$	$n-1$
9 $prev2 = next$	$n-1$
10 **return** $next$	1

If $n == 0$ or $n == 1$, only statements 1 and 2 will be executed. Then the total frequency count will be 2. Otherwise we get a frequency count of $4n+1$. (Add the frequency counts of all statements except line 2 which is not executed when $n > 1$). Thus

$$T(n) = \begin{cases} 2, & \text{if } n == 0 \text{ or } n == 1. \\ 4n+1, & \text{otherwise.} \end{cases}$$

■

1.2.2.3 Analyzing recursive algorithms

We adopt a slightly different strategy to analyse algorithms that have recursive procedures. We associate with each recursive procedure, the time complexity function $T(n)$, where n measures the input size. Thus the time for recursive calls in the procedure can then be expressed in terms of this function T. While analyzing recursive procedures, we don't get a closed form expression for $T(n)$ (like $n^2+3n+1, 2n^2+2n+1$). What we get is a recurrence for $T(n)$. A **recurrence** is an equation or inequality that describes a function in terms of its value on smaller inputs i.e. we describe $T(n)$ in terms of $T(k)$ for various values of k such that $k < n$. These recurrences are to be solved to get the closed form expression for $T(n)$. Several techniques exist for solving recurrences (which we don't discuss here). The following examples should make concepts clearer.

Example 1.10.

RECURSIVE-ARRAY-SUM(A, n)	Time Complexity
1 **if** $n == 0$	c_1
2 **return** 0	c_2
3 **else**	0
4 **return** RECURSIVE-ARRAY-SUM($A, n-1$) + $A[n]$	$c_3 + T(n-1)$

- Lines 1 and 2 take constant time and the corresponding running time values are denoted as c_1 and c_2 respectively.
- Let $T(n)$ be the running time of RECURSIVE-ARRAY-SUM(A, n). Then the time complexity of recursive call RECURSIVE-ARRAY-SUM($A, n-1$) can be expressed as $T(n-1)$.

- The running time of line 4 will be as $c_3 + T(n-1)$. Here $T(n-1)$ accounts for the recursive call and the constant c_3 accounts for addition, return and array indexing operations in the statement.
- $T(n)$ can then be expressed as

$$T(n) = \begin{cases} c_1 + c_2, & \text{if } n == 0 \\ c_1 + c_3 + T(n-1), & \text{otherwise} \end{cases}$$

Letting $c = c_1 + c_2$ and $d = c_1 + c_3$, the time complexity of RECURSIVE-ARRAY-SUM(A, n) could be represented by the recurrence:

$$T(n) = \begin{cases} c, & \text{if } n == 0 \\ d + T(n-1), & \text{otherwise} \end{cases}$$

Equivalently we could also write $T(n) = d + T(n-1)$, with $T(0) = c$

Example 1.11.

FACTORIAL(n)	Time Complexity
1 **if** $n == 0$	c_1
2 **return** 1	c_2
3 **else**	0
4 **return** $n \times$ FACTORIAL$(n-1)$	$c_3 + T(n-1)$

- Lines 1 and 2 take constant time and are taken as c_1 and c_2 respectively.
- The running time of line 4 can be taken as $c_3 + T(n-1)$. Here $T(n-1)$ accounts for the recursive call and the constant c_3 accounts for multiplication and return.
- $T(n)$ can then be expressed as

$$T(n) = \begin{cases} c_1 + c_2, & \text{if } n == 0 \\ c_1 + c_3 + T(n-1), & \text{otherwise} \end{cases}$$

Letting $c = c_1 + c_2$ and $d = c_1 + c_3$, the time complexity of FACTORIAL(n) could be represented by the recurrence:

$$T(n) = \begin{cases} c, & \text{if } n == 0 \\ d + T(n-1), & \text{otherwise} \end{cases}$$

Example 1.12.

DO-SOMETHING(n)	Time Complexity
1 **if** $n <= 1$	c_1
2 **return** 2	c_2
3 **else**	0
4 **return** $3 \times$ DO-SOMETHING$\left(\frac{2n}{3}\right)$ + DO-SOMETHING$\left(\frac{n}{3}\right) + 5$	$T\left(\frac{2n}{3}\right) + T\left(\frac{n}{3}\right) + c_3$

- Lines 1 and 2 take constant time and are taken as c_1 and c_2 respectively.
- The running time of line 4 can be taken as $T\left(\frac{2n}{3}\right)+T\left(\frac{n}{3}\right)+c_3$. Here $T\left(\frac{2n}{3}\right)$ and $T\left(\frac{n}{3}\right)$ account for the recursive calls and the constant c_3 accounts for multiplication, addition and return.
- $T(n)$ can then be expressed as

$$T(n)=\begin{cases}c_1+c_2, & \text{if } n\leq 1\\ T\left(\frac{2n}{3}\right)+T\left(\frac{n}{3}\right)+c_1+c_3, & \text{otherwise}\end{cases}$$

Letting $c=c_1+c_2$ and $d=c_1+c_3$, the time complexity of DO-SOMETHING(n) could be represented by the recurrence:

$$T(n)=\begin{cases}c, & \text{if } n\leq 1\\ d+T\left(\frac{2n}{3}\right)+T\left(\frac{n}{3}\right), & \text{otherwise}\end{cases}$$

1.2.3 Space complexity

Space complexity of an algorithm is the expressing of its memory requirements as a function of its input size. It includes the memory needed to store the input data, temporary variables and also the space needed to store the code itself. As for time complexity, we want a machine independent view for space complexity as well. Space complexity is taken as the number of words used up in memory. The actual word size depends on the architecture used. For some architectures, 1 word = 32 bits while in some others, 1 word = 64 bits.

Space required by any algorithm is the sum of following components:

1. **Fixed Component** : This is independent of the characteristics of the inputs and outputs. This component includes instruction space (space for the code), space for simple variables and space for constants. It also includes the space for static data structures whose sizes are known prior.
2. **Variable Component** : This consists of the space needed by component variables whose size is dependent on the particular problem instance being solved, the space needed by reference variables and the recursion stack space. Also this includes the space for dynamic data structures.

☞ The total space requirements $S(n)$ for any algorithm A can be written as $S(n)=C+S_A$, where n is the input size, C denotes the fixed component and S_A represents the variable component.

Example 1.13. Consider the procedure ABC(a,b,c). If we assume that each of the variables x,y and z need one word for storage, $C=c_1+3$, where c_1 accounts for other contributions to the fixed component (including the space for code itself). The variable component $S_{ABC}=0$ as the algorithm is independent of any instance characteristics. Thus $S(n)=c_1+3=c_2$.

ABC(x,y,z)

1 **return** $x+y*z$

Example 1.14. See the pseudocode ARRAY-SUM(A,n). For this algorithm, you require one word each for the variables n, i and sum. As before, take c_1 for other contributions. Now, regarding the space requirements for array A, you may have to bring the programmatic implementation into picture. For languages like C, arrays are always passed by reference. This means, only the base address of A needs to be stored by ARRAY-SUM(A,n). The entire array contents can be accessed if the base address is known. Assuming that addresses require 1 word of storage, we write $C = c_1 + 4$. Since there are no input-specific memory requirements, $S_{\text{ARRAY-SUM}} = 0$, and $S(n) = C = c_1 + 4$.

```
ARRAY-SUM(A, n)
1  sum = 0
2  for i = 1 to n
3        sum = sum + A[i]
4  return sum
```

On the other hand, programming languages like Pascal pass arrays by value. This means the pseudocode needs to store the entire array. Thus $S_{\text{ARRAY-SUM}} = n$ (array size is n) $\implies S(n) = c_1 + 3 + n$. ■

Let us move to recursive procedures. Recursion is implemented using the stack data structure. The *recursion stack* stores the formal parameters (input values to the procedure), local variables and return address. Since these values might differ for each recursive call, you require a separate stack for each of those calls.

> ☞ The number of recursive calls (including the initial one) to a function is called *recursion depth*.

Example 1.15. Consider RECURSIVE-FACTORIAL(n). Assuming that return address requires only one word, each recursive call requires 3 words (space for n, return address and return value). Since recursion depth is $(n+1)$, total recursion stack space = $3(n+1)$. Thus $S_{\text{RECURSIVE-FACTORIAL}} = 3(n+1)$. This means $S(n) = C + 3(n+1)$, where C is the fixed component.

```
RECURSIVE-FACTORIAL(n)
1  if n == 0
2        return 1
3  else
4        return n × RECURSIVE-FACTORIAL(n − 1)
```

Example 1.16. Finally consider RECURSIVE-ARRAY-SUM(A,n). Assuming pass by reference implementation for arrays, each recursive call requires 4 words (space for A, n, return address and return value). Since recursion depth is $(n+1)$, total recursion stack space = $4(n+1)$. Thus the variable component, $S_{\text{RECURSIVE-ARRAY-SUM}} = 4(n+1)$. Let the fixed component be C. Then $S(n) = C + 4(n+1)$.

```
RECURSIVE-ARRAY-SUM(A, n)
1  if n == 0
2        return 0
3  else
4        return RECURSIVE-ARRAY-SUM(A, n − 1) + A[n]
```

■

Time complexity vs. space complexity : Time-space trade-off

In general, we can often decrease the time complexity of an algorithm but this will incur an increase in the space complexity and vice versa (decrease space ⇒ increase time). This is the time-space trade-off. For example, the time complexity of an iterative sorting algorithm (say, bubble sort) is higher when compared to that of a recursive algorithm (take quick sort). But the recursion (which uses stack) causes an increase in memory requirements (i.e. space complexity goes up). On the contrary, the space requirements for a sparse matrix is much lower with the linked list representation when compared with array representation. But the other side is that, with linked list, the time taken to access each element will rise (i.e. the time complexity will rise).

1.2.4 Best, average and worst case analyses of algorithms

The time taken by an algorithm can vary considerably between two different instances of the same size. In other words, for many algorithms, the running time is not only a function of the input size but also of the particular input instance. This brings us to the notion of best, average and worst case complexities of an algorithm.

The worst case analysis deals with the behaviour of the algorithm with respect to the worst possible case of the input instance. The worst-case running time is the largest running time for any input of size n. It gives us an upper bound on the running time for any input. Knowing it provides a guarantee that the algorithm will never take any longer. As an illustration, the worst case for linear search occurs when the element to be searched is not present in the array. Worst-case analysis is appropriate for an algorithm whose response time is critical. For example, if it is a question of controlling a nuclear power plant, it is crucial to know an upper limit on the system's response time, regardless of the particular instance to be solved.

Often it happens that an algorithm may run faster on some inputs than it does on others of the same size. In such cases, we may wish to express the running time of an algorithm as the function of the input size by taking the average over all possible inputs of the same size. Thus the average-case running time of an algorithm is an estimate of the running time for an "**average**" input. The scope of average-case analysis is limited, because it may not be clear what constitutes an average input for a particular problem. Computation of average-case running time requires knowing all possible input sequences, the probability distribution of occurrence of these sequences, and the running times for the individual sequences. Often it is assumed that all inputs of a given size are equally likely.

In the best case analysis, we calculate the lower bound on the running time of an algorithm. The best case running time is the smallest running time for any input of size n. Best case analysis, generally is of less practical importance. For linear search, best case occurs when the required element is at the first position of array.

Taking $T_{avg}(n)$ as the average case running time, $T_{best}(n)$ as the best case running time and $T_{worst}(n)$ as the worst case running time, we can write $T_{best}(n) \leq T_{avg}(n) \leq T_{worst}(n)$.

Unless otherwise specified, $T(n)$ is taken as the worst case running time, because it provides a bound for all inputs including particularly bad input, which an average case analysis does not provide. One algorithm is considered more efficient than another if it has lower worst case running time.

CST306 : Algorithm analysis and design

Chapter 2
Asymptotic analysis

Narasimhan T.

2.1 Introduction

Performing frequency count calculations for large algorithms is very tedious and could be error prone also. A natural way is to determine an upper bound for the time complexity instead of the exact frequency count. **Order of growth** or **rate of growth** of an algorithm gives a simple characterization of the algorithm's efficiency by identifying the relatively significant terms in its frequency count. As an example, for an algorithm with a frequency count of $2n^2+3n+1$, the order of growth depends only on $2n^2$ for large n. Other terms are relatively insignificant as n increases. For fairly large values of n, even the multiplicative constant 2 can be omitted and time complexity is dependent only on n^2. Thus for very large values of n, lower order terms and multiplicative constants in an algorithm's time complexity function can be ignored.

Once we dispose of the lower order terms and leading constants from the time complexity function, what we get is the **asymptotic** running time of the algorithm. In other words, we are performing an **asymptotic analysis** of the algorithm. The term "asymptotic" is used in the sense that we are dealing with the study of behaviour of functions in the limit, that is for sufficiently large values of its parameters (say input size, n).

2.2 Asymptotic notations

To formalize the notion of order of growth of time complexity functions, asymptotic analysis uses special mathematical notations called **asymptotic notations**. Asymptotic notation is a convenient way of describing the growth rate of a function and hence the time complexity of an algorithm. It allows us to ignore constant factors and lower order terms and focus on the main components of a function that affect its growth. The goal of asymptotic notations is to find an upper bound or a lower bound for the time/space complexity of a particular algorithm. The various asymptotic notations are:

1. Big-oh - O
2. Big-omega - Ω

3. Big-theta - Θ
4. Small-oh - o
5. Small-omega - ω

2.2.1 Big-Oh notation

One of the most commonly used asymptotic notation is Big-Oh. This is pictorially shown in Figure 2.3. For values of $n < n_0$, there is no consistent relationship between n and $g(n)$. For some values of n, $f(n) > c \cdot g(n)$. For some other values, $f(n) < c \cdot g(n)$. But for all values of $n \geq n_0$, we see that $f(n) \leq c \cdot g(n)$. Thus we can formally define big-oh as follows:

Definition 2.1. *A function $f(n) = O(g(n))$ (read as "big-oh of g of n" or "order of g of n") if and only if there exists two positive constants c and n_0 (i.e. $c, n_0 > 0$) such that $0 \leq f(n) \leq c \cdot g(n) \; \forall \; n \geq n_0$.*

When we say that an algorithm has computing time $O(g(n))$, we mean that when the algorithm is run on some computer on the same type of data but for increasing values of n, the resulting time is always some constant times $g(n)$. The statement "$f(n)$ is $O(g(n))$" means that the growth rate of $f(n)$ is no more than the growth rate of $g(n)$. Thus, the O-notation provides an upper bound on the running time of an algorithm.

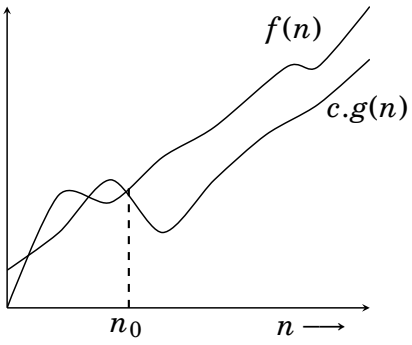


Figure 2.1: $f(n) = \Omega(g(n))$

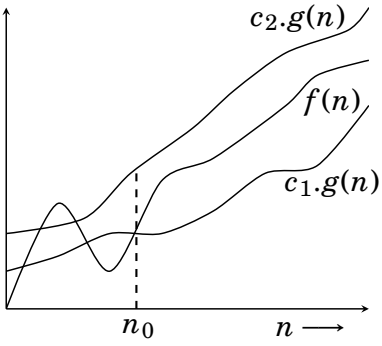


Figure 2.2: $f(n) = \Theta(g(n))$

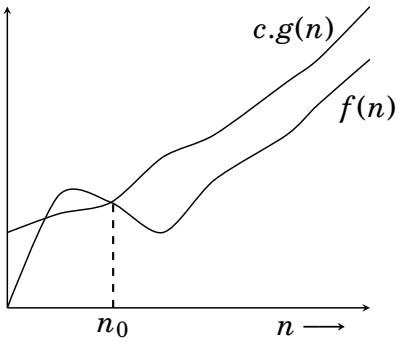


Figure 2.3: $f(n) = O(g(n))$

Having seen the definition of big-oh, let us now try to express some time complexity functions in big-oh notation. The following result may come handy.

☞ If $f(n)$ is a polynomial of degree d, then $f(n) = O(n^d)$. (2.1)

When the given function contains non polynomial terms also, we need a way to identify the term that determines the asymptotic growth rate of the function. We can use the big-oh notation to rank various time complexity functions according to their growth rate as follows:

$$O(1) < O(\log\log n) < O(\log n) < O(\log^2 n) < O(\sqrt{n}) < O(n) < O(n\log n) < O(n^2) < O(n^3) < O(2^n) < O(n!) = O(n^n)$$

The last equality $O(n!) = O(n^n)$ comes from the Stirling's approximation:- $n! \approx n^n$.We write $\log^2 n$ to mean $(\log n)^2$. $O(1)$ means that the execution time is a constant that doesn't depend on the input size. $O(\log n)$ is

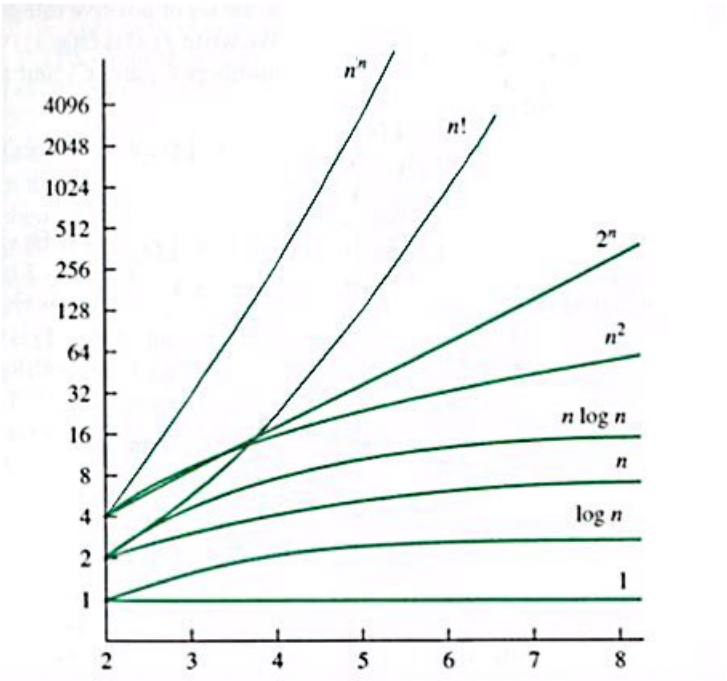


Figure 2.4: Comparison of growth of functions

called logarithmic. [Unless otherwise specified, $\log n$ will be taken as $\log_2 n$.] $O(n)$ is linear, $O(n \log n)$ is linearithmic, $O(n^2)$ is quadratic, $O(n^3)$ is cubic and $O(2^n)$ is exponential (Algorithms with exponential complexity are practically not feasible).

The above comparison means that if an algorithm runs in logarithmic time, it is faster than if it had taken linear time (for sufficiently large n). Similarly we can compare other pairs of functions. Figure 2.4 depicts the comparison of growth rates of some functions commonly encountered in algorithm analysis.

Example 2.1. Express the following functions in big-oh notation:

2.1.1 $3n+2$

Claim: $3n+2 = O(n)$

We need to prove that $3n+2 \leq c.n \ \forall n \geq n_0$

Since $2 \leq 2n, \ \forall n \geq 1$ we have

$3n+2 \leq 3n+2n, \ \forall n \geq 1$

$= (3+2)n$

$= 5n$

Thus $3n+2 \leq 5n \ \forall n \geq 1$

$\implies 3n+2 = O(n)$ with $c = 5, n_0 = 1$

2.1.2 $10n^2+4n+2$

Claim: $10n^2+4n+2 = O(n^2)$

$10n^2+4n+2 \leq (10+4+2)n^2, \ \forall n \geq 1$

$= 16n^2$

Thus $10n^2+4n+2 \leq 16n^2 \ \forall n \geq 1$

$\implies 10n^2+4n+2 = O(n^2)$ with $c = 16, n_0 = 1$

2.1.3 $100n^5+10n^3+6$

Claim: $100n^5+10n^3+6 = O(n^5)$

$100n^5+10n^3+6 \leq (100+10+6)n^5, \ \forall n \geq 1$

$= 116n^5$

Thus $100n^5 + 10n^3 + 6 \leq 116n^5 \ \forall n \geq 1$
$\implies 100n^5 + 10n^3 + 6 = O(n^5)$ with $c = 116, n_0 = 1$

2.1.4 $3n\log n + 5$
Claim: $3n\log n + 5 = O(n\log n)$
$3n\log n + 5 \leq (3+5)n\log n, \ \forall n \geq 2$
$= 8n\log n$
Thus $3n\log n + 5 \leq 8n\log n \ \forall n \geq 2$
$\implies 3n\log n + 5 = O(n\log n)$ with $c = 8, n_0 = 2$
Note that when $n = 1, n\log n$ becomes 0
$\implies 3n\log n + 5 \not\leq 8n\log n \implies n_0 = 2$

2.1.5 $6 \cdot 2^n + n^2$
Claim: $6 \cdot 2^n + n^2 = O(2^n)$
$6 \cdot 2^n + n^2 \leq 7 \cdot 2^n \ \forall \ n \geq 4.$
$\implies 6 \cdot 2^n + n^2 = O(2^n)$ with $c = 7, n_0 = 4$
Note that $n^2 \leq 2^n$ only for values of $n \geq 4$
For example when $n = 3, n^2 \not\leq 2^n$.

2.1.6 $2^{n+2} + 100$
Claim: $2^{n+2} + 100 = O(2^n)$
$2^{n+2} + 100 = 2^2 \cdot 2^n + 100$
$= 4 \cdot 2^n + 100$
$\leq 104 \cdot 2^n \ \forall \ n \geq 7.$
$\implies 2^{n+2} + 100 = O(2^n)$ with $c = 104, n_0 = 7$
Note that $100 \leq 2^n$ only for values of $n \geq 7$

■

Consider Problem 2.1.1. $3n+2$ is also $O(n^2)$ since $3n+2 \leq 3n^2 \forall n \geq 2$. But we chose $3n+2 = O(n)$. Similarly we could write $10n^2 + 4n + 2 = O(n^4)$ as $10n^2 + 4n + 2 \leq 10n^4 \ \forall n \geq 2$. $10n^2 + 4n + 2$ is also $O(n^3)$. But we choose $10n^2 + 4n + 2 = O(n^2)$. In other words, while choosing upper bound, we pick the least upper bound possible. That is, when we write $f(n) = O(g(n))$, we want the function $g(n)$ to be as small as possible. Nevertheless, $10n^2 + 4n + 2 \neq O(n)$ as $10n^2 + 4n + 2 \not\leq c.n$ for any positive values of c and n_0. Similarly $3n + 2 \neq O(1)$.

Example 2.2. Analyse the following code for its time complexity and represent the same in big-oh notation:

STILL-ANOTHER-NESTED-LOOP(n)	Frequency count
1 **for** $i = 1$ **to** n	$n+1$
2 **for** $j = 1$ **to** n **by** j	$n(\lg n + 2)$
3 $x = x + y$	$n(\lg n + 1)$

- Line 1 executes $n + 1$ times.
- From Example 1.4, it is clear that the **for** loop of line 2 executes $\lg n + 2$ times on its own right. But since line 2 is part of the outer loop, it will execute $n(\lg n + 2)$ times in total.
- Line 3 will execute only for the true calls of line 2. Among the total $n(\lg n + 2)$ calls of line 2, there are n false calls (one for each iteration of outer loop). This means that line 3 executes $n(\lg n + 1)$ times in total.

Thus

$$\begin{aligned} T(n) &= (n+1) + n(\lg n + 2) + n(\lg n + 1) \\ &= 2n\lg n + 4n + 1 \\ &\leq 7n\lg n \quad \forall n \geq 2 \end{aligned}$$

$\implies T(n) = O(n\lg n)$ with $c = 7, n_0 = 2$

2.2.2 Big Omega notation

Just as big-oh gives the upper bound of the time complexity of an algorithm, big omega (Ω) gives its lower bound. The plot for Ω-notation is shown in Figure 2.1. This takes us to the following definition:

Definition 2.2. *A function $f(n) = \Omega(g(n))$ if and only if there exists two positive constants c and n_0 (i.e. $c, n_0 > 0$) such that $0 \leq c \cdot g(n) \leq f(n)\ \forall n \geq n_0$.*

☞ If $f(n)$ is a polynomial of degree d, then $f(n) = \Omega(n^d)$. (2.2)

Example 2.3. Express the following functions in big-omega notation:

2.3.1 $100n + 6$
$100n + 6 \geq 100n\ \forall n \geq 1$. [This holds for all $n \geq 0$, but we want n_0 to be positive.]
$\implies 100n + 6 = \Omega(n)$ with $c = 100, n_0 = 1$.

2.3.2 $10n^2 + 4n + 2$
$10n^2 + 4n + 2 \geq 10n^2\ \forall n \geq 1$.
$\implies 10n^2 + 4n + 2 = \Omega(n^2)$ with $c = 10, n_0 = 1$.

2.3.3 $6 \cdot 2^n + n^2$
$6 \cdot 2^n + n^2 \geq 6 \cdot 2^n\ \forall n \geq 1$.
$\implies 6 \cdot 2^n + n^2 = \Omega(2^n)$ with $c = 6, n_0 = 1$.

Observe that in Problem 2.3.1, $100n + 6 = \Omega(1)$ since $100n + 6 \geq 100\ \forall n \geq 1$. But we chose $100n + 6 = \Omega(n)$. Similarly we could write $6 \cdot 2^n + n^2 = \Omega(n^2)$ or $6 \cdot 2^n + n^2 = \Omega(n)$ or even $6 \cdot 2^n + n^2 = \Omega(1)$, although we prefer to write $6 \cdot 2^n + n^2 = \Omega(2^n)$. That is, of all the lower bounds possible, we choose the greatest lower bound. When we write $f(n) = \Omega(g(n))$, we want the function $g(n)$ to be as large as possible.

2.2.3 Theta notation

Theta notation gives an asymptotically tight bound for the time complexity. This notation is plotted in Figure 2.2. From the graph, we get the following definition:

Definition 2.3. *A function $f(n) = \Theta(g(n))$ if and only if there exists positive constants c_1, c_2 and n_0 such that $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)\ \forall n \geq n_0$.*

When we write $f(n) = \Theta(g(n))$, the function $f(n)$ has $g(n)$ as its lower and upper bounds. That is,

$$f(n) = \Theta(g(n)) \iff f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n)) \tag{2.3}$$

☞ If $f(n)$ is a polynomial of degree d, then $f(n) = \Theta(n^d)$. (2.4)

The above result follows immediately from (2.1), (2.2) and (2.3).

Example 2.4. Express the following functions in theta notation:

2.4.1 $3n+2$
$3n+2 \geq 3n \; \forall n \geq 1$ and $3n+2 \leq 5n \; \forall n \geq 1$
Thus $3n \leq 3n+2 \leq 5n \; \forall n \geq 1$
$\implies 3n+2 = \Theta(n)$ with $c_1 = 3, c_2 = 5, n_0 = 1$.

2.4.2 $10\log n + 4$
$10\log n \leq 10\log n + 4 \leq 14\log n \; \forall n \geq 2$.
$\implies 10\log n + 4 = \Theta(\log n)$ with $c_1 = 10, c_2 = 14, n_0 = 2$.

2.4.3 $6 \cdot 2^n + n^2$
$6 \cdot 2^n \leq 6 \cdot 2^n + n^2 \leq 7 \cdot 2^n \; \forall n \geq 4$.
$\implies 6.2^n + n^2 = \Theta(2^n)$ with $c_1 = 6, c_2 = 7, n_0 = 4$.

2.2.4 Small oh and small omega notations

Definition 2.4. *A function $f(n) = o(g(n))$ if and only if for any positive constant $c > 0$, there exists a constant $n_0 > 0$ such that $0 \leq f(n) < c \cdot g(n) \; \forall \; n \geq n_0$.*

For example, $2n = o(n^2)$ but $2n^2 \neq o(n^2)$

The definitions of O-notation and o-notation are similar. The decisive difference is that when we write $f(n) = O(g(n))$, the bound $0 \leq f(n) \leq c \cdot g(n)$ holds for some constant $c > 0$, but when we write $f(n) = o(g(n))$, the bound $0 \leq f(n) < c \cdot g(n)$ should hold for all constants $c > 0$.

Intuitively, if $f(n) = o(g(n))$, the function $f(n)$ becomes insignificant relative to $g(n)$ as n approaches infinity; that is,

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$$

Definition 2.5. *A function $f(n) = \omega(g(n))$ if and only if for any positive constant $c > 0$, there exists a constant $n_0 > 0$ such that $0 \leq c \cdot g(n) < f(n) \; \forall \; n \geq n_0$.*

For example, $\frac{n^2}{2} = \omega(n)$ but $\frac{n^2}{2} \neq \omega(n^2)$.

Again, the definitions of Ω-notation and ω-notation are similar. The crucial difference is that when we write $f(n) = \Omega(g(n))$, the bound $0 \leq c \cdot g(n) \leq f(n)$ holds for some constant $c > 0$, but when we write $f(n) = \omega(g(n))$, the bound $0 \leq c \cdot g(n) < f(n)$ should hold for all constants $c > 0$.

When we write $f(n) = \omega(g(n))$, the function $f(n)$ becomes arbitrarily large relative to $g(n)$ as n approaches infinity. Thus we have,

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty,$$

2.3 Properties of asymptotic notations

1. **Reflexivity**
   (a) $f(n) = \Theta(f(n))$
   (b) $f(n) = \Omega(f(n))$
   (c) $f(n) = O(f(n))$
2. **Symmetry**
   (a) $f(n) = \Theta(g(n))$ if and only if $g(n) = \Theta(f(n))$
3. **Transpose Symmetry**
   (a) $f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$
   (b) $f(n) = o(g(n))$ if and only if $g(n) = \omega(f(n))$
4. **Transitivity**
   (a) $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n)) \implies f(n) = \Theta(h(n))$
   (b) $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n)) \implies f(n) = \Omega(h(n))$
   (c) $f(n) = O(g(n))$ and $g(n) = O(h(n)) \implies f(n) = O(h(n))$
   (d) $f(n) = \omega(g(n))$ and $g(n) = \omega(h(n)) \implies f(n) = \omega(h(n))$
   (e) $f(n) = o(g(n))$ and $g(n) = o(h(n)) \implies f(n) = o(h(n))$

2.4 Solved Problems

2.4.1 Express 2^{100} and $\frac{5}{n}$ in O-notation.

$2^{100} \leq 2^{100}.1$ for all $n \geq 1$. Thus $2^{100} = O(1)$. Note that variable n doesn't appear in the inequality since 2^{100} is a constant valued function.

$\frac{5}{n} \leq 5.\frac{1}{n}$ for all $n \geq 1$. Thus $\frac{5}{n} = O\left(\frac{1}{n}\right)$

2.4.2 Prove that $f(n) = 12n^2 + 6n$ is $o(n^3)$ and $\omega(n)$

$$\lim_{n\to\infty} \frac{12n^2+6n}{n^3} = \lim_{n\to\infty}\left(\frac{12}{n} + \frac{6}{n^2}\right) = 0 \implies 12n^2 + 6n \text{ is } o(n^3)$$

Similarly,

$$\lim_{n\to\infty} \frac{12n^2+6n}{n} = \lim_{n\to\infty} (12n+6) = \infty \implies 12n^2 + 6n \text{ is } \omega(n)$$

2.4.3 Let $f(n)$ and $g(n)$ be asymptotically nonnegative functions. Using the basic definition of Θ-notation, prove that $\max(f(n), g(n)) = \Theta(f(n) + g(n))$.

Since $f(n)$ $g(n)$ are asymptotically non-negative, we can assume that

There exists n_1, n_2 such that $f(n) \geq 0 \quad$ for $n > n_1$

$g(n) \geq 0 \quad$ for $n > n_2$

Let $n_0 = \max(n_1, n_2)$. Then for all $n > n_0$:

$f(n) \leq \max(f(n), g(n))$
$g(n) \leq \max(f(n), g(n))$

$(f(n)+g(n)) \le 2.\max(f(n),g(n)) \implies \frac{1}{2}(f(n)+g(n)) \le \max(f(n),g(n))$
We also have $\max(f(n),g(n)) \le f(n)+g(n)$

From the last two inequalities, we get:

$0 \le \frac{1}{2}(f(n)+g(n)) \le \max(f(n),g(n)) \le f(n)+g(n) \quad \text{for } n > n_0$

Thus $\max(f(n),g(n)) = \Theta(f(n)+g(n))$ with $c_1 = \frac{1}{2}, c_2 = 1$.

2.4.4 Show that $(n+a)^b = \Theta(n^b)$ for any real constants a and b, where $b > 0$.

To show that $(n+a)^b = \Theta(n^b)$, we want to find constants $c_1, c_2, n_0 > 0$ such that $0 \le c_1 n^b \le (n+a)^b \le c_2 n^b$ for all $n \ge n_0$.

Note that

$$\begin{aligned} n + a &\le n + |a| \\ &\le 2n \quad \text{when } |a| \le n, \end{aligned}$$

and

$$\begin{aligned} n + a &\ge n - |a| \\ &\ge \frac{1}{2}n \quad \text{when } |a| \le \frac{1}{2}n. \end{aligned}$$

Thus, when $n \ge 2|a|$,

$0 \le \frac{1}{2}n \le n + a \le 2n.$

Since $b > 0$, the inequality still holds when all terms are raised to the power b:

$0 \le \left(\frac{1}{2}n\right)^b \le (n+a)^b \le (2n)^b,$
$0 \le \left(\frac{1}{2}\right)^b n^b \le (n+a)^b \le 2^b n^b.$

Thus, $c_1 = \left(\frac{1}{2}\right)^b$, $c_2 = 2^b$, and $n_0 = 2|a|$ satisfy the definition.

2.4.5 Is $2^{n+1} = O(2^n)$? Is $2^{2n} = O(2^n)$?

$2^{n+1} = O(2^n)$, but $2^{2n} \ne O(2^n)$

To show that $2^{n+1} = O(2^n)$, we must find constants $c, n_0 > 0$ such that

$0 \le 2^{n+1} \le c.2^n \text{ for all } n \ge n_0$

Since $2^{n+1} = 2.2^n$ for all n, we can satisfy the definition with $c = 2$ and $n_0 = 1$.

To show that $2^{2n} \ne O(2^n)$, assume there exist constants $c, n_0 > 0$ such that

$0 \le 2^{2n} \le c.2^n \text{ for all } n \ge n_0$

Then $2^{2n} = 2^n.2^n \le c.2^n \implies 2^n \le c$. But no constant is greater than all 2^n, and so the assumption leads to a contradiction. Thus we conclude $2^{2n} \ne O(2^n)$.

2.4.6 Prove the statement : "For any two functions $f(n)$ and $g(n)$, we have $f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$ ".

We need to prove in both directions, i.e., if $f(n) = \Theta(g(n))$, then $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$ and if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$, then $f(n) = \Theta(g(n))$.

Assume $f(n) = \Theta(g(n))$, then we have that:

$0 \le c_1 g(n) \le f(n) \le c_2 g(n) \quad \text{for } n > n_0$

$c_1 g(n) \le f(n) \implies f(n) = \Omega(g(n))$ and $f(n) \le c_2 g(n) \implies f(n) = O(n)$

For the reverse direction, assume that $f(n) = \Omega(g(n))$ and $f(n) = O(g(n))$. Then:

$0 \leq c_3 g(n) \leq f(n)$ for all $n \geq n_1$
$0 \leq f(n) \leq c_4 g(n)$ for all $n \geq n_2$

If we let $n_3 = \max(n_1, n_2)$ and merge the inequalities, we get:

$0 \leq c_3 g(n) \leq f(n) \leq c_4 g(n)$ for all $n > n_3$

which is the definition of Θ.

2.4.7 Prove that the running time of an algorithm is $\Theta(g(n))$ if and only if its worst-case running time is $O(g(n))$ and its best-case running time is $\Omega(g(n))$.

If T_w is the worst-case running time and T_b is the best-case running time, we know that:

$0 \leq c_1 g(n) \leq T_b(n)$ for $n > n_b$
$0 \leq T_w(n) \leq c_2 g(n)$ for $n > n_w$

Combining them we get:

$0 \leq c_1 g(n) \leq T_b(n) \leq T_w(n) \leq c_2 g(n)$ for $n > \max(n_b, n_w)$

Since the running time is bound between T_b and T_w and the above is the definition of the Θ-notation, we have our proof.

2.4.8 Prove $o(g(n)) \cap \omega(g(n))$ is the empty set.

Assume $o(g(n)) \cap \omega(g(n)) \neq \phi$; further assume that $f(n) \in o(g(n)) \cap \omega(g(n))$. Thus $f(n) = o(g(n))$ and $f(n) = \omega(g(n))$. It follows from $f(n) = o(g(n))$ that for every constant $c > 0$, there exists a constant $n_0 > 0$ such that $0 \leq f(n) < c.g(n)$. If we let $c = 1$, then there exists a constant $n_0 > 0$ such that for all $n \geq n_0$

$$0 \leq f(n) < g(n) \tag{2.5}$$

Similarly it follows from $f(n) = \omega(g(n))$ that there exists a constant $n_1 > 0$ such that for all $n \geq n_1$

$$0 \leq g(n) < f(n) \tag{2.6}$$

From inequalities (2.5) and (2.6), we have for all $n \geq \max(n_0, n_1)$

$$f(n) < g(n) < f(n)$$

which is impossible. Thus our assumption that $o(g(n)) \cap \omega(g(n)) \neq \phi$ proves to be false and $o(g(n)) \cap \omega(g(n))$ is the empty set.

CST306 : Algorithm analysis and design

Chapter 3

Recurrences

Narasimhan T.

3.1 Introduction

When an algorithm contains a recursive call, its running time is described by a recurrence. A recurrence is an equation or inequality that describes a function in terms of its value on smaller inputs. Solving recurrences is a typical way to obtain asymptotic complexity results for algorithms, i.e. for obtaining the asymptotic bounds on the solution. We discuss three methods for solving recurrences:

1. **Iteration method** : converts the recurrence into a summation and then relies on techniques for bounding summations to solve the recurrence.
2. **Substitution method**: We guess a bound and then use mathematical induction to prove our guess correct.
3. **Master method** : This provides bounds for recurrences of the form $T(n) = aT(\frac{n}{b}) + f(n)$, where $a \geq 1, b > 1$ and $f(n)$ is a function in n.

Some useful results

$$a^{\log_b n} = n^{\log_b a} \tag{R3.1.1}$$

$$\sum_{i=0}^{\infty} x^i = \frac{1}{1-x}, \text{ for } x < 1 \tag{R3.1.2}$$

$$\sum_{i=1}^{n} \frac{1}{i} = \Theta(\lg n) \tag{R3.1.3}$$

$$\sum_{i=1}^{\log n} \frac{1}{i} = \Theta(\log\log n) \tag{R3.1.4}$$

3.2 Iteration method

The core idea here is to expand (iterate) the recurrence and express it as a summation of terms dependent only on n and the initial conditions. Techniques for evaluating summations can then be used to provide bounds on the solution. This method focuses on two parameters : the number of times the recurrence needs to be iterated to reach the boundary condition, and the sum of the terms arising from each level of the iteration process.

3.2.1 Problems and Solutions

3.2.1 $T(n) = T(n-1) + 1$

$$\begin{aligned} T(n) &= T(n-1)+1 \\ &= T(n-2)+1+1 \\ &= T(n-2)+2 \\ &= T(n-3)+1+2 \\ &= T(n-3)+3 \\ &\vdots \\ &= T(n-k)+k \end{aligned} \tag{3.2.1.1}$$

How far must we iterate the recurrence before we reach a boundary condition? We stop at $T(1)$ because at this point there is no more recursion, we just return a constant number for the amount of work to do. Thus we stop when $n-k=1 \implies k=n-1$. Putting $k=n-1$ in (3.2.1.1) yields

$$\begin{aligned} T(n) &= T(1)+n-1 \\ &= c+n-1 \text{[Assuming } T(1)=c, \text{ a constant]} \\ &= O(n). \end{aligned}$$

3.2.2 $T(n) = T(n-1) + n$

$$\begin{aligned} T(n) &= T(n-1)+n \\ &= T(n-2)+(n-1)+n \\ &= T(n-3)+(n-2)+(n-1)+n \\ &\vdots \\ &= T(n-(k+1))+(n-k)+\ldots\ldots\ldots+(n-1)+n \end{aligned} \tag{3.2.1.2}$$

We stop the recursion when $(n-(k+1))=1 \implies k=n-2$. Putting $k=n-2$ in (3.2.1.2) yields

$$\begin{aligned} T(n) &= T(1)+2+3+\ldots\ldots+(n-1)+n \\ &= c-1+1+2+3+\ldots\ldots+(n-1)+n \\ &= c-1+\frac{n(n+1)}{2} \\ &= c-1+\frac{n^2+n}{2} \\ &= O(n^2). \end{aligned}$$

3.2.3 $T(n) = T(n-1) + \log n$

$$\begin{aligned}
T(n) &= T(n-1) + \log n \\
&= T(n-2) + \log(n-1) + \log n \\
&= T(n-3) + \log(n-2) + \log(n-1) + \log n \\
&\vdots \\
&= T(n-(k+1)) + \log(n-k) + \ldots\ldots\ldots + \log(n-1) + \log n
\end{aligned} \tag{3.2.1.3}$$

We stop the recursion when $(n-(k+1)) = 1 \implies k = n-2$. Putting $k = n-2$ in (3.2.1.3) yields

$$\begin{aligned}
T(n) &= T(1) + \log 2 + \log 3 + \ldots\ldots + \log(n-1) + \log n \\
&= c + \log 1 + \log 2 + \log 3 + \ldots\ldots + \log(n-1) + \log n \quad [\log 1 = 0] \\
&= c + \log(1.2.3\ldots\ldots n) \\
&= c + \log n! \\
&= c + \log n^n \quad [\because n! \approx n^n] \\
&= c + n\log n \\
&= O(n\log n)
\end{aligned}$$

3.2.4 $T(n) = 3T\left(\frac{n}{4}\right) + n$

$$\begin{aligned}
T(n) &= 3T\left(\frac{n}{4}\right) + n \\
&= 3\left(3T\left(\frac{n}{16}\right) + \frac{n}{4}\right) + n \\
&= 3^2.T\left(\frac{n}{4^2}\right) + \left(\frac{3}{4}\right)n + n \\
&= 3^2\left(3T\left(\frac{n}{4^3}\right) + \frac{n}{4^2}\right) + \left(\frac{3}{4}\right)n + n \\
&= 3^3.T\left(\frac{n}{4^3}\right) + \left(\frac{3}{4}\right)^2 n + \left(\frac{3}{4}\right)n + n \\
&\vdots \\
&= 3^k.T\left(\frac{n}{4^k}\right) + \left(\frac{3}{4}\right)^{(k-1)} n + \ldots\ldots + \left(\frac{3}{4}\right)n + n
\end{aligned} \tag{3.2.1.4}$$

We stop the process when $\frac{n}{4^k} = 1 \implies n = 4^k \implies k = \log_4 n$. Putting $n = 4^k$ in (3.2.1.4) yields

$$\begin{aligned}
T(n) &= 3^k.T(1) + \left(n + \left(\frac{3}{4}\right)n + \left(\frac{3}{4}\right)^2 n + \ldots\ldots\ldots + \left(\frac{3}{4}\right)^{(k-1)} n\right) \\
&= 3^k.c + n\left(1 + \left(\frac{3}{4}\right) + \left(\frac{3}{4}\right)^2 + \ldots\ldots\ldots + \left(\frac{3}{4}\right)^{(k-1)}\right) \\
&= 3^k.c + n\sum_{i=0}^{k-1}\left(\frac{3}{4}\right)^i \\
&\leq 3^k.c + n\sum_{i=0}^{\infty}\left(\frac{3}{4}\right)^i \\
&\leq 3^k.c + n\left(\frac{1}{1-\frac{3}{4}}\right) \quad \text{using (R3.1.2) on page 22} \\
&\leq 3^k.c + 4n \\
&\leq c.3^{\log_4 n} + 4n \\
&\leq c.n^{\log_4 3} + 4n \quad \text{using (R3.1.1) on page 22} \\
&= O(n) \left[\text{ In the previous inequality 4n is the dominant term as } \log_4 3 < 1\right]
\end{aligned}$$

3.2.2 Recursion tree method

A recursion tree is a convenient way to visualize what happens when a recurrence is iterated. Recursion trees give a better intuitive idea than iterating the recurrence of how the recurrence progresses. In a recursion tree, each node represents the cost of a single subproblem at some level in the set of recursive function invocations. Thus each level of the tree represents a level of recursion. We sum the costs at each level to obtain a set of per-level costs, and then we sum all the per-level costs to determine the total cost of all levels of the recursion which is a reasonable estimate of the time complexity. The steps involved in recursion tree method are summarized below:

1. Draw the tree based on the recurrence.
2. From the tree determine:
   (a) number of levels in the tree
   (b) cost per level
   (c) number of nodes in the last level
   (d) cost of the last level (which is based on the value found above in part (c))
3. Write down the summation using Σ notation - this summation sums up the cost of all the levels in the recursion tree.
4. Simplify the summation using standard formulae and represent using asymptotic notations.

3.2.2.1 Problems and Solutions

3.2.2.1 $T(n) = 2T(\frac{n}{2}) + c$

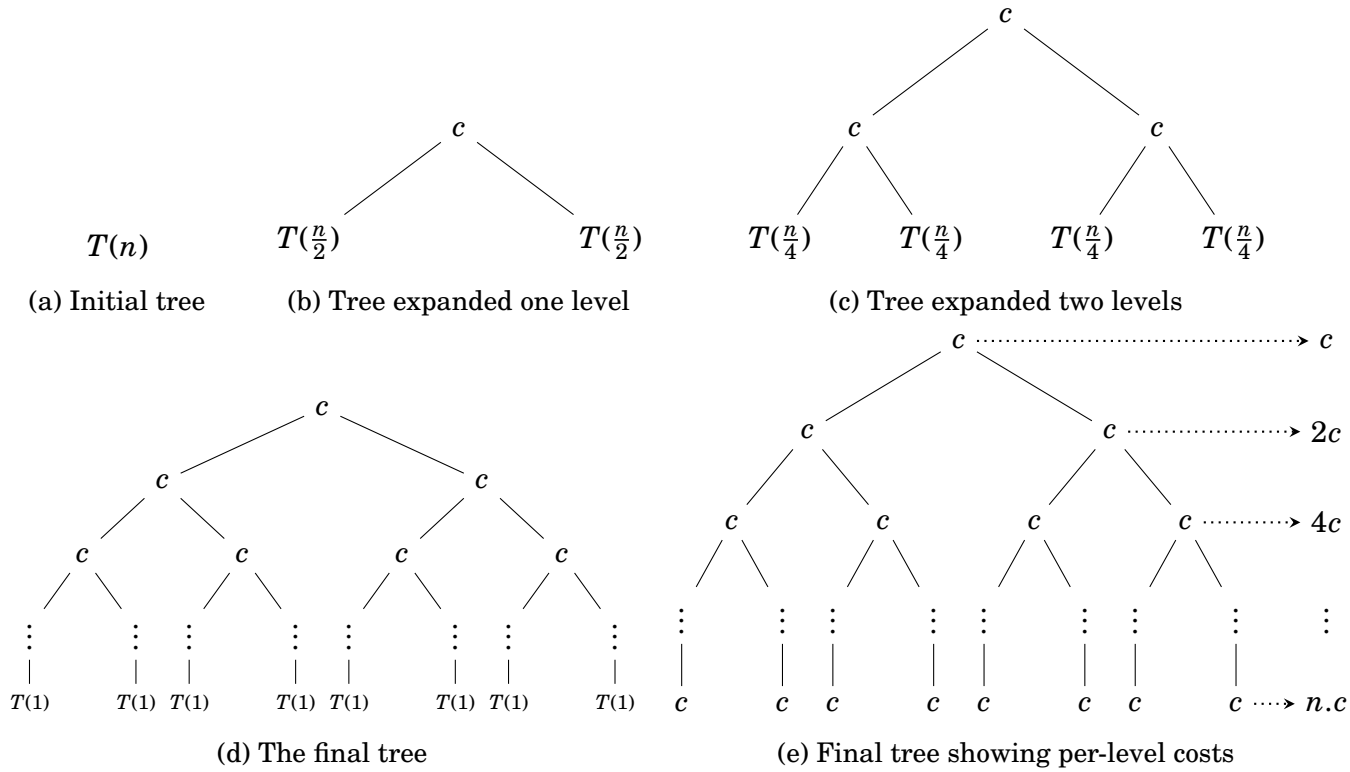
The recurrence can be interpreted as follows: "In order to solve a problem of size n we must solve two subproblems of size $\frac{n}{2}$ and do c amount of additional work". The construction of the recursion tree is shown in Figure 3.1. To begin with, we show at level 1 that we need to solve a problem of size n.

Figure 3.1a shows the initial tree which has just a single node $T(n)$. In 3.1b, the tree is expanded into an equivalent tree representing the recurrence. The constant term c is the root (the cost at the top level of recursion), and the two subtrees of the root are the two smaller recurrences $T(\frac{n}{2})$. Part 3.1c shows this process carried one step further by expanding $T(\frac{n}{2})$. The cost for each of the two nodes at the second level of recursion is c. Each of these nodes has two children representing the recurrence $T(\frac{n}{4})$. We continue expanding each node in the tree by breaking it into its constituent parts as determined by the recurrence, until a boundary condition is reached. Part 3.1d shows the final tree. At the leaf level, we have $T(1)$. Recall that in iteration method, we stopped the process when we reached $T(1)$. Part 3.1e shows the tree, depicting the cost incurred at each level.

At level 1, the root node is $T(n) = T(\frac{n}{1})$ and the cost is c. At the second level, we have $T(\frac{n}{2})$ as the nodes and cost is $2c$. At leaf level (l^{th} level, where l is the number of levels), we have $T(1) = T(\frac{n}{n})$ and the cost is $n.c$. From the tree, we find that the problem size shrinks as $n, \frac{n}{2}, \frac{n}{4}.....\frac{n}{2^{l-1}}$. But problem size at leaf level is 1 (We stop the recursion when problem size is 1). Thus $\frac{n}{2^{l-1}} = 1 \implies n = 2^{l-1}$.

The total cost is therefore

$$T(n) = c + 2c + 4c + ... + nc$$
$$= c\sum_{0}^{l-1} 2^i = c.\left(1.\frac{2^l - 1}{2-1}\right) = c.(2.2^{l-1} - 1) = c.(2n-1) \implies T(n) = O(n)$$



(a) Initial tree (b) Tree expanded one level (c) Tree expanded two levels

(d) The final tree (e) Final tree showing per-level costs

Figure 3.1: Constructing recursion tree for the recurrence $T(n) = 2T(\frac{n}{2}) + c$

3.2.2.2 $T(n) = \begin{cases} 2T(\frac{n}{2}) + n & \text{if } n > 1 \\ 1 & \text{if } n \leq 1 \end{cases}$

The recursion tree is shown in Figure 3.2. The figure also shows the per-level costs. From the tree, we

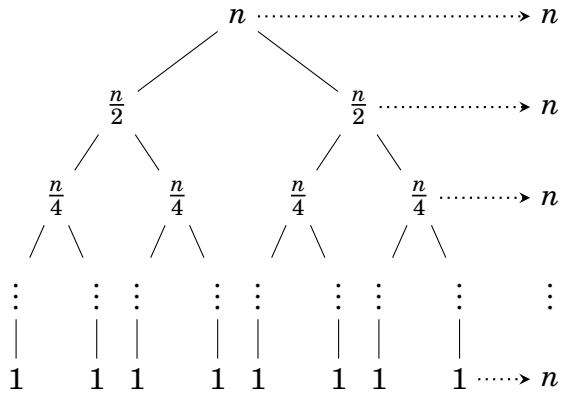


Figure 3.2: Recursion tree for the recurrence $T(n) = 2T(\frac{n}{2}) + n$

find that the problem size shrinks as $n, \frac{n}{2}, \frac{n}{4}.....\frac{n}{2^{l-1}}$ (l is the number of levels). Since, the problem size at leaf level is 1, we have $\frac{n}{2^{l-1}} = 1 \implies n = 2^{l-1} \implies 2n = 2^l \implies l = \lg 2n$. The total per-level cost is *number of levels* $\times$ n. Thus $T(n) = n.l = n \lg 2n = O(n \lg n)$.

3.2.2.3 $T(n) = 2T(\frac{n}{2}) + n^2$

The recursion tree is shown in Figure 3.3. From the tree, we find that the problem size shrinks as

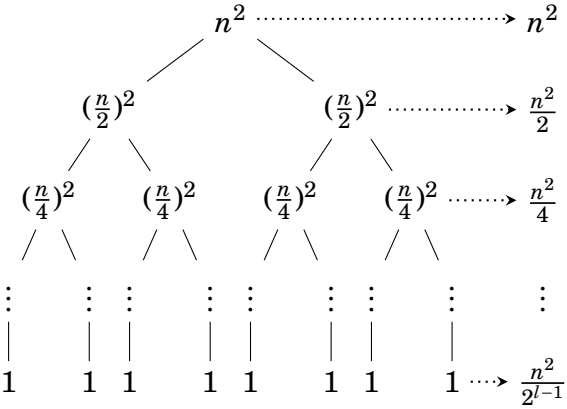


Figure 3.3: Recursion tree for the recurrence $T(n) = 2T(\frac{n}{2}) + n^2$

$n, \frac{n}{2}, \frac{n}{4}.....\frac{n}{2^{l-1}}$ (l is the number of levels). Since,the problem size at leaf level is 1, we have $\frac{n}{2^{l-1}} = 1 \implies n = 2^{l-1} \implies 2n = 2^l \implies l = \lg 2n$. The total per-level cost is

$$\begin{aligned} T(n) &= n^2 + \frac{n^2}{2} + \frac{n^2}{4} + ... + \frac{n^2}{2^{l-1}} \\ &= n^2.\sum_{i=0}^{l-1} \frac{1}{2^i} \\ &\leq n^2.\sum_{i=0}^{\infty} \left(\frac{1}{2}\right)^i \\ &= n^2.\left(\frac{1}{1-\frac{1}{2}}\right) \quad \text{using (R3.1.2) on page 22} \end{aligned}$$

Thus $T(n) \leq 2n^2 \implies T(n) = O(n^2)$.

3.2.2.4 Solve $T(n) = T(\frac{n}{4}) + T(\frac{n}{2}) + n^2$ using recursion tree method.

The recursion tree is shown in Figure 3.4.

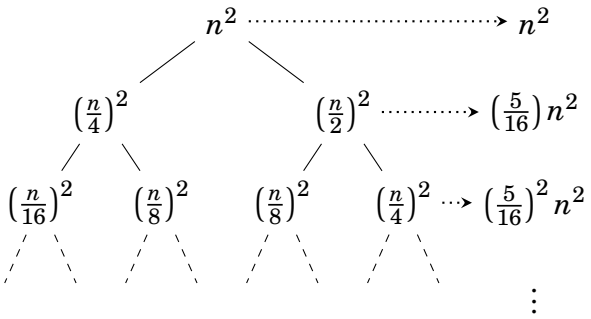


Figure 3.4: Recursion tree for the recurrence $T(n) = T(\frac{n}{2}) + T(\frac{n}{4}) + n^2$

The total per-level cost is

$$\begin{aligned} T(n) &= n^2 + \left(\frac{5}{16}\right)n^2 + \left(\frac{5}{16}\right)^2 n^2 + \ldots\ldots + \left(\frac{5}{16}\right)^{l-1} n^2 \\ &= n^2 . \sum_{i=0}^{l-1} \left(\frac{5}{16}\right)^i \\ &\leq n^2 . \sum_{i=0}^{\infty} \left(\frac{5}{16}\right)^i \\ &= n^2 . \left(\frac{1}{1-\frac{5}{16}}\right) \quad \text{using (R3.1.2) on page 22} \end{aligned}$$

Thus $T(n) \leq \frac{16}{11} . n^2 \implies T(n) = O(n^2)$.

3.3 Substitution method

The substitution method for solving recurrences involves two steps:

1. Guess the form of the solution.
2. Use mathematical induction to find the constants and show that the solution works.

This method can be applied only in the cases when it is easy to guess the form of the solution. Making a good guess is really a challenging task as there is no general way to guess the correct solution to recurrences. Often a recursion tree is used as a precursory tool to get a feeling of the growth of the recurrence which helps in making a guess.

3.3.1 Problems and Solutions

3.3.1 $T(n) = 2T(\lfloor \frac{n}{2} \rfloor) + n$

We make a guess that $T(n) = O(n \lg n)$. We need to prove that $T(n) \leq cn \lg n$ for some constant $c > 0$. We can choose c large enough to make this bound true for the base case. Now assume, by induction, that

this bound holds for all positive values of $m < n$. Then we have

$$\begin{aligned}
T(n) &= 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + n \\
&\leq 2\left(c\left\lfloor \frac{n}{2} \right\rfloor \lg\left(\left\lfloor \frac{n}{2} \right\rfloor\right)\right) + n \quad \left[\frac{n}{2} < n. \text{ So the induction hypothesis holds true. } \right] \\
&\leq cn\lg\left(\frac{n}{2}\right) + n \\
&= cn\lg n - cn\lg 2 + n \\
&= cn\lg n - cn + n \quad [\lg 2 = 1] \\
&= cn\lg n - (c-1)n \\
&\leq cn\lg n \quad \forall c \geq 1
\end{aligned}$$

3.3.2 $T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 1$

We guess that the solution is $T(n) = O(n)$, and we try to show that $T(n) \leq cn$ for an appropriate choice of the constant c such that the base condition holds. Substituting our guess in the recurrence, we obtain

$$\begin{aligned}
T(n) &= T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + 1 \\
&\leq c.\left\lfloor \frac{n}{2} \right\rfloor + c.\left\lceil \frac{n}{2} \right\rceil + 1 \\
&\leq c.\left(\frac{n}{2}\right) + c.\left(\frac{n}{2}\right) + 1 \\
&= cn + 1 \not\Rightarrow T(n) \leq cn \text{ for any choice of } n
\end{aligned}$$

But our guess of $T(n) \leq cn$ is indeed correct [You may verify this using recursion tree]. To prove this, we proceed in a different manner. Note that we got $T(n) \leq cn + 1$ and which is away from our guess only by the constant 1, a lower-order term. So we generate a new guess by subtracting a lower-order term from our previous guess. Our new guess is thus $T(n) \leq cn - d$, where $d \geq 0$ is a constant. We now have

$$\begin{aligned}
T(n) &= T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + 1 \\
&\leq \left(c.\left\lfloor \frac{n}{2} \right\rfloor - d\right) + \left(c.\left\lceil \frac{n}{2} \right\rceil - d\right) + 1 \\
&\leq \left(c.\left(\frac{n}{2}\right) - d\right) + \left(c.\left(\frac{n}{2}\right) - d\right) + 1 \\
&= cn - 2d + 1 \\
&\leq cn - d \quad \forall d \geq 1
\end{aligned}$$

Thus $T(n) \leq cn - d \implies T(n) = O(n)$

3.3.3 $T(n) = 2T(\frac{n}{2}) + bn\lg n$

We guess that $T(n) = O(n\lg^2 n)$. For this, we need to prove that $T(n) \leq cn\lg^2 n$ for some constant $c > 0$. We can choose c large enough to make this bound true for the base case. Now assume, by induction,

that this bound holds for all positive values of $m < n$. Then we have

$$\begin{aligned}
T(n) &= 2T\left(\frac{n}{2}\right) + bn\lg n \\
&\leq 2\left(c.\left(\frac{n}{2}\right)\lg^2\left(\frac{n}{2}\right)\right) + bn\lg n \\
&= 2\left(c.\left(\frac{n}{2}\right)(\lg n - \lg 2)^2\right) + bn\lg n \\
&= cn\left(\lg^2 n - 2\lg n + 1\right) + bn\lg n \\
&= cn\lg^2 n - 2cn\lg n + cn + bn\lg n \\
&\leq cn\lg^2 n \text{ , provided } c \geq \frac{b}{2}
\end{aligned}$$

3.3.4 $T(n) = 2T(\frac{n}{2}) + \lg n$

We guess that the solution is $T(n) = O(n)$, and we try to show that $T(n) \leq cn$ for an appropriate choice of the constant c such that the base condition holds. Substituting our guess in the recurrence, we obtain

$$\begin{aligned}
T(n) &= 2T\left(\frac{n}{2}\right) + \lg n \\
&\leq 2c\left(\frac{n}{2}\right) + \lg n \\
&= cn + \lg n \implies T(n) \leq cn \text{ for any choice of } n
\end{aligned}$$

But $T(n) = O(n)$ is indeed correct. To prove this, we make a new guess, $T(n) \leq c(n - \lg n)$. We then have

$$\begin{aligned}
T(n) &= 2T\left(\frac{n}{2}\right) + \lg n \\
&\leq 2c\left(\left(\frac{n}{2}\right) - \lg\left(\frac{n}{2}\right)\right) + \lg n \\
&= cn - 2c\lg n + 2c\lg 2 + \lg n \\
&= cn - c\lg n - c\lg n + 2c + \lg n \\
&= c(n - \lg n) - c\lg n + 2c + \lg n \\
&\leq c(n - \lg n) \quad \forall c \geq 3
\end{aligned}$$

3.4 Master method

The master method provides a "cookbook"method for solving recurrences of the form:

$$T(n) = a.T\left(\frac{n}{b}\right) + f(n) \tag{3.4.1}$$

where $a \geq 1$ and $b > 1$ are constants and $f(n)$ is an asymptotically positive function.

The recurrence 3.4.1 describes the running time of an algorithm that divides a problem of size n into a subproblems, each of size $\frac{n}{b}$, where a and b are positive constants. The a subproblems are solved recursively, each in time $T\left(\frac{n}{b}\right)$. The function $f(n)$ denotes the cost of dividing the problem and combining the results of the subproblems.

$T(n)$ then has the following asymptotic bounds:

Case 1 : If $f(n) = O\left(n^{\log_b a - \epsilon}\right)$ for some constant $\epsilon > 0$, then $T(n) = \Theta\left(n^{\log_b a}\right)$

Case 2 : If $f(n) = \Theta\left(n^{\log_b a}\right)$, then $T(n) = \Theta\left(n^{\log_b a} \lg n\right)$

Case 3 : If $f(n) = \Omega\left(n^{\log_b a + \epsilon}\right)$ for some constant $\epsilon > 0$, and if $a.f\left(\frac{n}{b}\right) \leq c.f(n)$ for some constant $c < 1$ and for all sufficiently large n ,then $T(n) = \Theta(f(n))$

In each of the three cases, we compare the function $f(n)$ with the function $n^{\log_b a}$. Intuitively, the larger of these two functions determines the solution to the recurrence. If, as in case 1, the function $n^{\log_b a}$ is larger, then the solution is $T(n) = \Theta(n^{\log_b a})$. If, as in case 3, the function $f(n)$ is larger, then the solution is $T(n) = \Theta(f(n))$. If, as in case 2, the two functions are of the same size, we multiply by a logarithmic factor, and the solution is $T(n) = \Theta(n^{\log_b a} \lg n)$. It should be noted that in the first case, not only must $f(n)$ be smaller than $n^{\log_b a}$, it must be polynomially smaller. That is, $f(n)$ must be asymptotically smaller than $n^{\log_b a}$ by a factor of n^{ϵ} for some constant $\epsilon > 0$. Similarly in the third case, not only must $f(n)$ be larger than $n^{\log_b a}$, it also must be polynomially larger and in addition satisfy the condition $a.f(\frac{n}{b}) \leq c.f(n)$ which is called "regularity condition".

To find which among the two functions $f(n)$ and $g(n)$ is polynomially smaller (or larger), we extract the polynomial parts from $f(n)$ and $g(n)$ and then compare them. For example, to compare n^3 and $n \lg n$, we extract the polynomial parts i.e., n^3 and n and conclude that n^3 is polynomially larger than $n \lg n$ by a factor of n^2. But with the functions n and $n \lg n$, neither of the two is polynomially smaller/larger than the other as both have the same polynomial part n.

3.4.1 Problems and Solutions

3.4.1 $T(n) = 9T(\frac{n}{3}) + n$
Here $a = 9, b = 3, f(n) = n$
$n^{\log_b a} = n^{\log_3 9} = n^2$
Comparing $n^{\log_b a}$ and $f(n)$, we find that $f(n) = O(n) = O(n^{\log_3 9 - \epsilon})$, with $\epsilon = 1$. Also $n^{\log_b a} = n^2$ is polynomially larger than $f(n) = n$ by a factor of n. Hence, by case 1 of master theorem we have $T(n) = \Theta(n^{\log_b a}) = \Theta(n^2)$

3.4.2 $T(n) = T(\frac{2n}{3}) + 1$
Here $a = 1, b = \frac{3}{2} = 1.5, f(n) = 1$
$n^{\log_b a} = n^{\log_{1.5} 1} = n^0 = 1$
Comparing $n^{\log_b a}$ and $f(n)$, we find that $f(n) = \Theta(1) = \Theta(n^{\log_{1.5} 1})$. Hence, by case 2 of master theorem we have $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(\lg n)$

3.4.3 $T(n) = 2T(\frac{n}{2}) + n$
Here $a = 2, b = 2, f(n) = n$
$n^{\log_b a} = n^{\log_2 2} = n$
Comparing $n^{\log_b a}$ and $f(n)$, we find that $f(n) = \Theta(n) = \Theta(n^{\log_2 2})$. Hence, by case 2 of master theorem we have $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(n \lg n)$

3.4.4 $T(n) = 3T(\frac{n}{4}) + n \lg n$
Here $a = 3, b = 4, f(n) = n \lg n$
$n^{\log_b a} = n^{\log_4 3} = n^{0.792}$
Comparing $n^{\log_b a}$ and $f(n)$, we find that $f(n) = \Omega(n) = \Omega(n^{\log_4 3 + \epsilon})$, with $\epsilon = 0.208$. Also $f(n) = n \lg n$ is polynomially larger than $n^{\log_b a} = n^{0.792}$ by a factor of $n^{0.208}$. These facts take us to case 3. Checking the regularity condition, $3.\frac{n}{4} \lg \frac{n}{4} \leq c.n \lg n \implies \frac{3}{4} n(\lg n - \lg 4) \leq c.n \lg n$. Since $\frac{3}{4} n(\lg n - \lg 4) \leq \frac{3}{4} n \lg n, c = \frac{3}{4}, < 1$. Thus regularity condition also holds. Hence, by case 3 of master theorem we have $T(n) = \Theta(f(n)) = \Theta(n \lg n)$

3.4.5 $T(n) = 2T(\frac{n}{2}) + n \lg n$

Here $a = 2, b = 2, f(n) = n \lg n$
$n^{\log_b a} = n^{\log_2 2} = n$
But $f(n) = n \lg n$ is not polynomially larger or smaller than $n^{\log_b a} = n$ since both have the same polynomial part. Thus cases 1 and 3 of master theorem cannot be applied here. Finding that $f(n) \neq \Theta(n^{\log_b a})$, case 2 is also not satisfied. Thus master theorem cannot be used to solve this problem. Recursion tree method can be used instead.

3.4.6 $T(n) = 2T(\frac{n}{2}) + \Theta(n)$
$a = 2, b = 2, f(n) = \Theta(n) = \Theta(n^{\log_b a}) \implies T(n) = \Theta(n \lg n)$ [Case 2 applies].

3.4.7 $T(n) = 8T(\frac{n}{2}) + \Theta(n^2)$
$a = 8, b = 2, f(n) = \Theta(n^2)$ is polynomially smaller than $n^{\log_b a} = n^3$ by a factor of n.
$\therefore T(n) = \Theta(n^3)$ [Case 1].

3.4.8 $T(n) = 7T(\frac{n}{2}) + \Theta(n^2)$
$a = 7, b = 2, f(n) = \Theta(n^2)$ is polynomially smaller than $n^{\log_b a} = n^{\log_2 7} = n^{2.807}$ by a factor of $n^{0.807}$.
$\therefore T(n) = \Theta(n^{\log_2 7})$ [Case 1].

3.5 More Solved problems

3.5.1 $T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \lg n$

We can take $\lfloor \sqrt{n} \rfloor$ as $\sqrt{n}$ as rounding off values does not cause any issue in asymptotic notation. We solve this problem by change of variables technique. Take $m = \lg n \implies n = 2^m$. Then the recurrence becomes

$T(2^m) = 2T(2^{\frac{m}{2}}) + m$

Renaming $T(2^m)$ as $S(m)$, we get the recurrence

$S(m) = 2S(\frac{m}{2}) + m.$

Using a recursion tree or substitution method, we get $S(m) = O(m \lg m)$. Changing back from $S(m)$ to $T(n)$, we obtain $T(n) = T(2^m) = S(m) = O(m \lg m) = O(\lg n \lg \lg n)$.

3.5.2 Prove that $T(n) = 2T(\lfloor \frac{n}{2} \rfloor) + n$ is $\Theta(n \lg n)$ using substitution method.

We need to prove that $T(n) = O(n \lg n)$ as well as $T(n) = \Omega(n \lg n)$. We had already proved that $T(n) = O(n \lg n)$ in problem (3.1) on page 28. So we what remains is to prove that $T(n) = \Omega(n \lg n)$. For this we will have to prove that $T(n) \geq cn \lg n$ for some constant $c > 0$. We can choose c large enough to make this bound true for the base case. Now assume, by induction, that this bound holds for all positive values of $m < n$. Then we have

$$
\begin{aligned}
T(n) &= 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + n \\
&\geq 2\left(c\left\lfloor \frac{n}{2} \right\rfloor \lg\left(\left\lfloor \frac{n}{2} \right\rfloor\right)\right) + n \quad \left[\frac{n}{2} < n. \text{ So the induction hypothesis holds true.}\right] \\
&\geq cn \lg\left(\frac{n}{2}\right) + n \\
&= cn \lg n - cn \lg 2 + n \\
&= cn \lg n - cn + n \\
&= cn \lg n - (c-1)n \\
&= cn \lg n + (1-c)n \\
&\geq cn \lg n \quad \forall\, c < 1
\end{aligned}
$$

3.5.3 Show that the solution of $T(n) = T(n-1) + n$ is $O(n^2)$ using substitution method.

We need to show that $T(n) \leq cn^2$ for some constant $c > 0$. We have

$$\begin{aligned} T(n) &= T(n-1) + n \\ &\leq c(n-1)^2 + n \\ &= cn^2 - 2cn + c + n \\ &= cn^2 + c(1-2n) + n \\ &\leq cn^2 \text{ if } c(1-2n) + n \leq 0 \implies c \geq \frac{n}{2n-1}, \text{ true for all } n \geq 1, c \geq 1. \end{aligned}$$

3.5.4 Show that the solution of $T(n) = 8T(\frac{n}{2}) + \Theta(n^2)$ is $O(n^3)$ using substitution method.

We need to show that $T(n) \leq cn^3$ for some constant $c > 0$. We have

$$\begin{aligned} T(n) &= 8T\left(\frac{n}{2}\right) + \Theta(n^2) \\ &= 8T\left(\frac{n}{2}\right) + dn^2 \\ &\leq 8c\left(\frac{n}{2}\right)^3 + dn^2 \\ &= cn^3 + dn^2 \\ &\not\leq cn^3. \end{aligned}$$

To prove that $T(n)$ is $O(n^3)$, we subtract $d'n^2$ (where d' is a constant) from our original guess. Thus our new guess is $T(n) \leq cn^3 - d'n^2$. We have

$$\begin{aligned} T(n) &= 8T\left(\frac{n}{2}\right) + \Theta(n^2) \\ &= 8T\left(\frac{n}{2}\right) + dn^2 \\ &\leq 8c\left(\frac{n}{2}\right)^3 - 8d'\left(\frac{n}{2}\right)^2 + dn^2 \\ &= cn^3 - 2d'n^2 + dn^2 \\ &= cn^3 - d'n^2 - \left(d'n^2 - dn^2\right) \\ &\leq cn^3 - d'n^2 \text{ if } d'n^2 - dn^2 \geq 0 \text{ i.e. if } d' \geq d \end{aligned}$$

3.5.5 Show that the solution of $T(n) = T(\lceil \frac{n}{2} \rceil) + 1$ is $O(\lg n)$ using substitution method.

We must show that $T(n) \leq c \lg n$ for some constant $c > 0$. We can choose c large enough to make this bound true for the base case. Now assume, by induction, that this bound holds for all positive values of $m < n$. Then we have

$$\begin{aligned} T(n) &= T\left(\left\lceil \frac{n}{2} \right\rceil\right) + 1 \\ &\leq c \lg\left(\frac{n}{2}\right) + 1 \\ &= c(\lg n - \lg 2) + 1 \\ &= c \lg n - c + 1 \\ &\leq c \lg n \text{ if } c \geq 1. \end{aligned}$$

Thus $T(n) \leq c \lg n \implies T(n) = O(\lg n)$.

3.5.6 $T(n) = 2T(\sqrt{n}) + 1$

We solve by change of variables technique. Take $m = \lg n \implies n = 2^m$. Then the recurrence becomes

$T(2^m) = 2T(2^{\frac{m}{2}}) + 1$

Renaming $T(2^m)$ as $S(m)$, we get the recurrence

$S(m) = 2S(\frac{m}{2}) + 1.$

Using master method, we get $S(m) = \Theta(m)$. Changing back from $S(m)$ to $T(n)$, we obtain $T(n) = T(2^m) = S(m) = \Theta(m) = \Theta(\lg n)$.

3.5.7 Prove that $T(n) = 2T(\lfloor \frac{n}{2} \rfloor + 17) + n$ is $O(n \lg n)$

We guess $T(n) \le c(n-a)\lg(n-a)$:

$$\begin{aligned}
T(n) &\le 2c\left(\left\lfloor \frac{n}{2} \right\rfloor + 17 - a\right)\lg\left(\left\lfloor \frac{n}{2} \right\rfloor + 17 - a\right) + n \\
&\le 2c\left(\frac{n}{2} + 17 - a\right)\lg\left(\frac{n}{2} + 17 - a\right) + n \\
&= c(n + 34 - 2a)\lg\left(\frac{n + 34 - 2a}{2}\right) + n \\
&= c(n + 34 - 2a)\lg(n + 34 - 2a) - c(n + 34 - 2a) + n \\
&\le c(n + 34 - 2a)\lg(n + 34 - 2a) && (c > 1, n > n_0 = f(a)) \\
&\le c(n-a)\lg(n-a) \quad \forall a > 17
\end{aligned}$$

3.5.8 Prove that $T(n) = 4T(\frac{n}{2}) + n$ is $O(n^2)$ using substitution method.

We need to show that $T(n) \le cn^2$ for some constant $c > 0$. We can choose c large enough to make this bound true for the base case. Now assume, by induction, that this bound holds for all positive values of $m < n$. Then we have

$$\begin{aligned}
T(n) &= 4T\left(\frac{n}{2}\right) + n \\
&\le 4c\left(\frac{n}{2}\right)^2 + n \\
&= cn^2 + n \\
&\not\le cn^2 \text{ for any value of } c > 0.
\end{aligned}$$

To prove that $T(n)$ is $O(n^2)$, we subtract dn (where d is a constant) from our original guess. Thus our new guess is $T(n) \le cn^2 - dn$. We have

$$\begin{aligned}
T(n) &= 4T\left(\frac{n}{2}\right) + n \\
&\le 4\left(c\left(\frac{n}{2}\right)^2 - d\left(\frac{n}{2}\right)\right) + n \\
&= cn^2 - 2dn + n \\
&= cn^2 - dn - dn + n \\
&= cn^2 - dn - (d-1)n \\
&\le cn^2 - dn, \text{if } d \ge 1
\end{aligned}$$

3.5.9 $T(n) = 3T(\sqrt{n}) + \log n$

We solve by change of variables technique. Take $m = \lg n \implies n = 2^m$. Then the recurrence becomes

$T(2^m) = 3T(2^{\frac{m}{2}}) + m$

Renaming $T(2^m)$ as $S(m)$, we get the recurrence

$S(m) = 2S(\frac{m}{2}) + m.$

Using master method, we get $S(m) = \Theta(m^{\log_2 3})$. Changing back from $S(m)$ to $T(n)$, we obtain $T(n) = T(2^m) = S(m) = \Theta(m^{\log_2 3}) = \Theta((\lg n)^{\log_2 3})$. i.e. $T(n) = \Theta(\lg^{\lg 3} n)$

3.5.10 The solution to the recurrence is $T(n) = 4T(\frac{n}{3}) + n$ is $T(n) = O(n^{\log_3 4})$. Show that a substitution proof with the assumption $T(n) \leq cn^{\log_3 4}$ fails. Then show how to subtract off a lower-order term to make the substitution proof work.

First we guess $T(n) \leq cn^{\log_3 4}$. Thus:

$$\begin{aligned} T(n) &= 4T\left(\frac{n}{3}\right) + n \\ &\leq 4c\left(\frac{n}{3}\right)^{\log_3 4} + n \\ &= cn^{\log_3 4} + n, \text{ since } 3^{\log_3 4} = 4^{\log_3 3} = 4 \\ &\not\leq cn^{\log_3 4} \end{aligned}$$

Let's guess $T(n) \leq cn^{\log_3 4} - dn$. Thus:

$$\begin{aligned} T(n) &\leq 4\left(c\left(\frac{n}{3}\right)^{\log_3 4} - dn\right) + n \\ &= cn^{\log_3 4} - 4dn + n \\ &= cn^{\log_3 4} - dn - (3dn - n) \\ &\leq cn^{\log_3 4} - dn, \text{ if } 3dn - n \geq 0, \text{ i.e., if } d \geq \frac{1}{3} \end{aligned}$$

3.5.11 Use a recursion tree to determine a good asymptotic upper bound on the recurrence $T(n) = 3T(\lfloor \frac{n}{2} \rfloor) + n$. Use the substitution method to verify your answer.

For making a guess, we can omit the floor and write $T(n) = 3T(\frac{n}{2}) + n$ whose recursion tree is shown in Figure 3.5. The figure also shows the per-level costs. From the tree, we find that the problem size shrinks as $n, \frac{n}{2}, \frac{n}{4}, \ldots\ldots \frac{n}{2^{l-1}}$ (l is the number of levels). Since, the problem size at leaf level is 1, we have $\frac{n}{2^{l-1}} = 1 \implies n = 2^{l-1} \implies 2n = 2^l \implies l = \log_2 2n$, i.e.,$l = \lg n + 1$.

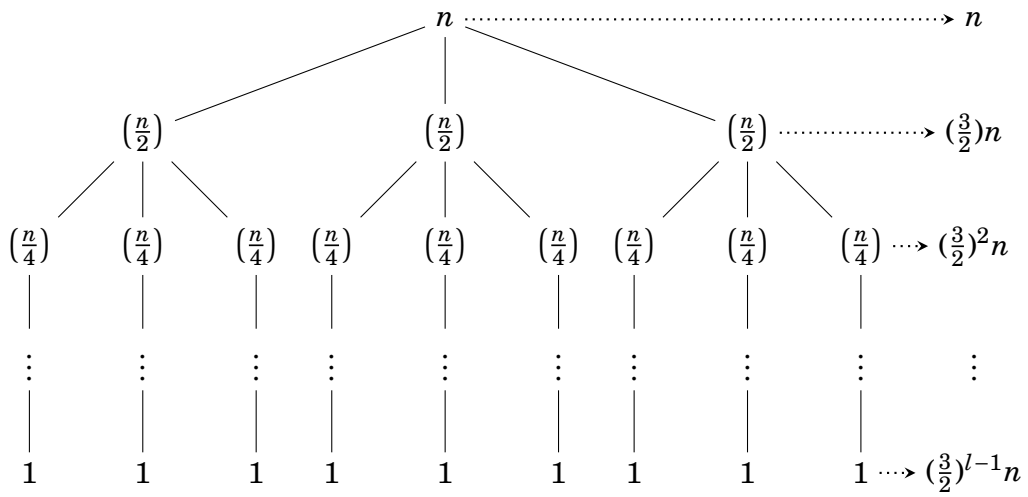


Figure 3.5: Recursion tree for the recurrence $T(n) = 3T\left(\frac{n}{2}\right) + n$

$$
\begin{aligned}
\text{Total per-level cost } &= n + \left(\frac{3}{2}\right)n + \left(\frac{3}{2}\right)^2 n + \ldots\ldots + \left(\frac{3}{2}\right)^{l-1} n \\
&= n.\sum_{i=0}^{l-1}\left(\frac{3}{2}\right)^i \\
&= n.\sum_{i=0}^{\lg n}\left(\frac{3}{2}\right)^i \\
&= n.1.\left(\frac{\left(\frac{3}{2}\right)^{\lg n+1}-1}{\frac{3}{2}-1}\right) \\
&= 2n.\left(\left(\frac{3}{2}\right)^{\lg n+1}-1\right) \\
&= 2n.\left(\frac{3}{2}.\left(\frac{3}{2}\right)^{\lg n}-1\right) \\
&= 2n.\left(\frac{3}{2}n^{\lg\frac{3}{2}}-1\right) \\
&= 3n^{1+\lg\frac{3}{2}} - 2n = 3n^{\lg 3} - 2n
\end{aligned}
$$

Thus $T(n) = 3n^{\lg 3} - 2n \implies T(n) = O(n^{\lg 3})$.

So we need to prove that $T(n) \leq cn^{\lg 3}$. But instead we prove $T(n) \leq cn^{\lg 3} - dn$. Recall that we got

$T(n) = 3n^{\lg 3} - 2n$. We have

$$\begin{aligned}
T(n) &= 3T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + n \\
&\leq 3\left(c\left(\left\lfloor \frac{n}{2} \right\rfloor\right)^{\lg 3} - d\left(\frac{n}{2}\right)\right) + n \\
&\leq \left(\frac{3}{2^{\lg 3}}.cn^{\lg 3} - \frac{3}{2}.dn\right) + n \\
&= cn^{\lg 3} - dn - \left(\frac{dn}{2} - n\right) \quad \text{since } \left[\frac{3}{2^{\lg 3}} = \frac{3}{3^{\lg 2}} = 1\right] \\
&\leq cn^{\lg 3} - dn, \text{ if } \left(\frac{dn}{2} - n\right) \geq 0, \text{ i.e., if } d \geq 2
\end{aligned}$$

3.5.12 Use a recursion tree to determine a good asymptotic upper bound on the recurrence $T(n) = T(\frac{n}{2}) + n^2$. Use the substitution method to verify your answer.

The recursion tree is shown in Figure 3.6. Since there is only one node at each level, the nodes also represents the per-level costs.

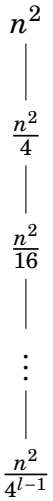


Figure 3.6: Recursion tree for the recurrence $T(n) = T(\frac{n}{2}) + n^2$

From the tree, we find that the problem size shrinks as $n, \frac{n}{2}, \frac{n}{4}, \ldots\ldots \frac{n}{2^{l-1}}$ (l is the number of levels). Since,the problem size at leaf level is 1, we have $\frac{n}{2^{l-1}} = 1 \implies n = 2^{l-1} \implies 2n = 2^l \implies l = \log_2 2n$, i.e.,$l = \lg n + 1$.

The total per-level cost is

$$\begin{aligned}
T(n) &= n^2 + \frac{n^2}{4} + \ldots\ldots + \frac{n^2}{4^{l-1}} \\
&= n^2.\sum_{i=0}^{l-1} \frac{1}{4^i} \\
&\leq n^2.\sum_{i=0}^{\infty} \left(\frac{1}{4}\right)^i \\
&= n^2.\left(\frac{1}{1 - \frac{1}{4}}\right) \quad \text{using (R3.1.2) on page 22}
\end{aligned}$$

Thus $T(n) \leq 4n^2 \implies T(n) = O(n^2)$.

We guess $T(n) \leq cn^2$

$$\begin{aligned}
T(n) &= T\left(\frac{n}{2}\right) + n^2 \\
&\leq c\left(\frac{n}{2}\right)^2 + n^2 \\
&= \frac{cn^2}{4} + n^2 \\
&= cn^2 - \frac{3cn^2}{4} + n^2 \\
&= cn^2 - \left(\frac{3c}{4} - 1\right).n^2 \\
&\leq cn^2 \text{ if } \left(\frac{3c}{4} - 1\right) \geq 0, \text{ i.e., if } c \geq \frac{4}{3}.
\end{aligned}$$

3.5.13 Use a reccursion tree to determine a good asymptotic upper bound on the recurrence $T(n) = 2T(n-1)+1$. Use the substitution method to verify your answer.

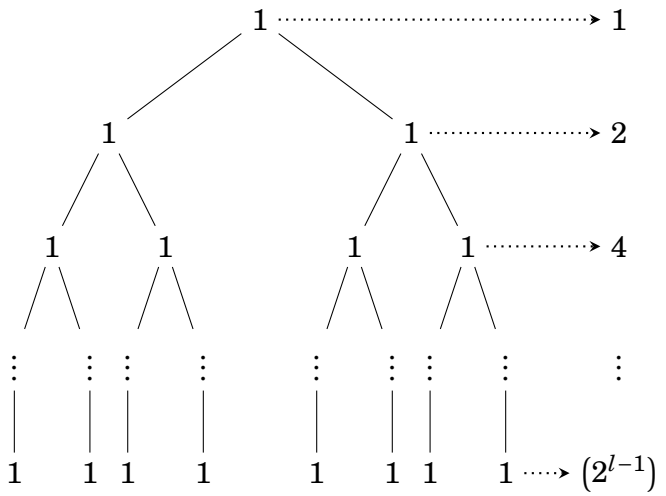


Figure 3.7: Recursion tree for the recurrence $T(n) = 2T(n-1) + 1$

The recursion tree is shown in Figure 3.7. The figure also shows the per-level costs.

From the tree, we find that the problem size decreases linearly as $n, n-1, n-2 \ldots\ldots n-(l-1)$ (l is the number of levels). Since,the problem size at leaf level is 1, we have $n-(l-1) = 1 \implies n = l$

The total per level cost is

$$\begin{aligned}
T(n) &= 1 + 2 + 4 + \ldots\ldots 2^{l-1} \\
&= \sum_{i=0}^{l-1} 2^i \\
&= \sum_{i=0}^{n-1} 2^i \\
&= \frac{2^n - 1}{2 - 1} \\
\implies T(n) &= O(2^n)
\end{aligned}$$

We need to prove that $T(n) \le c.2^n$. Instead we prove $T(n) \le c.2^n - d$. (We got $T(n) = 2^n - 1$). We guess $T(n) \le c.2^n - d$. Thus:

$$\begin{aligned} T(n) &= 2T(n-1)+1 \\ &\le 2\left(c.2^{n-1} - d\right) + 1 \\ &= c.2^n - d - (d-1) \\ &\le c.2^n - d \text{ if } d \ge 1. \end{aligned}$$

3.5.14 Use a recursion tree to determine a good asymptotic upper bound on the recurrence $T(n) = 4T(\frac{n}{2}+2)+n$. Use the substitution method to verify your answer.

The recursion tree is shown in Figure 3.8.

$$\begin{aligned} T(n) &= n + \frac{4n}{2} + 8 + \frac{16n}{4} + 32 + \cdots + O(n^{\lg 4}) \\ &= \sum_{i=0}^{\lg(n-1)} 2^i n + 2 \sum_{i=0}^{\lg(n-1)} 4^i + O(n^2) \\ &= \frac{2^{\lg n} - 1}{2-1} n + 2 \cdot \frac{4^{\lg n} - 1}{4-1} + O(n^2) \\ &= (2^{\lg n} - 1)n + \frac{2}{3}(4^{\lg n} - 1) + O(n^2) \\ &= (n-1)n + \frac{2}{3}(n^2 - 1) + O(n^2) \\ &= O(n^2) \end{aligned}$$

Now we prove this using substitution. Lets assume, $T(n) \le cn^2$, for all $n \ge n_0$, where c and n_0 are some positive constants.

$$\begin{aligned} T(n) &= 4T(n/2+2)+n \\ &\le 4c(n/2+2)^2 + n \\ &= 4c(n^2/4 + 2n + 4) + n \\ &= cn^2 + 8cn + 16c + n \\ &= cn^2 + (8c+1)n + 16c \implies T(n) \le cn^2 \text{ for any choice of } n \end{aligned}$$

We make a new guess by subtracting a lower order term. Lets assume, $T(n) \le cn^2 - bn$, for all $n \ge n_0$, where c, b and n_0 are some positive constants.

$$\begin{aligned} T(n) &= 4T(n/2+2)+n \\ &\le 4(c(n/2+2)^2 - b(n/2+2)) + n \\ &= 4c(n^2/4 + 2n + 4) - 4b(n/2+2)) + n \\ &= cn^2 + 8cn + 16c - 2bn - 8b + n \\ &= cn^2 - bn - bn + 8cn + n + 16c - 2b \\ &= cn^2 - bn - (bn - 8cn - n - 16c + 2b) \\ &\le cn^2 - bn \quad \text{if } (bn - 8cn - n - 16c + 2b) \ge 0 \end{aligned}$$

That is, $n \ge (16c - 2b)/(b - 8c - 1) \ge 0$. By choosing b and c, the value of n_0 can be determined.

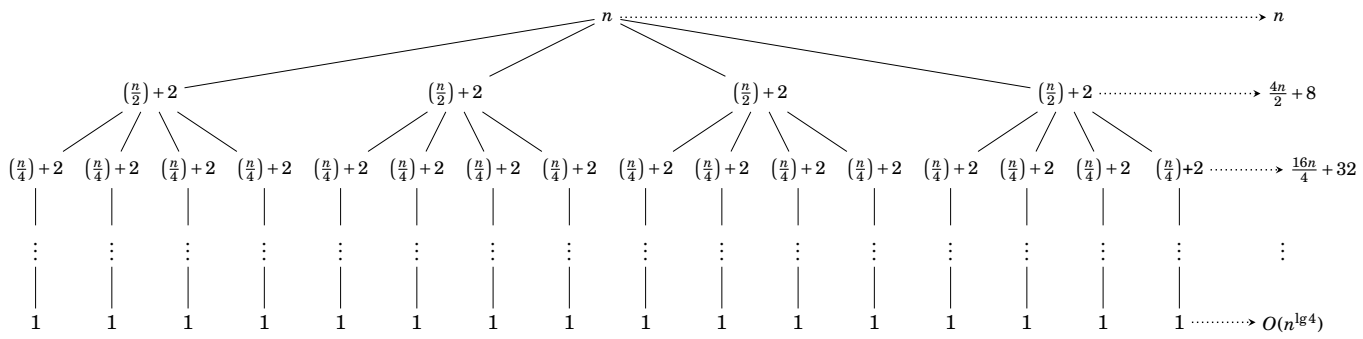


Figure 3.8: Recursion tree for the recurrence $T(n) = 4T(\frac{n}{2}+2)+n$

3.5.15 Argue that the solution to the recurrence $T(n) = T(\frac{n}{3}) + T(\frac{2n}{3}) + O(n)$, where c is a constant, is $\Theta(n \lg n)$ by appealing to the recursion tree.

The equation $T(n) = T(\frac{n}{3}) + T(\frac{2n}{3}) + O(n)$ could be rewritten as $T(n) = T(\frac{n}{3}) + T(\frac{2n}{3}) + cn$, whose recursion tree is shown in Figure 3.9.

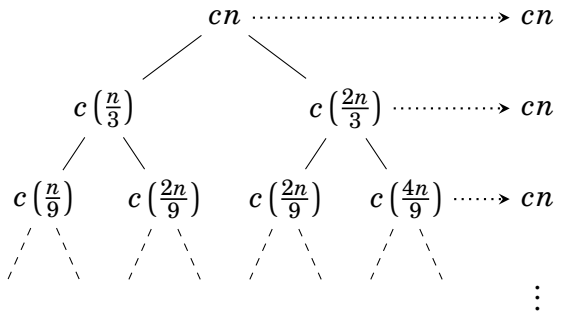


Figure 3.9: Recursion tree for the recurrence $T(n) = T(\frac{n}{3}) + T(\frac{2n}{3}) + cn$

Unlike earlier problems, here we find that all the leaves are not at the same level. The longest simple path from the root to a leaf is $n \longrightarrow \left(\frac{2}{3}\right)n \longrightarrow \left(\frac{2}{3}\right)^2 n \ldots\ldots\ldots \longrightarrow 1$. We have $\left(\frac{2}{3}\right)^k n = 1 \implies k = \log_{\frac{3}{2}} n$, i.e. height of the tree is $\log_{\frac{3}{2}} n$. Since the per level cost at each level is $cn, T(n)$ will be at most the number of levels $\times$ the cost of each level, or $O(cn \log_{\frac{3}{2}} n) \approx O(n \lg n)$. We use substitution method to verify this.

We guess $T(n) \leq dn \lg n$. Thus:

$$\begin{aligned}
T(n) &= T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + cn \\
&\leq d\left(\frac{n}{3}\right)\lg\left(\frac{n}{3}\right) + d\left(\frac{2n}{3}\right)\lg\left(\frac{2n}{3}\right) + cn \\
&= d\left(\frac{n}{3}\right)\lg\left(\frac{n}{3}\right) + d\left(\frac{2n}{3}\right)\lg\left(\frac{n}{\frac{3}{2}}\right) + cn \\
&= \frac{dn}{3}(\lg n - \lg 3) + \frac{2dn}{3}\left(\lg n - \lg\frac{3}{2}\right) + cn \\
&= \frac{dn}{3}(\lg n - \lg 3) + \frac{2dn}{3}(\lg n - \lg 3 + \lg 2) + cn \\
&= dn\lg n - dn\left(\lg 3 - \frac{2}{3}\right) + cn \\
&= dn\lg n - \left(dn\left(\lg 3 - \frac{2}{3}\right) - cn\right) \\
&\leq dn\lg n, \text{ if } dn\left(\lg 3 - \frac{2}{3}\right) - cn \geq 0, \text{ i.e., if } d \geq \frac{c}{\lg 3 - \frac{2}{3}}
\end{aligned}$$

The shortest simple path from the root to a leaf is $n \longrightarrow \left(\frac{1}{3}\right)n \longrightarrow \left(\frac{1}{3}\right)^2 n \ldots\ldots\ldots \longrightarrow 1$. We have $\left(\frac{1}{3}\right)^k n = 1 \implies k = \log_3 n$, i.e. height of the tree will be atleast $\log_3 n$. Since the per level cost at each level is cn, $T(n)$ will be at least the number of levels $\times$ the cost of each level, or $\Omega(cn\log_3 n) = \Omega(n\lg n)$. We use substitution method to verify this. We guess $T(n) \geq d'n\lg n$. Thus:

$$\begin{aligned}
T(n) &= T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + cn \\
&\geq d'\left(\frac{n}{3}\right)\lg\left(\frac{n}{3}\right) + d'\left(\frac{2n}{3}\right)\lg\left(\frac{2n}{3}\right) + cn \\
&= d'\left(\frac{n}{3}\right)\lg\left(\frac{n}{3}\right) + d'\left(\frac{2n}{3}\right)\lg\left(\frac{n}{\frac{3}{2}}\right) + cn \\
&= \frac{d'n}{3}(\lg n - \lg 3) + \frac{2d'n}{3}\left(\lg n - \lg\frac{3}{2}\right) + cn \\
&= \frac{d'n}{3}(\lg n - \lg 3) + \frac{2d'n}{3}(\lg n - \lg 3 + \lg 2) + cn \\
&= d'n\lg n - d'n\left(\lg 3 - \frac{2}{3}\right) + cn \\
&= d'n\lg n + \left(cn - d'n\left(\lg 3 - \frac{2}{3}\right)\right) \\
&\geq d'n\lg n, \text{ if } cn - d'n\left(\lg 3 - \frac{2}{3}\right) \geq 0, \text{ i.e., if } d' \leq \frac{c}{\lg 3 - \frac{2}{3}}
\end{aligned}$$

Since $T(n)$ is both $O(n\lg n)$ and $\Omega(n\lg n)$, we have $T(n) = \Theta(n\lg n)$.

3.5.16 Draw the recursion tree for $T(n) = 4T(\lfloor\frac{n}{2}\rfloor) + cn$, where c is a constant, and provide a tight asymptotic bound on its solution. Verify your answer with the substitution method.

We can omit the floor to make a guess. The resultant recursion tree is shown in Figure 3.10. The figure also shows the per-level costs.

From the tree, we get $l = \lg n + 1$.

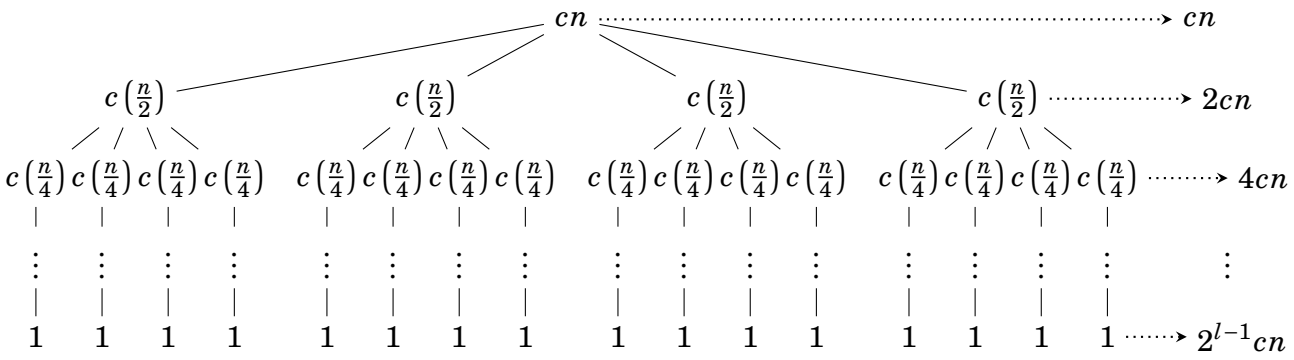


Figure 3.10: Recursion tree for the recurrence $T(n) = 4T(\frac{n}{2}) + cn$

The total per-level cost is

$$\begin{aligned}
T(n) &= \sum_{i=0}^{\lg n-1} 2^i cn \\
&= cn \sum_{i=0}^{\lg n-1} 2^i \\
&= cn\left(\frac{2^{\lg n}-1}{2-1}\right) \\
&= cn\left(\frac{n^{\lg 2}-1}{2-1}\right) \\
&= cn^2 - cn
\end{aligned}$$

Thus $T(n) = O(n^2)$. We guess $T(n) \le cn^2 - dn$. Then:

$$\begin{aligned}
T(n) &= 4T\left(\frac{n}{2}\right) + cn \\
&\le 4\left(c\left(\frac{n}{2}\right)^2 - d\left(\frac{n}{2}\right)\right) + cn \\
&= cn^2 - (2d - c)n \\
&= cn^2 - dn - (d - c)n \\
&\le cn^2 - dn \text{ if } d \ge c.
\end{aligned}$$

3.5.17 Use a recursion tree to give an asymptotically tight solution to the recurrence $T(n) = T(n-a) + T(a) + cn$, where $a \ge 1$ and $c > 0$ are constants.

See Figure 3.11 for the recursion tree.

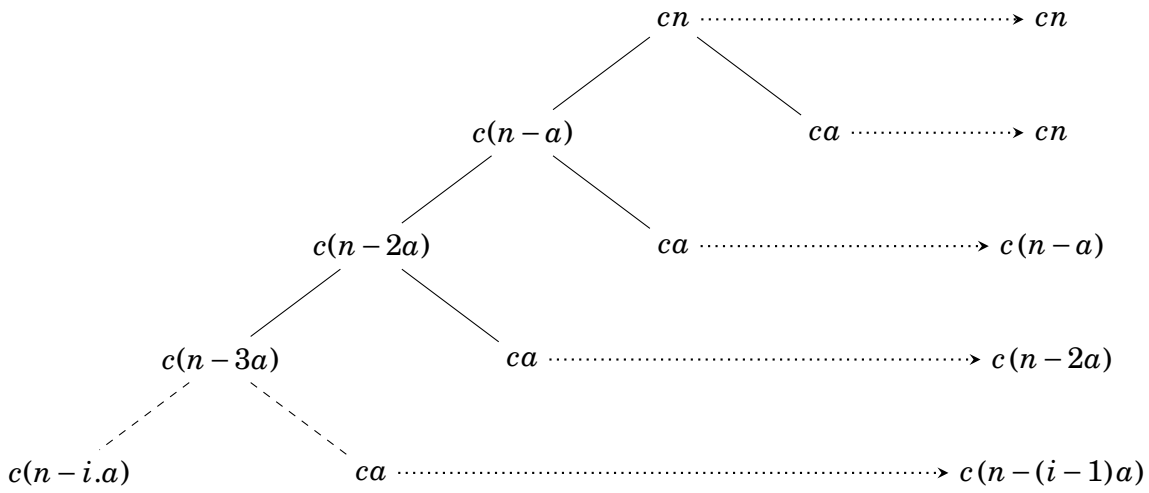


Figure 3.11: Recursion tree for the recurrence $T(n) = T(n-a) + T(a) + cn$

We have

$$\begin{aligned}
T(n) &= \sum_{i=0}^{\frac{n}{a}} c(n-ia) + cn \\
&= \sum_{i=0}^{\frac{n}{a}} cn - \sum_{i=0}^{\frac{n}{a}} cia + cn \\
&= cn.\sum_{i=0}^{\frac{n}{a}} 1 - ca.\sum_{i=0}^{\frac{n}{a}} i + cn \\
&= \frac{cn^2}{a} - \left(\frac{\frac{n}{a}.\left(\frac{n}{a}+1\right)}{2}\right) + cn \\
&= \Theta(n^2)
\end{aligned}$$

Thus we need to prove that $T(n)$ is $O(n^2)$ as well as $\Omega(n^2)$.

We guess $T(n) \leq cn^2$. Then we have

$$\begin{aligned}
T(n) &\leq c(n-a)^2 + ca + cn \\
&\leq cn^2 - 2acn + ca + cn \\
&\leq cn^2 - c(2an - a - n) && (a > \frac{1}{2}, n > 2a) \\
&\leq cn^2 - cn \\
&\leq cn^2 \\
&= O(n^2)
\end{aligned}$$

Next we guess $T(n) \geq cn^2$. We have

$$\begin{aligned}
T(n) &\geq c(n-a)^2 + ca + cn \\
&\geq cn^2 - 2acn + ca + cn \\
&\geq cn^2 - c(2an - a - n) && (a > \frac{1}{2}, n > 2a) \\
&\geq cn^2 + cn \\
&\geq cn^2 \\
&= \Omega(n^2)
\end{aligned}$$

3.5.18 Use a recursion tree to give an asymptotically tight solution to the recurrence $T(n) = T(\alpha n) + T((1-\alpha)n) + cn$, where α is a constant in the range $0 < \alpha < 1$, and $c > 0$ is also a constant.

Without loss of generality, we assume that $\alpha \geq \frac{1}{2}$. This means that $\alpha \geq 1 - \alpha$. Even if it was the other way, we could just swap the terms $T(\alpha n)$ and $T((1-\alpha)n)$ and thus the recurrence is unaffected.

We draw the recursion tree as shown in Figure 3.12. As is evident from the tree, the leaves are not at the same level. The longest path is the branch contributed by the αn factor. For this branch, at the leaf level, $\alpha^k n = 1$. That is

$$\alpha^k = \frac{1}{n} \Rightarrow \log \alpha^k = \log \frac{1}{n} \Rightarrow k \log \alpha = -\log n \Rightarrow k = \frac{-\log n}{\log \alpha} = \frac{\log n}{\log(1/\alpha)} = \log_{1/\alpha} n$$

Since the cost at each of these levels is cn, the upper bound for the recurrence could be taken as $O(n \log_{1/\alpha} n) = O(n \lg n)$

We prove it mathematically. We guess $T(n) \leq dn \lg n$

$$\begin{aligned}
T(n) &\leq d\alpha n \lg(\alpha n) + d(1-\alpha)n \lg((1-\alpha)n) + cn \\
&\leq d\alpha n \lg \alpha + d\alpha n \lg n + d(1-\alpha)n \lg(1-\alpha) + d(1-\alpha)n \lg n + cn \\
&\leq dn \lg n + n\big(d\alpha \lg \alpha + d \lg(1-\alpha) - d\alpha \lg(1-\alpha) + c\big) \\
&\leq dn \lg n \text{ if } d\alpha \lg \alpha + d \lg(1-\alpha) - d\alpha \lg(1-\alpha) + c \leq 0 \\
&\leq dn \lg n \text{ if } d\alpha \lg \alpha + d \lg(1-\alpha)(1-\alpha) + c \leq 0 \\
&\leq dn \lg n \text{ if } d \leq -\frac{c}{\alpha \lg \alpha + (1-\alpha)\lg(1-\alpha)}
\end{aligned}$$

The shortest path is the branch contributed by the $(1-\alpha)n$ factor. Arguing in a similar fashion as earlier, we find that such a path has $\log_{1/(1-\alpha)} n$ levels each with cost cn. Thus, the lower bound for the recurrence could be taken as $\Omega(n \log_{1/(1-\alpha)} n) = \Omega(n \lg n)$

We prove it mathematically. We guess $T(n) \geq dn \lg n$

$$\begin{aligned}
T(n) &\geq d\alpha n \lg(\alpha n) + d(1-\alpha)n \lg((1-\alpha)n) + cn \\
&\geq d\alpha n \lg \alpha + d\alpha n \lg n + d(1-\alpha)n \lg(1-\alpha) + d(1-\alpha)n \lg n + cn \\
&\geq dn \lg n + n\big(d\alpha \lg \alpha + d \lg(1-\alpha) - d\alpha \lg(1-\alpha) + c\big) \\
&\geq dn \lg n \text{ if } d\alpha \lg \alpha + d \lg(1-\alpha) - d\alpha \lg(1-\alpha) + c \geq 0 \\
&\geq dn \lg n \text{ if } d\alpha \lg \alpha + d \lg(1-\alpha)(1-\alpha) + c \geq 0 \\
&\geq dn \lg n \text{ if } d \geq -\frac{c}{\alpha \lg \alpha + (1-\alpha)\lg(1-\alpha)}
\end{aligned}$$

Since $T(n)$ is both $O(n \lg n)$ and $\Omega(n \lg n)$, we conclude that $T(n) = \Theta(n \lg n)$

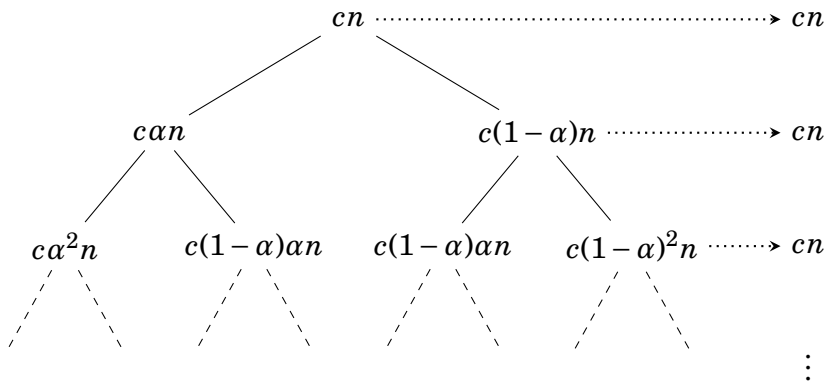


Figure 3.12: Recursion tree for the recurrence $T(n) = T(\alpha n) + T((1-\alpha)n) + cn$

3.5.19 Use the master method to give tight asymptotic bounds for the following recurrences:

(a) $T(n) = 2T(\frac{n}{4}) + 1$
(b) $T(n) = 2T(\frac{n}{4}) + \sqrt{n}$
(c) $T(n) = 2T(\frac{n}{4}) + n$
(d) $T(n) = 2T(\frac{n}{4}) + n^2$

Solution:

(a) $\Theta(n^{\log_4 2}) = \Theta(\sqrt{n})$ [Case 1. Identify that $\log_4 2 = \frac{1}{2}$]
(b) $\Theta(n^{\log_4 2} \lg n) = \Theta(\sqrt{n} \lg n)$ [Case 2]
(c) $\Theta(n)$ [Case 3. Regularity condition also holds (Verify this!).]
(d) $\Theta(n^2)$ [Case 3. Regularity condition also holds (Verify this!).]

3.5.20 Use the master method to show that the solution to the binary-search recurrence $T(n) = T(\frac{n}{2}) + \Theta(1)$ is $T(n) = \Theta(\lg n)$.

$a = 1, b = 2$
$f(n) = \Theta(1) = \Theta(n^{\log_2 1})$
$T(n) = \Theta(\lg n)$

3.5.21 Can the master method be applied to the recurrence $T(n) = 4T(\frac{n}{2}) + n^2 \lg n$? Why or why not? Give an asymptotic upper bound for this recurrence.

With $a = 4, b = 2$, we have $f(n) = n^2 \lg n \neq O(n^{2-\epsilon}) \neq \Omega(n^{2-\epsilon})$, So we cannot apply the master method.

We guess $\Theta(n^2 \lg^2 n)$: (It is easy to arrive at this guess by drawing a recursion tree)

$$\begin{aligned}
T(n) &\leq 4T(n/2) + n^2 \lg n \\
&\leq 4c(n/2)^2 \lg^2(n/2) + n^2 \lg n \\
&\leq cn^2 \lg(n/2) \lg n - cn^2 \lg(n/2) \lg 2 + n^2 \lg n \\
&\leq cn^2 \lg^2 n - cn^2 \lg n \lg 2 - cn^2 \lg(n/2) + n^2 \lg n \\
&\leq cn^2 \lg^2 n + (1-c)n^2 \lg n - cn^2 \lg(n/2) \qquad (c > 1) \\
&\leq cn^2 \lg^2 n - cn^2 \lg(n/2) \\
&\leq cn^2 \lg^2 n
\end{aligned}$$

3.5.22 Give asymptotic upper and lower bound for $T(n)$ in each of the following recurrences. Assume that $T(n)$ is constant for $n \leq 2$. Make your bounds as tight as possible, and justify your answers.

(a) $T(n) = 2T(\frac{n}{2}) + n^4$
(b) $T(n) = T(\frac{7n}{10}) + n$
(c) $T(n) = 16T(\frac{n}{4}) + n^2$
(d) $T(n) = 7T(\frac{n}{3}) + n^2$
(e) $T(n) = 7T(\frac{n}{2}) + n^2$
(f) $T(n) = 2T(\frac{n}{4}) + \sqrt{n}$
(g) $T(n) = T(n-2) + n^2$

Solution:

(a) $\Theta(n^4)$ [master method]
(b) $\Theta(n)$ (master method, $\log_{10/7} 1 = 0$)
(c) $\Theta(n^2 \lg n)$ [master method]
(d) $\Theta(n^2)$ [master method]
(e) $\Theta(n^{\log_2 7})$ [master method]
(f) $\Theta(\sqrt{n} \lg n)$ [master method]
(g) $T(n) = n^2 + T(n-2) = n^2 + (n-2)^2 + T(n-4) = \sum_{i=0}^{n/2} (n-2i)^2 = \Theta(n^3)$

3.5.23 Give asymptotic upper and lower bounds for $T(n)$ in each of the following recurrences. Assume that $T(n)$ is constant for sufficiently small n. Make your bounds as tight as possible, and justify your answers.

(a) $T(n) = 4T(\frac{n}{3}) + n \lg n$
(b) $T(n) = 3T(\frac{n}{3}) + \frac{n}{\lg n}$
(c) $T(n) = 4T(\frac{n}{2}) + n^2\sqrt{n}$
(d) $T(n) = 3T(\frac{n}{3} - 2) + \frac{n}{2}$
(e) $T(n) = 2T(\frac{n}{2}) + \frac{n}{\lg n}$
(f) $T(n) = T(\frac{n}{2}) + T(\frac{n}{4}) + T(\frac{n}{8}) + n$
(g) $T(n) = T(n-1) + \frac{1}{n}$
(h) $T(n) = \sqrt{n}T(\sqrt{n}) + n$
(i) $T(n) = T(n-1) + 2^n$

Solution:

(a) $\Theta(n^{\log_3 4})$ by the master method.
(b)

$$T(n) = 3T\left(\frac{n}{3}\right) + \frac{n}{\lg n} \tag{3.5.1}$$

Let $S(n) = \dfrac{T(n)}{n} \implies T(n) = n.S(n)$

Then equation 3.5.1 could be written as $n.S(n) = 3.\left(\dfrac{n}{3}\right) S\left(\dfrac{n}{3}\right) + \dfrac{n}{\lg n}$

Thus $S(n) = S\left(\frac{n}{3}\right) + \frac{1}{\lg n}$

$$\begin{aligned}
S(n) &= \frac{1}{\lg n} + \frac{1}{\lg \frac{n}{3}} + \frac{1}{\lg \frac{n}{9}} + \ldots \\
&= \frac{1}{\lg n} + \frac{1}{\lg n - \lg 3} + \frac{1}{\lg n - 2\lg 3} + \ldots \\
&\leq \frac{1}{\log_3 n} + \frac{1}{\log_3 n - \log_3 3} + \frac{1}{\log_3 n - 2\log_3 3} + \ldots \\
&= \frac{1}{\log_3 n} + \frac{1}{\log_3 n - 1} + \frac{1}{\log_3 n - 2} + \ldots \\
&= \sum_{i=0}^{\log_3 n - 1} \frac{1}{\log_3 n - i} \\
&= \sum_{i=1}^{\log_3 n} \frac{1}{i} \\
&= \Theta(\log_3 \log_3 n) \quad \text{using (R3.1.4) on page 22}
\end{aligned}$$

(c) $\Theta(n^2\sqrt{n}) = \Theta(n^{2.5})$ by the master method ($\log_2 4 = 2 < 2.5$)

(d) We can ignore the -2 and using master method, we arrive at $\Theta(n \lg n)$. Neglecting the -2 factor does not cause any harm in asymptotic analysis.

(e) We use iteration method:

$$\begin{aligned}
T(n) &= 2T\left(\frac{n}{2}\right) + \frac{n}{\lg n} \\
&= 4T\left(\frac{n}{4}\right) + 2.\frac{\frac{n}{2}}{\lg(\frac{n}{2})} + \frac{n}{\lg n} \\
&= 4T\left(\frac{n}{4}\right) + \frac{n}{\lg n - 1} + \frac{n}{\lg n} \\
&\vdots \\
&= nT(1) + \sum_{i=0}^{\lg n - 1} \frac{n}{\lg n - i} \\
&= nT(1) + n\sum_{i=1}^{\lg n} \frac{1}{i} \\
&= n.\Theta(\lg \lg n) \quad \text{using (R3.1.4) on page 22} \\
&= \Theta(n \lg \lg n).
\end{aligned}$$

(f) We construct a recursion tree as in Figure 3.13 to make a guess. The cost at root level is n, cost at first level is $\frac{n}{2} + \frac{n}{4} + \frac{n}{8} = \frac{7}{8}n$. Cost at second level is $\frac{49}{64}n = \left(\frac{7}{8}\right)^2 n$. Here we find that all the leaves are not at the same level. The shortest simple path from the root to a leaf is $n \longrightarrow \left(\frac{n}{2}\right) \longrightarrow \left(\frac{n}{4}\right) \ldots\ldots\ldots \longrightarrow 1$. We have $\frac{n}{2^l} = 1 \implies l = \lg n$. Thus $T(n)$ will be atleast $\sum_{i=0}^{\lg n} \left(\frac{7}{8}\right)^i n \approx 8n$ $\left[\text{Recall that } \sum_{i=0}^{\lg n} \left(\frac{7}{8}\right)^i n \leq n.\sum_{i=0}^{\infty} \left(\frac{7}{8}\right)^i = n.\left(\frac{1}{1-\frac{7}{8}}\right)\right]$. Thus $T(n) \geq 8n \implies T(n) = \Omega(n)$. We prove

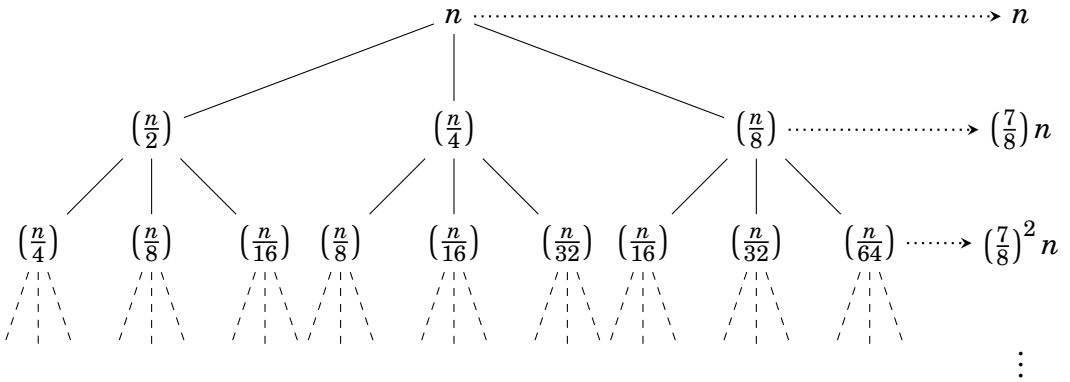


Figure 3.13: Recursion tree for the recurrence $T(n) = T(\frac{n}{2}) + T(\frac{n}{4}) + T(\frac{n}{8}) + n$

this:

$$\begin{aligned} T(n) &\geq c.\left(\frac{n}{2}\right) + c.\left(\frac{n}{4}\right) + c.\left(\frac{n}{8}\right) + n \\ &= \left(\frac{7}{8}\right)cn + n \\ &\geq cn \quad \forall c \leq 8 \\ &\implies T(n) = \Omega(n) \end{aligned}$$

The longest simple path from the root to a leaf is $n \longrightarrow \left(\frac{n}{8}\right) \longrightarrow \left(\frac{n}{64}\right)..... \longrightarrow 1$. We have $\frac{n}{8^l} = 1 \implies l = \log_8 n$. Thus $T(n)$ will be atmost $\sum_{i=0}^{\log_8 n} \left(\frac{7}{8}\right)^i n \leq \sum_{i=0}^{\lg n} \left(\frac{7}{8}\right)^i n \approx 8n$. Thus $T(n) \leq 8n \implies T(n) = O(n)$. We prove this:

$$\begin{aligned} T(n) &\leq c.\left(\frac{n}{2}\right) + c.\left(\frac{n}{4}\right) + c.\left(\frac{n}{8}\right) + n \\ &= \left(\frac{7}{8}\right)cn + n \\ &\leq cn \quad \forall c \geq 8 \\ &\implies T(n) = O(n) \end{aligned}$$

Since $T(n) = \Omega(n)$ and $T(n) = O(n)$, we conclude $T(n) = \Theta(n)$

(g) We use iteration method:

$$\begin{aligned} T(n) &= T(n-1) + \frac{1}{n} \\ &= \frac{1}{n} + \frac{1}{n-1} + T(n-2) \\ &= \frac{1}{n} + \frac{1}{n-1} + \frac{1}{n-2} + T(n-3) \\ &= \sum_{i=0}^{n-1} \frac{1}{n-i} \\ &= \sum_{i=1}^{n} \frac{1}{i} \\ &= \Theta(\lg n) \quad \text{using (R3.1.3) on page 22} \end{aligned}$$

(h) $T(n) = \sqrt{n}T(\sqrt{n}) + n$

Dividing by n, we get $\frac{T(n)}{n} = \frac{\sqrt{n}T(\sqrt{n})}{n} + 1$

$\implies \frac{T(n)}{n} = \frac{T(\sqrt{n})}{\sqrt{n}} + 1$

Let n = 2^m, then $\sqrt{n} = 2^{\frac{m}{2}}$, and $m = \lg n$

$\frac{T(2^m)}{2^m} = \frac{T\left(2^{\frac{m}{2}}\right)}{2^{\frac{m}{2}}} + 1$

Let $S(m) = \frac{T(2^m)}{2^m}$, then

$S(m) = S(\frac{m}{2}) + 1$

Now, we apply the Master Theorem to get $S(m) = \Theta(\lg m)$

$T(2^m) = 2^m S(m) \implies T(2^m) = 2^m \Theta(\lg m)$.

Changing back to n, we get $T(n) = n.\Theta(\lg\lg n) \implies T(n) = \Theta(n \lg\lg n)$.

(i) We proceed as follows:

Put $T(n) = S(2^n)$, then the recurrence becomes

$S(2^n) = S(2^{n-1}) + 2^n$

Let $m = 2^n$, then the recurrence becomes

$S(m) = S(\frac{m}{2}) + m$

Applying master theorem we get

$S(m) = \Theta(m)$

Thus $S(m) = \Theta(m) \implies S(2^n) = T(n) = \Theta(2^n)$

CST306 : Algorithm analysis and design

Chapter 4
Height balanced trees

Narasimhan T.

4.1 Introduction

The motivation behind storing data items as a tree structure rather than as a sequential structure (say array) is that a tree structure allows various operations (insert, delete, search) to be performed in logarithmic time. This is because the height of a tree with n nodes is $\log n$. But sometimes the height of a tree becomes linear as shown in figure 4.1. Such trees are called skewed binary trees. Thus to maintain the tree heights to be logarithmic, we define the notion of *height balanced trees*. A tree is ***perfectly height-balanced*** if the left and right subtrees of any node are of the same height. In practice, we try to make the tree as balanced as possible. i.e. make the heights of left and right subtrees of any node differ only a small constant factor (say 1). We discuss one type of balanced trees here – the AVL tree.



(a) A right skewed binary search tree

(b) A left-skewed binary search tree

Figure 4.1: Skewed binary search trees

4.2 AVL trees

An AVL tree is a binary search tree (named after its inventors, **A**delson-**V**elskii and **L**andis) where the heights of the two subtrees of any node differ by at most one. This is called the ***height-balance property***. We

associate with each node a ***balance-factor*** defined as

balance factor = height of left subtree – height of right subtree (4.2.1)

Figure4.2 shows an AVL tree. The balance factor is shown atop each node. The balance factor of a node x in AVL tree is denoted as $x.bf$. Since the heights of left and right subtrees cannot differ by more than 1, $x.bf$ can possibly take one of the three values -1, 0 or +1. The node x is called ***left-heavy*** if $x.bf$ = +1. If $x.bf$ = -1, x is ***right-heavy*** and if $x.bf$ = 0, x is ***balanced***. Otherwise x is said to be unbalanced.

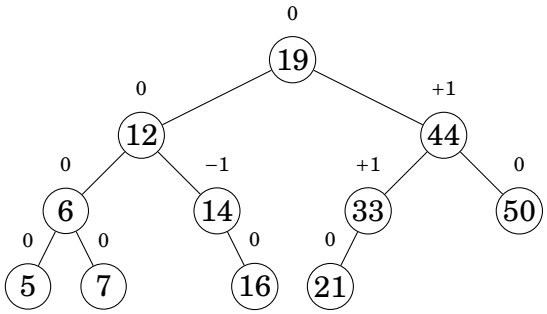


Figure 4.2: An AVL tree showing the balance factors of various nodes

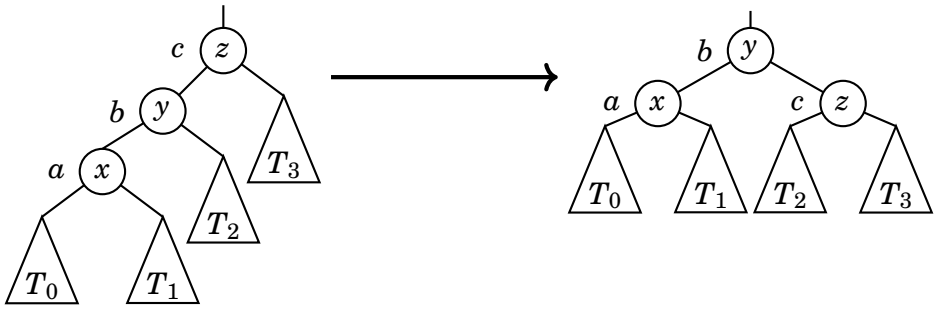
When various operations are performed on an AVL tree, if any of the node becomes unbalanced, we perform ***rotations*** to restore the height-balance property. Binary search tree rotations are operations on a binary search tree that change the structure (moving subtrees within the tree) without interfering with the order of the elements (which means binary-search-tree property is maintained). We describe a unified restructuring operation, called ***trinode restructuring*** that involves a node z, which has a child y and a grandchild x. This operation is described in the pseudocode RESTRUCTURE(z).

RESTRUCTURE(z)

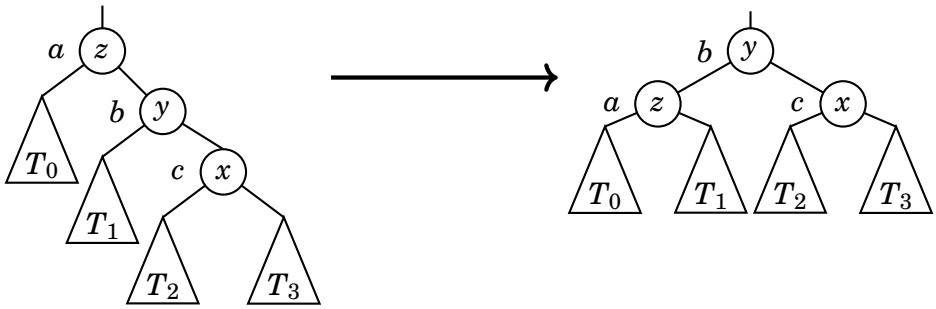
1 Let (n_1, n_2, n_3) be the inorder listing of the nodes x, y, z. Rename n_1 as a, n_2 as b and n_3 as c. Let (T_0, T_1, T_2, T_3) be the inorder listing of the four subtrees of x, y and z that are not rooted at x, y or z.
2 Replace the subtree rooted at z with a new subtree rooted at b.
3 Make a to be the left child of b and also make T_0 and T_1 to be the left and right subtrees of a respectively.
4 Make c to be the right child of b and also make T_2 and T_3 to be the left and right subtrees of c respectively.
5 Recalculate the heights of a, b and c.
6 **return** b.

There are four possible ways of mapping x, y and z to a, b and c as shown in Figure 4.3. When the node y gets relabeled as b, the trinode restructuring method is called a ***single rotation***, for it can be visualized as rotating y over z. (Figures 4.3 (a) and (b) depict this.) Otherwise, when x gets relabeled as b, the trinode restructuring operation is called ***double rotation***, for it can be visualized as first rotating x over y and then over z. (Figures 4.3 (c) and (d) depict this.)

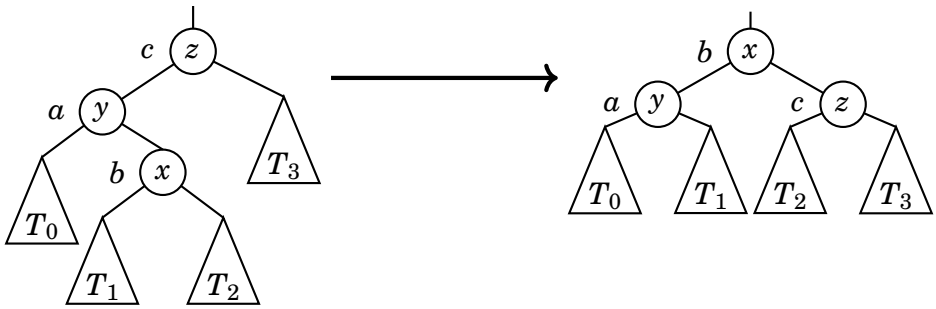
In the case of single rotation, when z is relabeled as c, the rotation is called ***right rotation*** as the rotation of y over z happens in clockwise direction. (See Figure 4.3 (a).) On the other hand, when z is relabeled as a, the rotation is called ***left rotation*** as the rotation of y over z happens in counter clockwise direction. (See Figure 4.3 (b).)



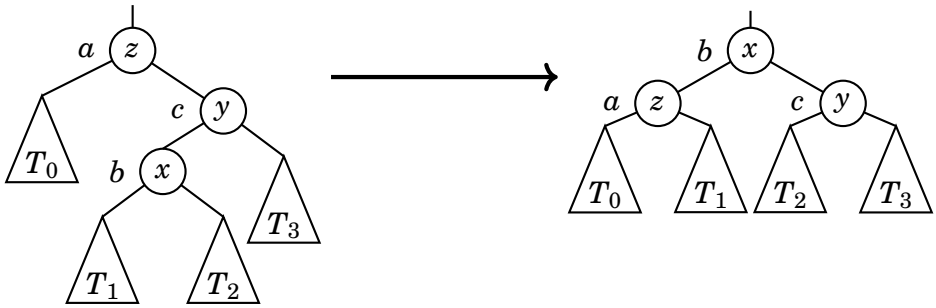
(a) Right rotation



(b) Left rotation



(c) Left Right rotation



(d) Right Left rotation

Figure 4.3: Four types of rotations in AVL tree

In the case of double rotation, when z is relabeled as c, the rotation is called ***left right rotation*** (LR rotation) as the first rotation of x over y happens in counter clockwise direction and the second rotation of x over z happens in clockwise direction. (See Figure 4.3 (c).) On the other hand, when z is relabeled as a, the rotation is called ***right left rotation*** (RL rotation) as the first rotation of x over y happens in clockwise

direction and the second rotation of x over z happens in counter clockwise direction. (See Figure 4.3 (d).)

4.2.1 Insertion in AVL trees

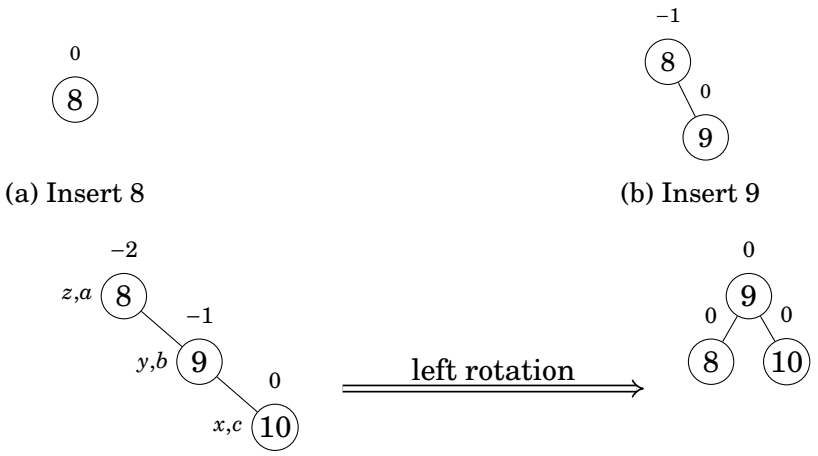
Assume we want to insert a node w into an AVL tree T. Since AVL tree is basically a binary search tree, to insert w, we do as for a binary search tree. But following this insertion, the balance factor of some nodes will change and certain nodes could become unbalanced too. It is noteworthy that when a new node is inserted, the balance factor of only those nodes will change which lie on any path from w to the root. To maintain the height balance property, we resort to the trinode restructuring. Usually we perform rotation about a specific node called ***pivot*** node (or critical node). To find the pivot node, we start from w and move up towards the root. The first unbalanced node that we encounter on this path is the pivot node. Trinode restructuring demands three nodes $x, y$and z which are identified as follows:

- pivot node is taken as z.
- take y to be that child of z with higher height (note that y must be an ancestor of w).
- take x be that child of y with higher height (if there is a tie, choose x to be an ancestor of w). Note that node x could be equal to w as well.

We then rebalance the subtree rooted at z by calling the trinode restructuring method.

4.2.1.1 Solved Problems

1. Insert the keys $8, 9, 10, 2, 1, 5, 3, 6, 4, 7$ in that order into an initially empty AVL tree.

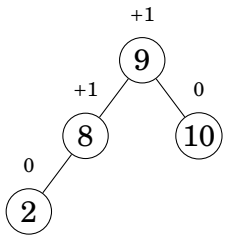


(a) Insert 8

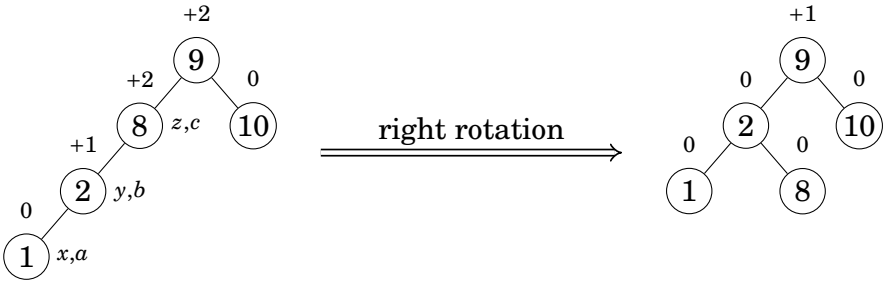
(b) Insert 9

(c) Insertion of 10 and subsequent left rotation

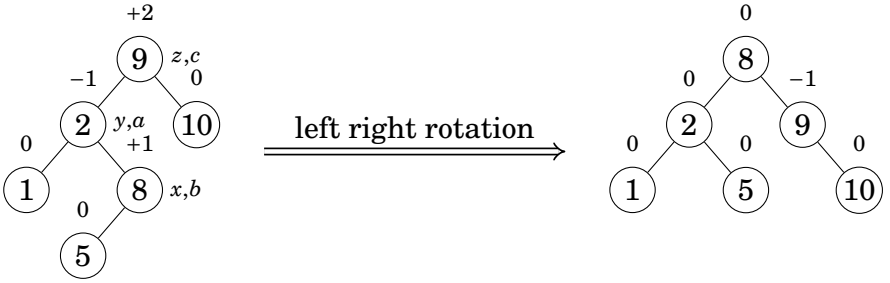
Figure 4.4: Insertion of keys into an AVL tree



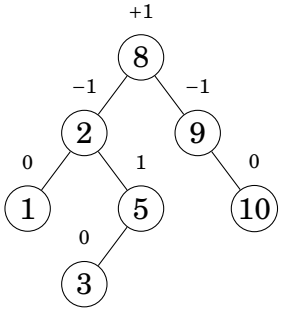
(d) Insert 2



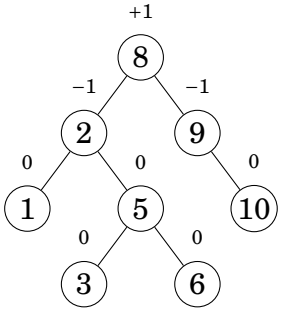
(e) Insertion of 1 and subsequent right rotation



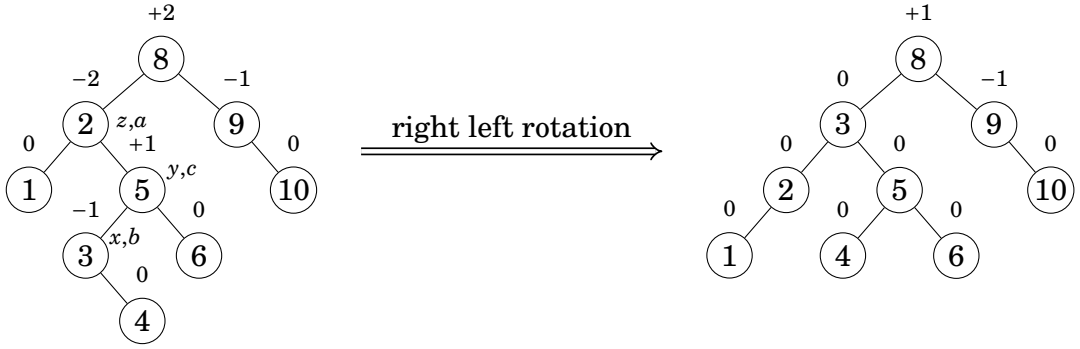
(f) Insertion of 5 and subsequent LR rotation



(g) Insert 3

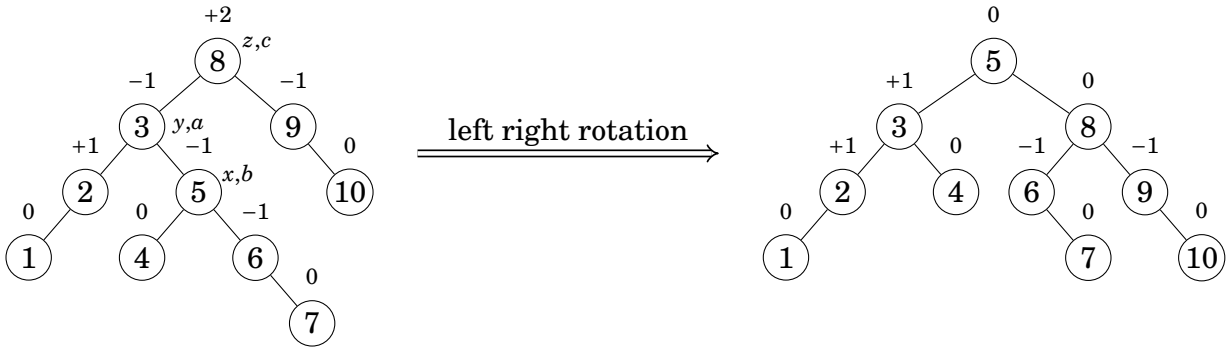


(h) Insert 6



(i) Insertion of 4 and subsequent RL rotation

Figure 4.4: Continued from page 53 - Insertion of keys into an AVL tree



(j) Insertion of 7 and subsequent LR rotation

Figure 4.4: (Continued from page 54) - Insertion of keys into an AVL tree

4.2.2 Deletion from AVL trees

We proceed with the deletion as with a binary search tree. Let w be the parent of the deleted node. As with insertion, we use trinode restructuring to restore balance in the AVL tree. We identify the three nodes $x, y$and z as follows:

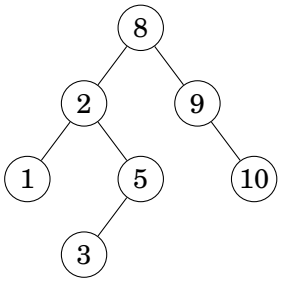
- pivot node is taken as z.
- take y to be that child of z with higher height (note that node y is the child of z that is not an ancestor of w).
- choose x as follows:
  - if one of the children of y is taller than the other, take x to be the taller child of y.
  - else (both children of y have the same height), take x to be the child of y on the same side as y (that is, if y is a left child, take x to be the left child of y, else take x to be the right child of y).

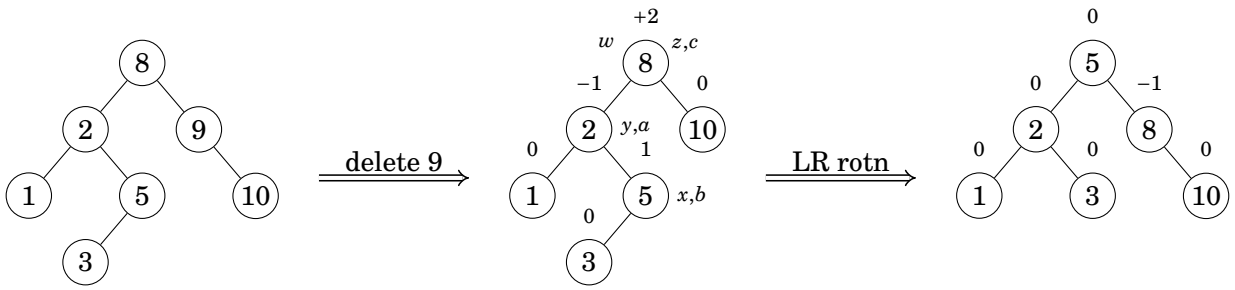
We then rebalance the subtree rooted at z by calling the trinode restructuring method.

Unlike insertion, a single trinode restructuring may not restore the height-balance property globally after deletion of a node. So, after rebalancing z, we continue walking up in the tree looking for unbalanced nodes. If we find another unbalanced node, we perform a restructure operation to restore its balance, and continue marching up in the tree looking for more, all the way to the root.

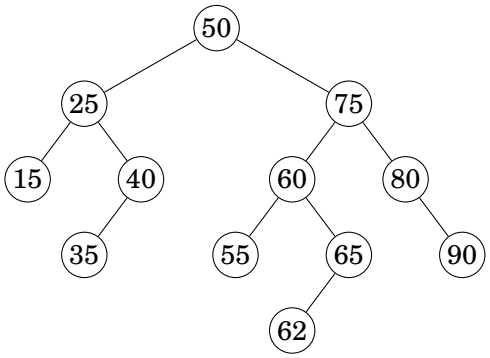
4.2.2.1 Solved Problems

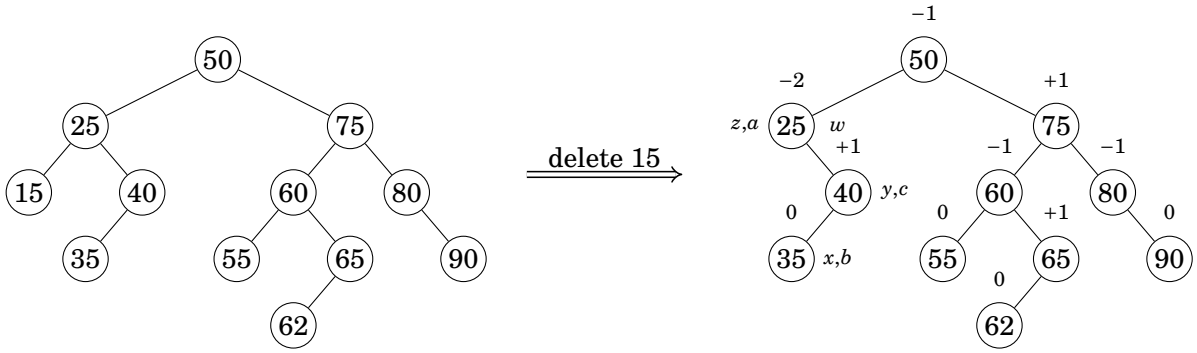
1. Delete 9 from the following figure:

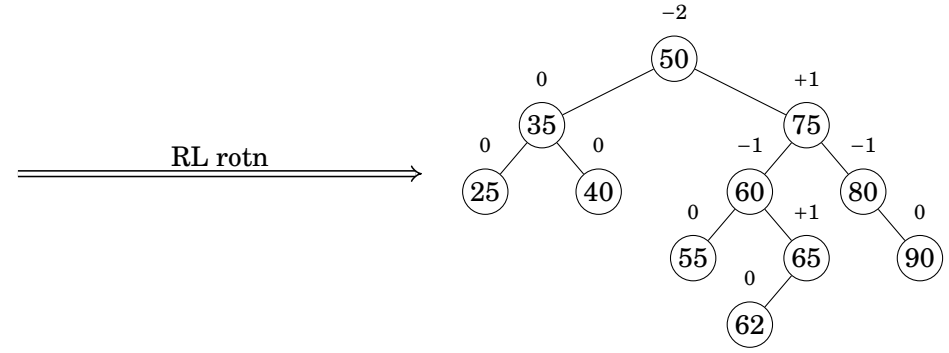




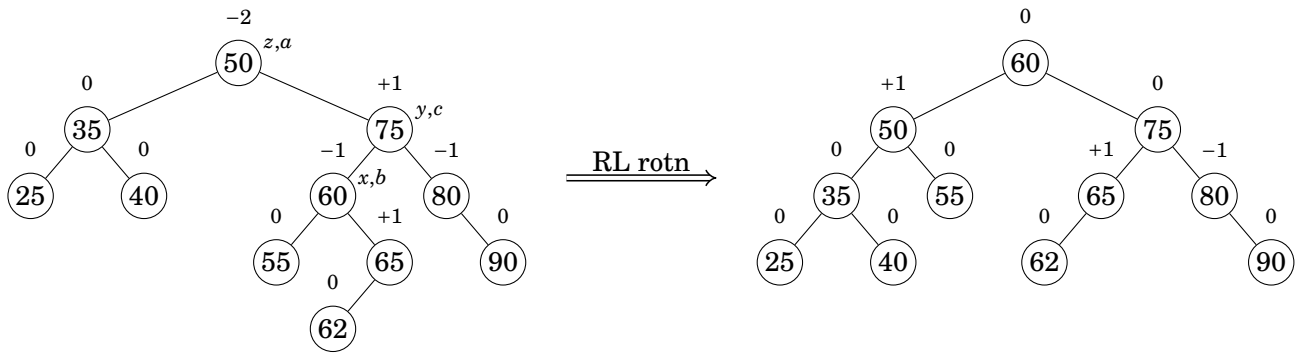
2. Delete 15 from the following tree:







We see that the tree is unbalanced even after this restructuring. This means us that we have to repeat the restructuring until the tree becomes balanced.



4.3 Do rotations really matter?

The intuition behind rotations is to restore the height balance property of unbalanced nodes. This property is required so as to maintain the logarithmic height of the AVL tree.

Theorem 4.1. *Any AVL tree with n nodes has height atmost* $1.44 \lg n$.

Proof: Instead of trying to find an upper bound for the height of an AVL tree directly, let us instead concentrate on the "inverse problem" of characterizing the minimum number of nodes $n(h)$ in an AVL tree with height h. As base cases for a recursive definition, notice that

- $n(0) = 1$, because an AVL tree of height 0 will have exactly one node, and
- $n(1) = 2$, because an AVL tree of height 1 must have at least two nodes.

Now, for the general case of an AVL tree T with the minimum number of nodes for height h, note that the root of such a tree will have as one of its children's subtrees, an AVL tree with the minimum number of nodes for height $h-1$. Now the other subtree of root should be of height $h-1$ or $h-2$ so as to maintain the height balance property. This further boils down to the fact that the other child's subtree should be an AVL tree with the minimum number of nodes for height $h-2$. Taking the root itself into account, we obtain the following recurrence:

$$n(h) = 1 + n(h-1) + n(h-2)$$

Claim: $n(h) \geq \phi^h \quad \forall h \geq 0$, where $\phi = \dfrac{1+\sqrt{5}}{2} \approx 1.618$ is the Golden ratio. We prove this using induction on h.

Basis: When $h = 0$, $n(h) = 1 = \phi^0$ and when $h = 1$, $n(h) = 2 > \phi^1$

Induction: Assume that the claim is true for all m such that $0 \leq m \leq h$ and $h \geq 1$.

We have

$$\begin{aligned} n(h+1) &= 1 + n(h) + n(h-1) \\ &\geq 1 + \phi^h + \phi^{h-1} \qquad \text{by induction hypothesis} \\ &= 1 + \phi^{h-1}(1+\phi) \\ &= 1 + \phi^{h+1} \qquad \text{using the fact that } 1+\phi = \phi^2 \\ &> \phi^{h+1} \end{aligned}$$

Thus the claim is true.

Now, in an AVL tree of height h with n nodes,

$$n \geq n(h) \geq \phi^h$$

This means that

$$\begin{aligned} h &\leq \log_{\phi} n \\ &= \frac{\lg n}{\lg \phi} \\ &\approx 1.44 \lg n \end{aligned}$$

Thus, for an AVL tree of height h with n nodes,

$$h \leq 1.44 \lg n$$

■

CST306 : Algorithm analysis and design

Chapter 5
Computability theory

Narasimhan T.

5.1 Foundations

So far, our discussion has focused on *algorithmic problems*, where the goal was to design algorithms to solve them efficiently. The emphasis was on how to "efficiently solve" such problems.

We now broaden our perspective to the more general notion of *computational problems*, which include algorithmic problems as a special case. Our objective is to understand the limits of algorithmic solvability. In particular, we aim to classify problems based on their inherent difficulty and determine whether they can be solved within a given model of computation.

Not all computational problems are solvable. A classical example is the *halting problem*, which is not solvable on any computational model. This leads us to the fundamental distinction between solvable and unsolvable problems. Studying unsolvability is useful for two main reasons:

1. If a problem is known to be algorithmically unsolvable, it indicates that the problem must be simplified or reformulated before an algorithmic solution can be sought.
2. Understanding unsolvable problems provides deeper insights into the power and limitations of computational models.

A computational problem can be either an *optimization problem* or a *decision problem*. In an optimization problem, each feasible solution has an associated value, and the goal is to find a solution with the optimal (best) value. For example, in the shortest path problem, we are given a directed weighted graph G along with two vertices u and v, and the objective is to find a path from u to v with minimum total weight.

In contrast, a decision problem has a binary outcome: "Yes" or "No" (equivalently, 1 or 0). For example, in the halting problem, we are given a Turing machine M and an input string w, and we ask whether M eventually halts when run on w.

An *instance* of a problem is a specific input obtained by assigning values to all its parameters. For example, in the shortest path problem, an instance consists of a graph together with two designated vertices. For a decision problem, an instance for which the answer is "Yes" is called a *yes-instance*, while one for which the answer is "No" is called a *no-instance*.

For the remainder of the discussion, we will confine ourselves to the realm of decision problems. By this, we don't lose any generality because of the fact that any optimization problem can be cast as a series of decision problem by introducing a bound on the objective value and asking whether a solution exists within that bound.

For example, consider the shortest path problem. The corresponding decision problem can be framed as:

PATH: Given a directed graph G with non-negative edge weights, two vertices u and v, and a cost bound k, does there exist a path from u to v whose total cost is at most k?

To solve the optimization version (i.e., find the minimum-cost path), we proceed as follows:

- Choose an initial value $k = K$, where K is an upper bound on the maximum possible path cost (for example, the sum of all edge weights).
- Solve the decision problem for this value of k.
- If the answer is YES, decrease k (i.e., try $k-1$) and repeat.
- Continue this process until the answer becomes NO.
- The smallest value of k for which the answer is YES is the cost of the shortest path.

5.2 Tractable and Intractable Problems

Algorithms whose worst-case running time on inputs of size n is $O(n^c)$, for some constant c, are called *polynomial-time algorithms*.

A natural question that arises is whether all problems can be solved in polynomial time. The answer is no. There exist problems that are solvable, but for which no polynomial-time algorithms are known; that is, they cannot be solved in time $O(n^c)$ for any constant c.

Problems that can be solved by polynomial-time algorithms are called *tractable* (or easy), whereas problems that require super-polynomial time (i.e., time not bounded by $O(n^c)$ for any constant c) are considered *intractable* (or hard).

A subtle point. Not all algorithms that appear polynomial are truly polynomial in the size of the input. For instance, the dynamic programming solution to the 0/1 Knapsack problem runs in $O(nW)$ time, where W is the capacity of the knapsack and n is the number of items.

At first glance, this seems polynomial. Indeed, the running time is polynomial in the *numeric value* of the parameter W.

However, the input size is measured in the number of bits required to represent W, which is $O(\log W)$. Therefore,

$$O(nW) = O(n \cdot 2^{\log W}),$$

which is exponential in the size of the input (measured in bits).

Algorithms whose running time is polynomial in the numeric value of the input (but not in its encoding length) are called *pseudo-polynomial time algorithms*. This highlights an important distinction: polynomial in value does not imply polynomial in input size.

What is input size?

The running time of an algorithm is measured as a function of the size of the input, where the size is defined as the number of bits required to represent the input.

For some problems, such as sorting n numbers, the running time depends primarily on n, the number of elements, and not directly on the magnitude of the numbers. Hence, the distinction between numeric value and encoding length is usually not significant.

However, for problems like 0/1 Knapsack or primality testing, the input includes numerical values whose magnitudes can be arbitrarily large. These values are represented in binary; for example, a number such as W or N requires $O(\log W)$ or $O(\log N)$ bits, respectively.

Polynomial vs Pseudo-Polynomial vs Exponential

- *Polynomial-time algorithms:* Running time is polynomial in the size of the input (measured in bits), e.g., $O(n^2), O(n^3)$.
- *Pseudo-polynomial-time algorithms:* Running time is polynomial in the numeric value of the input, but exponential in the input size, e.g., $O(nW)$ for 0/1 Knapsack.
- *Exponential-time algorithms:* Running time grows exponentially with input size, e.g., $O(2^n), O(n!)$. These typically arise when all possible configurations (subsets, assignments, etc.) must be explored.

While many important problems admit exponential-time algorithms, it is widely believed that some of them do not have polynomial-time solutions. For example, problems such as satisfiability and several combinatorial optimization problems are not known to have efficient algorithms.

There are also problems arising from generalized games that are computationally very hard. For instance, determining the winner in a generalized version of chess (played on an $n \times n$ board) or the game of Go is known to be computationally intractable under standard formulations.

In the next section, we will study some fundamental computational problems such as Clique, Vertex Cover, and Satisfiability that play a central role in complexity theory. These problems are widely believed to be intractable, and understanding their structure will help us formalize the notion of computational hardness.

Thus, computational problems exhibit a wide spectrum of complexity, ranging from efficiently solvable (polynomial time) to computationally hard (believed to require super-polynomial time), with pseudo-polynomial algorithms occupying an intermediate position.

5.3 Some Interesting Problems

Having discussed the broad classification of problems based on their computational complexity, we now turn our attention to some fundamental problems that will serve as key examples in the remainder of this chapter.

5.3.1 Clique Problem

A ***clique*** in an undirected graph $G = (V, E)$ is a set $V' \subseteq V$ such that every pair of vertices in V' is connected by an edge. In other words, a clique is a complete subgraph of G. The *size* of a clique is the number of vertices it contains. Figure 5.1 shows a graph in which the shaded vertices form a clique of size 5.

The clique problem is defined as follows:

CLIQUE: Given a graph G and an integer k, does G contain a clique of size k?

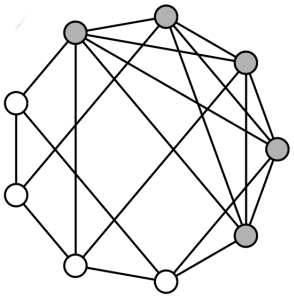


Figure 5.1: The shaded vertices constitute a clique

5.3.2 Vertex Cover Problem

A ***vertex cover*** in an undirected graph $G = (V, E)$ is a set $V' \subseteq V$ such that every edge in E has at least one endpoint in V'. That is, for every $(u, v) \in E$, either $u \in V'$ or $v \in V'$ (or both). The *size* of a vertex cover is the number of vertices in it.

For example, the graph in Figure 5.2 has vertex covers $\{A, B, C, D, E, F, G\}$ and $\{A, C, D, F\}$, and a smallest vertex cover of size 3: $\{B, E, G\}$.

The vertex cover problem is defined as follows:

VERTEX-COVER: Given a graph G and an integer k, does G contain a vertex cover of size k?

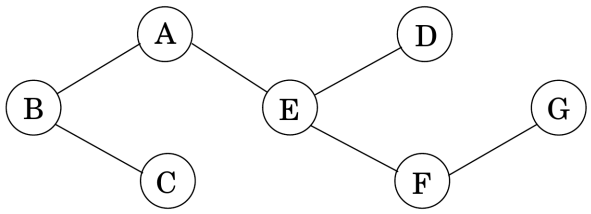


Figure 5.2: A graph with the smallest vertex cover $\{B, E, G\}$

5.3.3 Circuit Satisfiability Problem

A Boolean combinational circuit consists of logic gates (such as AND, OR, NOT) interconnected by wires. The *size* of a circuit is the total number of gates plus wires.

A *truth assignment* assigns Boolean values to the inputs of the circuit. A circuit is said to be *satisfiable* if there exists a truth assignment that makes the output equal to 1.

For example, in Figure 5.3, the assignment $\langle x_1 = 1, x_2 = 1, x_3 = 0 \rangle$ makes the output 1, and hence the circuit is satisfiable.

The circuit satisfiability problem is defined as follows:

CIRCUIT-SAT: Given a Boolean combinational circuit composed of AND, OR, and NOT gates, is it satisfiable?

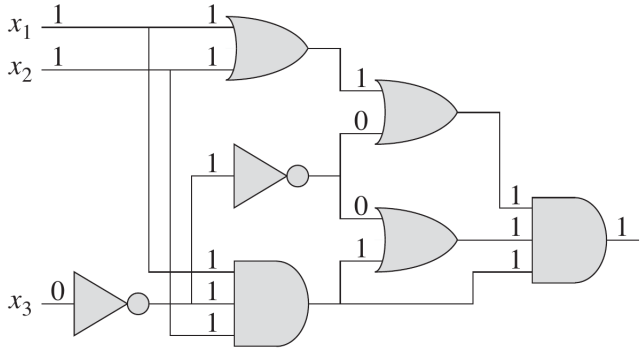


Figure 5.3: An example of a satisfiable Boolean circuit

5.3.4 Formula Satisfiability Problem

A Boolean formula ϕ is a parenthesized expression formed from Boolean variables using operations such as OR ($\vee$), AND ($\wedge$), and NOT ($\neg$) (as well as derived operations like $\rightarrow$ and $\leftrightarrow$).

A *truth assignment* assigns values to its variables, and a *satisfying assignment* is one that makes the formula evaluate to 1. A formula that has a satisfying assignment is called *satisfiable*.

For example, the formula

$$\phi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$$

is satisfied by $\langle x_1 = 0, x_2 = 0, x_3 = 1, x_4 = 1 \rangle$.

A Boolean formula is in ***conjunctive normal form*** (CNF) if it is a conjunction (AND) of clauses, where each clause is a disjunction (OR) of literals. A literal is a variable or its negation. For example:

$$(x_2 \vee x_1 \vee \neg x_5) \wedge (x_4 \vee x_6) \wedge x_2$$

A formula is in ***3-conjunctive normal form*** (3-CNF) if it is in CNF and each clause contains exactly three distinct literals. For example:

$$(\neg x_1 \vee x_2 \vee \neg x_5) \wedge (x_4 \vee \neg x_6 \vee x_7) \wedge (x_2 \vee x_5 \vee x_8) \wedge (x_1 \vee x_3 \vee \neg x_4)$$

The 3-SAT problem is defined as follows:

3-SAT: Given a Boolean formula ϕ in 3-CNF, is ϕ satisfiable?

All the above problems share a common feature: no polynomial-time algorithms are known for them. A straightforward approach to each problem involves checking all possible candidates:

CLIQUE: For a graph with n vertices, there are $\binom{n}{k}$ subsets of size k, each of which must be checked to determine whether it forms a clique.

VERTEX-COVER: For a graph with n vertices, there are $\binom{n}{k}$ subsets of size k, each of which must be checked to determine whether it forms a vertex cover.

CIRCUIT-SAT: For a circuit with n inputs, there are 2^n possible assignments, each of which must be checked.

3-SAT: For a formula with n variables, there are 2^n possible assignments, each of which must be checked.

For the first two problems, we note that

$$\binom{n}{k} \leq n^k$$

and in the worst case k can be as large as n, giving

$$\binom{n}{k} \leq n^n = 2^{n \log n}$$

which is exponential in n.

Thus, in all the above problems, the number of possibilities to be checked is exponential, and hence the corresponding algorithms run in exponential (and therefore super-polynomial) time.

Moreover, these algorithms share another important characteristic: they are all *deterministic*. That is, given a particular input, the sequence of steps executed by the algorithm is completely fixed, and the output can be predicted with certainty.

In contrast, a non-deterministic algorithm may exhibit different behaviours on the same input across different executions. In such a model, at each step of computation, there may be multiple possible choices for proceeding further. The algorithm effectively "guesses" one of these choices and continues.

This choice is abstractly captured by a function CHOOSE(S), which takes as input a set S and returns an arbitrary element of S. There is no rule governing how this choice is made; it may even be viewed as being made "magically" or "randomly".

A non-deterministic algorithm can be viewed as consisting of two stages:

- a *guessing stage*, where a potential solution is generated using CHOOSE, and
- a *verification stage*, where the guessed solution is checked for correctness using a deterministic procedure.

Since multiple choices may be available at each step, the computation can be thought of as branching into multiple paths. Conceptually, the algorithm can be imagined as creating several "clones", each exploring a different sequence of choices. If a solution exists, then at least one of these computation paths will lead to a correct solution, and one such clone will discover it.

Does adding the power of non-determinism help improve running time? To explore this, let us consider a non-deterministic algorithm for the clique problem on a graph $G(V,E)$.

NDET-CLIQUE-SOLVER(G,k)

```
1  V' = Φ        // Guessing phase begins
2  for i = 1 to k
3        v = CHOOSE(G.V)        // Choose a vertex to be added to the clique set V'
4        if v ∈ V'
5              error "vertex v already chosen"
6        else
7              V' = V' ∪ {v}
8  for each vertex u ∈ V'        // Verifying phase begins
9        for each vertex v ∈ V'
10             if u ≠ v AND (u,v) ∉ G.E
11                   error "V' does not form a clique"
12 return V'    // return the clique V'
```

Let us now analyse the running time of NDET-CLIQUE-SOLVER(G,k). The guessing stage selects a set V' of size k, which takes $O(k)$ time. Since $|V'| = k$, the verification stage (checking whether every pair of vertices in V' is connected) takes $O(k^2)$ time. Thus, NDET-CLIQUE-SOLVER(G,k) runs in non-deterministic $O(k^2)$ time, and hence we obtain a *non-deterministic polynomial-time algorithm* for the clique problem.

The above statement, however, requires careful interpretation. When we say that we have a non deterministic algorithm to "solve" a decision problem, it should be understood that such an algorithm is primarily a conceptual or definitional tool for capturing the notion of polynomial-time verifiability, rather than a practical method for solving problems.

It is important to note that non-determinism does not correspond to an executable model of computation. Real-world computers are deterministic in nature, and there is no direct way to implement the CHOOSE operation. Consequently, non-deterministic algorithms cannot be translated into executable code.

Nevertheless, non-determinism is extremely useful from a theoretical standpoint. It provides a powerful framework for understanding deterministic computation, particularly in studying the limitations of efficient computation.

CST306 : Algorithm analysis and design

Chapter 6
Computational complexity

Narasimhan T.

6.1 Introduction

In the previous chapter, we observed that some problems admit efficient (polynomial-time) algorithms, whereas others appear to require nondeterministic polynomial-time approaches. In this chapter, we formalize these notions by introducing various complexity classes and examining the relative difficulty of computational problems. Before proceeding, we establish a formal language-based framework for our discussion.

6.2 Setting the "formal" stage

An *alphabet* Σ is a finite set of symbols. A *language* $\mathcal{L}$ over Σ is any set of strings formed using symbols from Σ. The set of all possible strings over Σ is denoted by Σ^*.

Any input instance I of a decision problem Π can be encoded as a binary string over the alphabet $\Sigma = \{0,1\}$ and is denoted by $\langle I \rangle$. Thus, every instance x of Π can be viewed as a binary string, and the set of all possible instances corresponds to Σ^*.

The ***language of a problem*** Π is defined as the set of all *yes-instances* of the problem. For example, the decision problem PATH corresponds to the language:

$$\text{PATH} = \{\langle G, u, v, k\rangle \mid G = (V, E) \text{ is an undirected graph, } u, v \in V,\ k \geq 0 \text{ is an integer, and there exists a path from } u \text{ to } v \text{ in } G \text{ consisting of at most } k \text{ edges}\}.$$

A language $\mathcal{L}$ is ***decided*** by an algorithm A if $A(x) = 1$ for all $x \in \mathcal{L}$ and $A(x) = 0$ for all $x \notin \mathcal{L}$. We say that $\mathcal{L}$ is decided in polynomial time if there exists a constant c such that, for every input string x of length n, the algorithm determines whether $x \in \mathcal{L}$ in time $O(n^c)$.

6.3 The complexity classes $\mathcal{P}$ and $\mathcal{NP}$

Informally, a ***complexity class*** is a collection of languages characterized by the computational resources required to decide membership, such as time or space.

The class $\mathcal{P}$ consists of all decision problems that can be decided by a deterministic algorithm in polynomial time.

The class $\mathcal{NP}$ consists of all decision problems that can be solved by a nondeterministic algorithm in polynomial time. It is important to note that "$\mathcal{NP}$" does not stand for *non-polynomial*, but for *nondeterministic polynomial time*.

6.3.1 Looking at the class $\mathcal{NP}$ from a different perspective

Recall that a nondeterministic algorithm consists of a *guessing* stage followed by a *verification* stage, where the latter is deterministic. This viewpoint leads to an alternative characterization of the class $\mathcal{NP}$.

We say that a language $\mathcal{L}$ is ***verified*** by an algorithm A if, for any string $x \in \mathcal{L}$, there exists a string y such that $A(x, y) = 1$. The string y is called a *certificate* (or witness), as it certifies that x is a yes-instance of $\mathcal{L}$.

For example, in the case of the decision problem PATH, the input x is an encoding $\langle G, u, v, k\rangle$, and a certificate y could be a sequence of vertices representing a path of length at most k from u to v in G.

An algorithm A with this property is called a ***verification algorithm***. The ***language verified*** by A is

$$\mathcal{L} = \{x \in \{0,1\}^* \mid \exists\, y \in \{0,1\}^* \text{ such that } A(x, y) = 1\}.$$

It is important to note that, for any string $x \notin \mathcal{L}$, there must be no certificate y proving that $x \in \mathcal{L}$.

This notion leads to an alternative definition of $\mathcal{NP}$: a language $\mathcal{L}$ belongs to $\mathcal{NP}$ if and only if there exists a deterministic polynomial-time verification algorithm.

Formally,

$$\mathcal{L} \in \mathcal{NP} \iff \mathcal{L} = \{x \in \{0,1\}^* \mid \exists \text{ a certificate } y \text{ with } |y| = O(|x|^c) \text{ and a deterministic polynomial-time algorithm } A \text{ such that } A(x, y) = 1\}.$$

Notation Convention

We use the terms *problem* and *language* interchangeably when referring to decision problems. Formally, each decision problem is identified with the language of its yes-instances. Hence, when we say that a problem belongs to a complexity class such as $\mathcal{P}$ or $\mathcal{NP}$, we mean that its corresponding language belongs to that class.

6.3.2 The Million-Dollar "$\mathcal{P}$ vs $\mathcal{NP}$" Conjecture

The complexity class $\mathcal{NP}$ is defined to include the class $\mathcal{P}$. Indeed, if there exists a polynomial-time algorithm A that decides a language $\mathcal{L}$, then A can be trivially converted into a two-argument verification algorithm that ignores the certificate and accepts exactly those inputs it determines to be in $\mathcal{L}$. Thus, $\mathcal{P} \subseteq \mathcal{NP}$.

It is unknown whether $\mathcal{P} = \mathcal{NP}$. The *Clay Mathematics Institute* has offered a prize of $1 million for resolving this question. Most researchers believe that $\mathcal{P}$ and $\mathcal{NP}$ are distinct classes. Intuitively, it seems unlikely that all problems whose solutions can be verified quickly can also be solved quickly; that is, it is hard to believe that efficient algorithms exist for many problems that have resisted solution for decades.

At a high level, the class $\mathcal{P}$ consists of problems that can be solved efficiently, while $\mathcal{NP}$ consists of problems for which a proposed solution can be verified efficiently. Thus, the $\mathcal{P}$ vs. $\mathcal{NP}$ question can be viewed as asking whether *finding* solutions is inherently harder than *verifying* them.

When faced with a problem in $\mathcal{NP}$, we generally do not expect to prove unconditionally that it does not belong to $\mathcal{P}$. Instead, we aim for conditional results: showing that if $\mathcal{P} \neq \mathcal{NP}$, then the problem is not in $\mathcal{P}$.

A useful contrapositive viewpoint is the following: if a problem is as hard as any problem in $\mathcal{NP}$ and yet admits a polynomial-time algorithm, then every problem in $\mathcal{NP}$ can be solved in polynomial time; that is, $\mathcal{NP} = \mathcal{P}$.

This viewpoint motivates the theory of $\mathcal{NP}$-*completeness*, which is based on the notion of reductions, discussed next.

6.4 Reductions

On what basis do we believe that certain problems do not admit efficient algorithms (apart from the historical fact that many researchers have tried and failed to find such algorithms)? A fundamental tool for providing such evidence is the notion of *reductions*, which allow us to relate the difficulty of different computational problems.

A **reduction** is a way of transforming one problem into another such that a solution to the latter can be used to solve the former. Intuitively, a problem A can be reduced to another problem B if every instance of A can be "rephrased" as an instance of B, in such a way that solving the instance of B yields a solution to the original instance of A.

For example, solving linear equations in one variable reduces to solving quadratic equations: given an instance $ax + b = 0$, we can rewrite it as $0x^2 + ax + b = 0$, whose solution also solves the original equation.

We now generalize this idea. Suppose we wish to solve a decision problem A in polynomial time, and we already know how to solve another decision problem B in polynomial time. Further, suppose there exists a procedure that transforms any instance α of A into an instance β of B such that:

- the transformation runs in polynomial time, and
- the answers are preserved; that is, α is a yes-instance of A if and only if β is a yes-instance of B.

Such a procedure is called a ***polynomial-time reduction***, and we write $A \xrightarrow{poly} B$ or $A \leq_p B$.

Figure 6.1 illustrates how such a reduction can be used to solve A using an algorithm for B:

1. Given an instance α of A, transform it into an instance β of B.
2. Run the polynomial-time algorithm for B on β.
3. Output the answer obtained for β as the answer for α.

Since each step runs in polynomial time, the overall procedure is also polynomial. Thus, by reducing A to B, we use the easiness of B to establish the easiness of A.

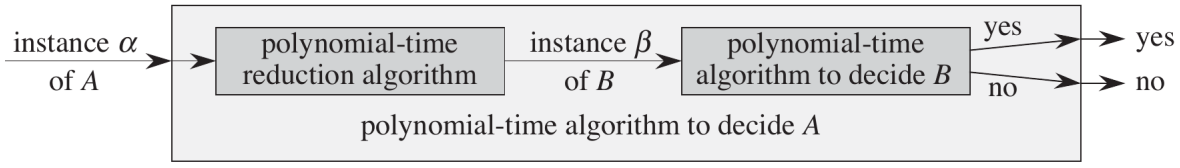


Figure 6.1: Using reductions to devise algorithms

Reductions play a central role in classifying problems according to their computational difficulty. If $A \leq_p B$, then solving A cannot be harder than solving B, since any algorithm for B yields an algorithm for A. In particular, if B is tractable, then so is A. Equivalently, if A is intractable and $A \leq_p B$, then B must also be intractable. Thus, difficulty flows in the direction of the reduction, while efficient algorithms flow in the opposite direction.

6.5 The complexity class $\mathcal{NP}$-*complete*

A decision problem Π is said to be $\mathcal{NP}$-*complete* if:

1. $\Pi \in \mathcal{NP}$, and
2. every problem $\Pi' \in \mathcal{NP}$ can be reduced to Π; that is, $\Pi' \leq_p \Pi$.

If a problem satisfies only the second property, it is called $\mathcal{NP}$-*hard*. Thus, a problem is $\mathcal{NP}$-*complete* if it is both $\mathcal{NP}$-hard and belongs to $\mathcal{NP}$.

An $\mathcal{NP}$-*complete* problem is, in a precise sense, among the hardest problems in $\mathcal{NP}$ with respect to polynomial-time solvability. The significance of such problems is the following: if any $\mathcal{NP}$-*complete* problem Π admits a polynomial-time algorithm, then every problem in $\mathcal{NP}$ can be solved in polynomial time. This is because, any problem $\Pi' \in \mathcal{NP}$ can be reduced to Π and then solved using the algorithm for Π.

Thus, the existence of a polynomial-time algorithm for even a single $\mathcal{NP}$-*complete* problem would imply that $\mathcal{P} = \mathcal{NP}$.

6.5.1 How to prove problems to be $\mathcal{NP}$-*complete*

In this section, we describe how to prove that problems are $\mathcal{NP}$-*complete* without directly reducing every problem in $\mathcal{NP}$ to the given problem. Once a single $\mathcal{NP}$-*complete* problem is known, other problems can be shown to be $\mathcal{NP}$-*complete* using polynomial-time reductions.

All such proofs follow a common structure. Given a new problem Π:

1. show that $\Pi \in \mathcal{NP}$,
2. reduce a known $\mathcal{NP}$-*complete* problem Π' to Π in polynomial time, thereby showing that Π is $\mathcal{NP}$-*hard*.

Thus, Π is both in $\mathcal{NP}$ and $\mathcal{NP}$-*hard*; hence, it is $\mathcal{NP}$-*complete*. The reduction from Π' to Π ensures that every problem in $\mathcal{NP}$ reduces to Π (via Π').

The problems introduced in the previous chapter are all $\mathcal{NP}$-*complete*. In fact, there are hundreds of such problems arising from diverse domains. We now prove the $\mathcal{NP}$-*completeness* of two representative problems.

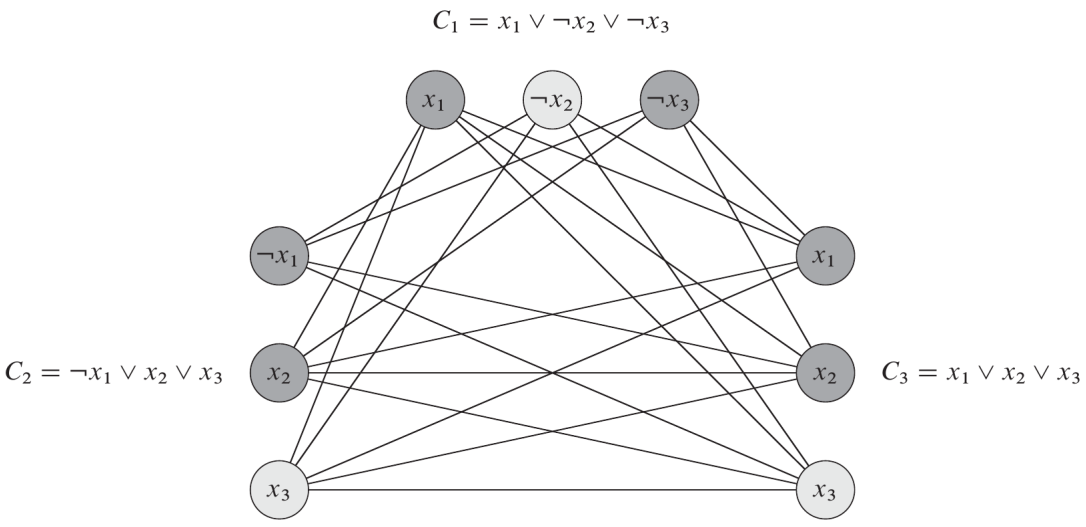


Figure 6.2: Construction of a graph from a 3-CNF formula

Theorem 6.1. *The clique problem is $\mathcal{NP}$-complete.*

Proof: Suppose we are given a graph $G = (V, E)$, an integer k, and a set $V' \subseteq V$ claimed to be a clique. The verification algorithm first checks that $|V'| = k$, and then verifies that V' is a clique by checking whether, for each pair $u, v \in V'$, the edge (u, v) belongs to E. This can be done in polynomial time. Hence, CLIQUE $\in \mathcal{NP}$, with V' serving as a certificate.

To prove *$\mathcal{NP}$-hardness*, we show that 3-SAT $\leq_p$ CLIQUE. Let $\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_k$ be a Boolean formula in 3-CNF with k clauses. For each $r = 1, 2, \ldots, k$, the clause C_r has exactly three literals l_1^r, l_2^r, l_3^r. We construct a graph $G = (V, E)$ such that ϕ is satisfiable if and only if G has a clique of size k.

The graph G is constructed as follows: for each clause $C_r = (l_1^r \vee l_2^r \vee l_3^r)$, we create a triple of vertices v_1^r, v_2^r, v_3^r. An edge is added between two vertices v_i^r and v_j^s if both of the following conditions hold:

1. the vertices belong to different triples (that is, $r \neq s$), and
2. the corresponding literals are consistent, i.e., l_i^r and l_j^s are not complements.

Thus, edges are added only between vertices belonging to two different triples. This construction can be carried out in polynomial time.

Consider the following 3-CNF formula:

$$\phi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3).$$

The constructed graph G is shown in Figure 6.2.

Claim: ϕ is satisfiable if and only if G has a clique of size k.

Forward direction: Suppose that ϕ has a satisfying assignment. Then each clause C_r contains at least one literal l_i^r that is assigned 1. Selecting one such literal from each clause yields a set V' of k vertices. Since no two chosen literals are complementary, every pair of corresponding vertices is connected by an edge. Hence, V' is a clique.

Backward direction: Suppose that G has a clique V' of size k. Since no edges connect vertices within the same triple, V' contains exactly one vertex from each triple. Assigning truth values so that the corresponding literals are TRUE yields a satisfying assignment for ϕ. (Any remaining variables may be assigned arbitrarily.) ■

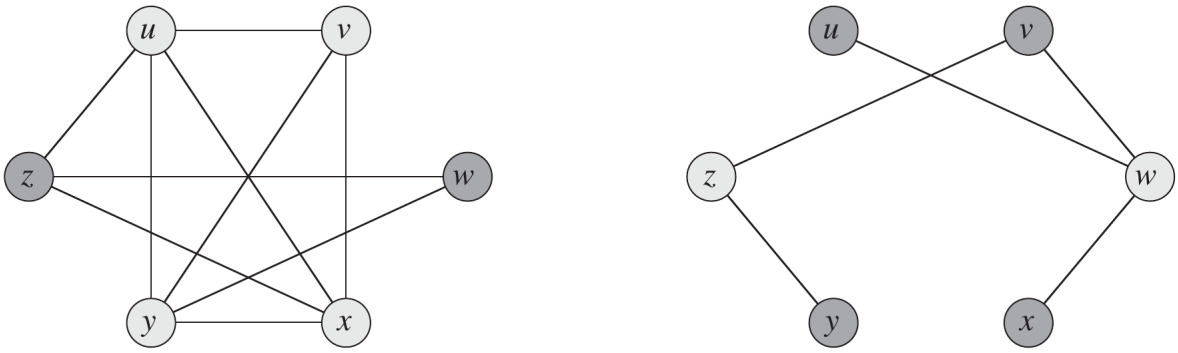


Figure 6.3: A graph G and its complement

Theorem 6.2. *The vertex cover problem is $\mathcal{NP}$-complete.*

Proof: Suppose we are given a graph $G = (V, E)$, an integer k, and a set $V' \subseteq V$ claimed to be a vertex cover. The verification algorithm first checks that $|V'| = k$, and then verifies that V' is a vertex cover by checking

whether, for each edge $(u,v) \in E$, at least one of u or v belongs to V'. This can be done in polynomial time. Hence, VERTEX-COVER $\in \mathcal{NP}$.

To show *$\mathcal{NP}$-hardness*, we prove that CLIQUE $\leq_p$ VERTEX-COVER. This reduction uses the notion of the complement of a graph. Given $G = (V,E)$, its complement is $\overline{G} = (V,\overline{E})$, where

$$\overline{E} = \{(u,v) \mid u,v \in V, u \neq v, \text{ and } (u,v) \notin E\}.$$

Figure 6.3 illustrates a graph and its complement. The graph G has a clique $V' = \{u,v,x,y\}$ of size 4, while $\overline{G}$ has a vertex cover $V - V' = \{w,z\}$ of size $|V| - k = 6 - 4 = 2$.

The reduction algorithm takes as input an instance $\langle G,k \rangle$ of CLIQUE. It constructs the complement graph $\overline{G}$, which can be computed in polynomial time by checking all pairs of vertices. The output of the reduction is the instance $\langle \overline{G}, |V| - k \rangle$ of VERTEX-COVER.

Claim: G has a clique of size k if and only if $\overline{G}$ has a vertex cover of size $|V| - k$.

Forward direction: Suppose that G has a clique $V' \subseteq V$ with $|V'| = k$. We claim that $V - V'$ is a vertex cover in $\overline{G}$.

Let (u,v) be any edge in $\overline{E}$. Then $(u,v) \notin E$. Since V' is a clique in G, every pair of vertices in V' is connected by an edge in E, so it cannot be the case that both u and v lie in V'. Hence, at least one of u or v is not in V', i.e., at least one endpoint lies in $V - V'$.

Thus, every edge in $\overline{G}$ is incident on a vertex in $V - V'$, so $V - V'$ is a vertex cover of $\overline{G}$. Since $|V'| = k$, we have $|V - V'| = |V| - k$.

Backward direction: Suppose that $\overline{G}$ has a vertex cover $V' \subseteq V$ with $|V'| = |V| - k$.

We prove that $V - V'$ is a clique in G. Let $u,v \in V - V'$. Then $u \notin V'$ and $v \notin V'$. Since V' is a vertex cover of $\overline{G}$, no edge in $\overline{E}$ can have both endpoints outside V'. Hence $(u,v) \notin \overline{E}$, which implies that $(u,v) \in E$.

Thus every pair of vertices in $V - V'$ is connected by an edge in G, so $V - V'$ is a clique. Its size is

$$|V - V'| = |V| - |V'| = k.$$

■

We established the *$\mathcal{NP}$-completeness* of CLIQUE via a reduction from 3-SAT. Subsequently, we proved the *$\mathcal{NP}$-completeness* of VERTEX-COVER by reducing CLIQUE to VERTEX-COVER. This yields the following chain of polynomial-time reductions:

$$\text{3-SAT} \xrightarrow{poly} \text{CLIQUE} \xrightarrow{poly} \text{VERTEX-COVER}.$$

Thus, hardness propagates through successive reductions, allowing new *$\mathcal{NP}$-complete* problems to be derived systematically from known ones.

6.5.2 The First *$\mathcal{NP}$-complete* Problem

The reduction chain discussed earlier ultimately rests on a foundational result that does not depend on any previously known *$\mathcal{NP}$-complete* problem.

This starting point is provided by the **Cook–Levin Theorem**, which establishes that CIRCUIT-SAT is *$\mathcal{NP}$-complete*. The key idea is that any nondeterministic polynomial-time computation can be simulated by a polynomial-size Boolean circuit, and satisfiability of such circuits captures all of $\mathcal{NP}$.

From this foundation, all other *$\mathcal{NP}$-complete* problems arise through chains of polynomial-time reductions, for example:

$$\text{CIRCUIT-SAT} \to \text{3-SAT} \to \text{CLIQUE} \to \text{VERTEX-COVER}.$$

CST306 : Algorithm analysis and design

Chapter 7
Coping with $\mathcal{NP}$-completeness

Narasimhan T.

7.1 Introduction

For a theoretical computer scientist, showing that a problem is *$\mathcal{NP}$-complete* is rarely the end of the story. There is usually a reason why the problem was considered in the first place, and that motivation does not disappear once we know that no polynomial-time algorithm is likely to exist. We still seek algorithms that solve the problem efficiently in practice. What we now understand is that we should not expect a polynomial-time algorithm that solves every instance optimally in the worst case. This naturally leads to the question: *what do we do next?*

This question lies at the heart of this chapter and has also driven much of modern research in algorithms and computational complexity. $\mathcal{NP}$-completeness is not a dead end; rather, it marks the beginning of a more nuanced exploration of algorithmic design.

Even when a problem is *$\mathcal{NP}$-complete*, there are several practical avenues to explore:

1. **Intelligent exhaustive search:** Exponential time does not necessarily imply impracticality. It indicates poor scalability. For small input sizes, algorithms with exponential running time may still be perfectly usable. Techniques such as backtracking and branch-and-bound fall into this category.
2. **Special cases:** Many *$\mathcal{NP}$-complete* problems admit polynomial-time solutions when restricted to specific instances or parameters. In practice, we are often interested only in such structured or constrained inputs, for which efficient algorithms may exist.
3. **Approximate solutions:** In this approach, we relax the requirement of exact optimality and instead aim for solutions that are close to optimal. In many real-world applications, near-optimal solutions are sufficient. Even for *$\mathcal{NP}$-complete* problems, it is often possible to design polynomial-time approximation algorithms with provable guarantees, either in the worst case or in expectation.

7.2 Approximation Algorithms

We call an algorithm that returns near-optimal solutions an *approximation algorithm*. Approximation is one of the most effective methods for constructing polynomial-time algorithms that come close to solving computa-

tionally hard problems. Although such algorithms do not always produce optimal solutions, in many cases we can still guarantee how close the output is to an optimal solution.

The general setting is as follows. We are given a problem instance I, which may represent an encoding of a set of numbers, a graph, or another combinatorial structure. For this instance, there is typically a large set $\mathcal{F}$ of feasible solutions. We also have a cost function C, which assigns a numeric value $C(S)$ to each solution $S \in \mathcal{F}$.

Optimization problems are of two types: *maximization* and *minimization*.

In the *maximization* version, we seek an optimal solution $S^* \in \mathcal{F}$ such that

$$C(S^*) = C^* = \max\{C(T) \mid T \in \mathcal{F}\}.$$

That is, we aim to find a solution with maximum cost.

The *minimization* version is defined as

$$C(S^*) = C^* = \min\{C(T) \mid T \in \mathcal{F}\}.$$

In both cases, C^* denotes the optimal value.

7.2.1 Performance parameters for approximation algorithms

The goal of an approximation algorithm is to compute solutions that are as close as possible to the optimal value within a reasonable amount of time. Ideally, we would like to quantify how close the computed solution is to the optimal value C^*. To formalize this notion, let C denote the cost of the solution returned by the algorithm.

For a maximization problem, we have $0 < C \leq C^*$, and a natural way to quantify the quality of the solution is the ratio

$$\frac{C^*}{C},$$

which measures how larger the optimal value is compared to the value returned by the algorithm.

For a minimization problem, we have $0 < C^* \leq C$, and the corresponding ratio is

$$\frac{C}{C^*},$$

which measures how much worse the computed solution is compared to the optimal.

In both cases, the goal of an approximation algorithm is to bound this ratio by a function of the input size. This leads to a unified notion of *approximation ratio*.

We say that an algorithm has an **approximation ratio $\rho(n)$** if, for any input of size n, the cost C of the solution produced by the algorithm satisfies

$$\max\left(\frac{C}{C^*}, \frac{C^*}{C}\right) \leq \rho(n).$$

If an algorithm achieves an approximation ratio of $\rho(n)$, we call it a **$\rho(n)$-approximation algorithm**. When $\rho(n)$ is a constant (i.e., independent of the input size n), we simply denote it by ρ and refer to the algorithm as a **ρ-approximation algorithm**.

Note that when $\rho = 1$, the algorithm always returns an optimal solution, since this implies $C = C^*$ for every input.

An approximation algorithm with a large approximation ratio may produce solutions that are far from optimal. For any input, the **relative error** is defined as

$$\frac{|C - C^*|}{C^*}.$$

Note that the relative error is always non-negative.

An approximation algorithm has a **relative error bound** of $e(n)$ if

$$\frac{|C - C^*|}{C^*} \leq e(n).$$

From the definition of the approximation ratio,

$$\max\left(\frac{C}{C^*}, \frac{C^*}{C}\right) \leq \rho(n),$$

which implies that both

$$\frac{C}{C^*} \leq \rho(n) \quad \text{and} \quad \frac{C^*}{C} \leq \rho(n).$$

Hence, in either case,

$$\frac{|C - C^*|}{C^*} \leq \rho(n) - 1.$$

Therefore,

$$e(n) \leq \rho(n) - 1.$$

7.2.2 Approximation schemes

The bound $e(n) \leq \rho(n) - 1$ shows that the approximation ratio directly controls the relative error of an algorithm. This suggests a natural question: can we design algorithms where the error can be made arbitrarily small, while still remaining computationally efficient?

This leads to the idea of an *approximation scheme* for an optimization problem. An **approximation scheme** is an approximation algorithm that, in addition to an input instance, takes a parameter $\varepsilon > 0$ which specifies the desired accuracy. The idea is that smaller values of ε correspond to solutions that are closer to optimal. Formally, for any fixed ε, the scheme produces a $(1+\varepsilon)$-approximate solution.

The running time of such an algorithm depends both on n, the size of the input instance, and on ε, which controls the accuracy. Thus, there is a trade-off between computational effort and closeness to optimality.

We say that an approximation scheme is a **Polynomial-Time Approximation Scheme** or **PTAS** if, for every fixed $\varepsilon > 0$, the algorithm runs in time polynomial in the input size n. In other words, for any fixed accuracy requirement, the algorithm remains efficient with respect to the input size.

We say that an approximation scheme is a **Fully Polynomial-Time Approximation Scheme** or **FPTAS** if it is an approximation scheme whose running time is polynomial in both n and $1/\varepsilon$. The dependence on $1/\varepsilon$ is natural because achieving higher accuracy (smaller ε) typically requires finer computation, and hence a proportionally larger computational effort. Thus, the algorithm remains efficient even as the required accuracy becomes increasingly fine.

We have now developed the basic framework of approximation algorithms, including approximation ratios and approximation schemes such as PTAS and FPTAS. The next natural step is to understand how these ideas are applied in practice.

In particular, we now turn to two classical optimization problems that are central to the study of approximation algorithms: BIN PACKING and GRAPH COLORING. These problems are $\mathcal{NP}$*-hard* and have been extensively studied as benchmark problems for designing approximation algorithms and approximation schemes.

They illustrate two different types of structural difficulty and will help demonstrate how the techniques developed in this chapter are applied in concrete settings.

7.2.3 The bin packing problem

In the BIN PACKING problem, we are given a set of n items, each with a weight in the interval $(0,1)$. The objective is to pack all items into the minimum number of unit-capacity bins such that the total weight in each bin does not exceed 1. Each bin may contain any subset of items as long as their total size respects this capacity constraint. Formally, the problem is defined as follows:

BIN-PACK: Given a set of n objects, where each object i has size s_i satisfying $0 < s_i < 1$, partition the objects into the minimum number of bins of capacity 1 each, such that the sum of sizes of objects in any bin does not exceed 1.

The BIN PACKING problem arises naturally in a variety of practical settings, including:

1. allocating files or data blocks across multiple storage disks,
2. scheduling jobs on identical machines with limited capacity,
3. packing advertisements into fixed-length television or radio broadcast slots,
4. storing collections of media (such as music or video files) on tapes, CDs, or other storage media.

7.2.3.1 Solving BIN-PACK

It is easy to see that the BIN PACKING problem is $\mathcal{NP}$-*hard*. Therefore, it is unlikely that a polynomial-time algorithm exists that always computes an optimal solution. However, there are several simple and effective approximation algorithms for this problem. In this section, we discuss one such algorithm, known as the *First Fit* algorithm.

Informally, the algorithm can be described as follows:

> *Scan the bins in order and place each new item in the first bin that has sufficient remaining capacity. A new bin is opened only when the current item does not fit into any of the existing bins.*

More precisely, we number the bins as $1,2,3,\ldots$. When processing item i, the First Fit algorithm APPROX-BIN-PACK-FF attempts to place it into the already opened bins in increasing index order. If no existing bin can accommodate the item, a new bin is opened and the item is placed in it.

The algorithm APPROX-BIN-PACK-FF(S) takes as input an array S, where $S[i] = s_i$ denotes the size of item i. Let b denote the number of bins used by an optimal solution. The boolean variable *packed* indicates whether an item has been assigned to a bin. The quantity $Total[j]$ denotes the total load in bin j, computed as the sum of sizes of all items assigned to that bin.

```
APPROX-BIN-PACK-FF(S)
 1  n = |S|
 2  b = 0    // initially there are no bins.
 3  for i = 1 to n    // pack the n items one at a time
 4       packed = FALSE
 5       j = 1    // Start from the first bin
 6       while j ≤ b AND packed == FALSE    // try to find space for the item in the existing b bins
 7            if Total[j] + S[i] ≤ 1    // Check if item i fits in bin j
 8                 Total[j] = Total[j] + S[i]
 9                 packed = TRUE
10            else
11                 j = j + 1    // Go to next bin
12       if packed == FALSE    // item i does not fit in any existing bin
13            b = b + 1    // Create a new bin
14            Total[b] = S[i]
15  return b
```

Example 7.1. Let us solve the bin packing problem for the following instance with 7 objects:

i	1	2	3	4	5	6	7
s_i	0.5	0.3	0.4	0.8	0.2	0.2	0.2

The steps are:

0. $b = 0$
1. $s_1 = 0.5$. Create bin_1 and put item 1. $b = 1, Total[1] = 0.5$
2. $s_2 = 0.3$. put item 2 in bin_1. $Total[1] = 0.8$
3. $s_3 = 0.4$. Create bin_2 and put item 3. $b = 2, Total[1] = 0.8, Total[2] = 0.4$
4. $s_4 = 0.8$. Create bin_3 and put item 4. $b = 3, Total[1] = 0.8, Total[2] = 0.4, Total[3] = 0.8$
5. $s_5 = 0.2$. put item 5 in bin_1. $Total[1] = 1, Total[2] = 0.4, Total[3] = 0.8$
6. $s_6 = 0.2$. put item 6 in bin_2. $Total[1] = 1, Total[2] = 0.6, Total[3] = 0.8$
7. $s_7 = 0.2$. put item 7 in bin_2. $Total[1] = 1, Total[2] = 0.8, Total[3] = 0.8$

Thus the optimal number of bins required is $b = 3$. The final bin contents are given below:

- $\text{bin}_1 - \{1, 2, 5\}$
- $\text{bin}_2 - \{3, 6, 7\}$
- $\text{bin}_3 - \{4\}$

■

Theorem 7.1. APPROX-BIN-PACK-FF *is a 2-approximation polynomial time algorithm.*

Proof: We first analyze the running time of APPROX-BIN-PACK-FF. For each of the n items, we scan the existing bins in order to find a feasible placement. Since at most n bins can be created (one per item in the worst case), each item requires at most $\mathcal{O}(n)$ work. Hence, the total running time is $\mathcal{O}(n^2)$.

We now analyze the quality of the solution. Let C denote the number of bins used by the algorithm, and let C^* denote the number of bins in an optimal solution.

We classify bins produced by the algorithm as follows: a bin is called *heavy* if its total load is greater than 1/2, and *light* otherwise.

Claim: At most one bin can be light.
Proof. Suppose, for contradiction, that there exist two distinct bins B_i and B_j with $i < j$, both having load at most 1/2. Let x be the first item placed into bin B_j. When x was considered, bin B_i was already open and had remaining capacity at least 1/2, so x would have fit in B_i. This contradicts the First Fit rule, which places an item in the first feasible bin. Hence, at most one bin is light. □

From the claim, all but at most one bin are heavy, i.e., have load strictly greater than 1/2. Therefore, if $C > 1$, we have

$$\sum_{i=1}^{n} s_i > \frac{C-1}{2}.$$

Rearranging gives

$$C < 2\sum_{i=1}^{n} s_i + 1.$$

Let $W = \sum_{i=1}^{n} s_i$ denote the total size of all items. Since every bin has capacity 1, any feasible packing (in particular the optimal one) satisfies

$$C^* \geq W.$$

Combining the inequalities, we obtain

$$C < 2W + 1 \leq 2C^* + 1.$$

For $C^* \geq 1$, this implies

$$C \leq 2C^* \implies \frac{C}{C^*} \leq 2.$$

Thus, APPROX-BIN-PACK-FF is a **2-approximation algorithm** for the BIN PACKING problem. ■

7.2.4 The graph colouring problem

A *proper colouring* of a graph is an assignment of colours to its vertices such that no two adjacent vertices receive the same colour. A *k-colouring* of a graph is a proper colouring that uses at most k colours.

More formally, given an undirected graph $G = (V, E)$, a k-colouring is a function

$$c : V \to \{1, 2, \ldots, k\}$$

such that $c(u) \neq c(v)$ for every edge $(u, v) \in E$. The values $1, 2, \ldots, k$ represent distinct colours.

A graph is said to be *k-colourable* if it admits a k-colouring. Any graph with n vertices is trivially n-colourable by assigning distinct colours to each vertex.

The *chromatic number* of a graph G, denoted $\chi(G)$, is the smallest integer k such that G is k-colourable.

The ***graph colouring problem*** is the problem of computing $\chi(G)$ for a given graph G. The related decision problem, called the ***k-colouring problem***, asks whether G admits a proper colouring using at most k colours.

It is *$\mathcal{NP}$-complete* to decide whether a graph is k-colourable for any fixed $k \geq 3$. In contrast, 2-colourability can be decided in polynomial time, as it is equivalent to testing whether a graph is bipartite. Since the problem is intractable for $k \geq 3$, we consider approximation algorithms and focus on the case $k = 3$.

7.2.4.1 3-colouring approximation

We aim to design a polynomial-time approximation algorithm that, given a 3-colourable graph, produces a colouring using as few colours as possible. The algorithm is based on the following facts.

Fact 1: Any graph G can be coloured in polynomial time using at most $1 + \Delta(G)$ colours, where $\Delta(G)$ is the maximum degree of G.

Why? Process the vertices in any arbitrary order. When colouring a vertex v, at most $\Delta(G)$ of its neighbours may already be coloured, so at most $\Delta(G)$ colours are forbidden. Hence, one of the colours from $\{1, \ldots, \Delta(G)+1\}$ is always available, yielding a proper colouring using at most $\Delta(G)+1$ colours. The procedure runs in polynomial time as $\Delta(G) \leq n - 1$. □

Fact 2: Let G be a 3-colourable graph and let $v \in V$. Then the subgraph induced by $\{v\} \cup \mathcal{N}(v)$, where $\mathcal{N}(v)$ denotes the set of neighbours of v, can be properly coloured using at most 3 colours in polynomial time.

Why? Since G is 3-colourable, every induced subgraph of G, including $G' = G[\{v\} \cup \mathcal{N}(v)]$, is also 3-colourable. Hence, $\chi(G') \leq 3$.

G' has at most $\Delta(G)+1$ vertices. Since the number of colours is constant (3), we can find a valid colouring of G' by checking feasible assignments over this fixed colour set and verifying consistency with edges. As the size of G' is bounded locally and the colour set is constant, this procedure runs in polynomial time in n. □

We now describe the approximation algorithm 3-COLOURING-APPROX(G) for the 3-colouring problem. The input is a 3-colourable graph G, and the output is the number of colours C used by the algorithm to produce a proper colouring of G.

3-COLOURING-APPROX(G)

```
1  n = |G.V|
2  C = 0
3  while Δ(G) ≥ √n
4        Let v be a vertex of maximum degree
5        3-colour v and its neighbours using at most 3 new colours      // From Fact 2
6        C = C + 3
7        Delete v and all its neighbours from G
8  Colour the remaining graph G' using at most √n colours      // From Fact 1
9  C = C + √n
10 return C
```

Theorem 7.2. 3-COLOURING-APPROX *is a polynomial-time $\mathcal{O}(\sqrt{n})$-approximation algorithm for colouring a 3-colourable graph.*

Proof: We first bound the number of iterations of the **while** loop. In each iteration, the algorithm selects a vertex v of maximum degree $\Delta(G) \geq \sqrt{n}$ and removes v together with all its neighbours. Hence at least $\sqrt{n}+1$ vertices are removed per iteration, so the number of iterations is at most $\mathcal{O}(\sqrt{n})$.

Next, the running time is polynomial since each iteration performs only polynomial work, and there are $\mathcal{O}(\sqrt{n})$ iterations.

For the approximation ratio, each iteration introduces at most 3 new colours (by **Fact 2**), contributing $\mathcal{O}(\sqrt{n})$ colours in total. The final step contributes at most $\sqrt{n}$ colours (**Fact 1**). Hence

$$C = \mathcal{O}(\sqrt{n}).$$

Since the input graph is 3-colourable, the optimal value is $C^* = 3$. Therefore,

$$\frac{C}{C^*} = \mathcal{O}(\sqrt{n}),$$

which completes the proof. ■

7.3 Randomized Algorithms

When an algorithm faces a choice, it may be more efficient to select an option at random rather than expend significant effort determining the optimal one. This is especially useful when the cost of finding the best choice outweighs the expected benefit. Consequently, the behavior of such algorithms depends not only on the input, but also on the outcomes of random events.

A ***randomized algorithm*** (or ***probabilistic algorithm***) is one that, in addition to its input, has access to a source of randomness used to guide its execution. In practice, this randomness is provided by a *random-number generator*. Since truly random generators are rarely available, algorithms usually rely on *pseudorandom generators*: deterministic procedures that produce sequences appearing random, initialized via a *seed*. The same seed always yields the same sequence.

Randomized algorithms should not be confused with ***nondeterministic algorithms***. A nondeterministic algorithm is a theoretical construct that, when faced with a choice, is assumed to explore *all* possible options simultaneously – as if spawning independent copies of itself, each pursuing a different path. Such algorithms do not exist in practice; they are a formal tool used in complexity theory to reason about computational hardness.

By contrast, a randomized algorithm is fully realizable: it makes choices at random using a concrete source of randomness, and follows a single execution path determined by those choices.

A key feature of randomized algorithms is that repeated executions on the same input may yield different behaviors: the running time, and in some cases even the output, can vary across runs. Randomness does *not*, however, imply complete unpredictability. While individual runs may differ, aggregate behavior stabilizes over many executions.

Stability Under Repetition

Consider the following simple randomized Python procedure:

```
import random

def biased_trial():
    return random.random() < 0.7

def estimate_probability(trials=1000):
    success = sum(biased_trial() for _ in range(trials))
    return success / trials
```

The function `biased_trial` simulates a Bernoulli trial returning `True` with probability 0.7. The function `estimate_probability` repeats this experiment and returns the fraction of successes, which converges to 0.7 as the number of trials grows.

A sample interactive session illustrates this convergence:

```
>>> estimate_probability(10)
0.8
>>> estimate_probability(10)
0.5
>>> estimate_probability(100)
0.72
>>> estimate_probability(100)
0.73
>>> estimate_probability(100)
0.72
>>> estimate_probability(1000)
0.707
>>> estimate_probability(1000)
0.683
>>> estimate_probability(1000)
0.707
>>> estimate_probability(10000)
0.7009
>>> estimate_probability(10000)
0.6941
>>> estimate_probability(10000)
0.7025
```

Each run yields a slightly different value. Yet as the trial count increases, the estimate converges to the

true probability (0.7) and the variance across runs diminishes. Individual outcomes are random; aggregate behavior is highly predictable.

This principle underlies the design of many randomized algorithms. In some settings, an algorithm may produce an incorrect result, provided the error probability is sufficiently small. Running the algorithm independently multiple times and combining results—for instance, by majority vote—reduces the error probability *exponentially*, yielding arbitrarily high confidence.

Randomness also permits tolerating occasional failures: if the algorithm fails to return an answer with small probability, it may simply be restarted on the same input with fresh randomness. Furthermore, when multiple correct solutions exist, different runs may yield distinct valid outputs—a property that can be advantageous when diversity of solutions is desirable.

Randomization is a powerful tool in algorithm design. It frequently produces algorithms that are simpler, more efficient, and easier to implement than their deterministic counterparts, often achieving better time or space complexity while guaranteeing correctness with high probability. Crucially, this success probability depends only on the algorithm's internal random choices, and not on any assumptions about the input.

We now formalize two fundamental ways in which randomness is used in algorithm design.

7.3.1 Las Vegas and Monte Carlo Algorithms

Randomized algorithms are broadly classified into two categories.

Las Vegas algorithms. These always produce a *correct* result whenever they return an answer, but may occasionally decline to answer. Correctness is guaranteed; only the running time is random.

Monte Carlo algorithms. These always return an answer, but the answer may be incorrect with some small probability. The error probability can be reduced arbitrarily by repeating the algorithm with independent random choices.

Example 7.2. The distinction is illustrated by the query: *"When did India gain independence?"*

A **Las Vegas** algorithm, run ten times, might produce:

1947, 1947, 1947, 1947, 1947, –, –, 1947, 1947, 1947

where ' – ' denotes a refusal to answer. Every answer given is correct, but some runs produce no output.

A **Monte Carlo** algorithm, run ten times, might produce:

1947, 1947, 2024, 1947, 1947, 1947, 2000, 1947, 1947, 1946

Every run returns an answer, but some answers are incorrect.

Remark. These two paradigms highlight a fundamental distinction: whether randomness affects the *correctness* of the output or only the *efficiency* of the computation.

7.3.2 Expected versus Average Time

When analyzing randomized algorithms, it is important to distinguish carefully between the terms *average* and *expected*, as they refer to fundamentally different sources of variability.

Average time. The ***average time*** of a *deterministic* algorithm is the mean running time over all inputs of a given size, assuming each input is equally likely. Here, the variability comes from the *input*: different inputs may cause the algorithm to run faster or slower, and we average over this distribution.

Expected time. The ***expected time*** of a *randomized* algorithm is defined for each fixed input individually: it is the mean running time taken over all possible random choices made by the algorithm on that input. Here, the input is held fixed, and the variability comes entirely from the *internal randomness* of the algorithm itself.

The distinction is subtle but significant. Average-case analysis of a deterministic algorithm says: *"on a typical input, the algorithm runs quickly."* Expected-time analysis of a randomized algorithm says instead: *"on any fixed input, the algorithm is unlikely to take long."* The latter is a stronger guarantee, since it holds for every input, independent of any assumption about the input distribution.

It is also meaningful to consider the ***worst-case expected time***: the maximum, over all inputs of a given size, of the expected running time. This combines the two notions — it asks how bad the expected performance can be in the worst case — and serves as the standard measure of efficiency for randomized algorithms. It must not be confused with the running time under the *worst possible sequence of random choices*, which is a much more pessimistic and less informative quantity. Las Vegas algorithms, in particular, are well-suited to this measure: although they may occasionally run for a long time, their expected running time is often significantly better than the worst-case running time of the best known deterministic algorithm for the same problem.

Having introduced the basic principles of randomized algorithms and the notion of expected running time, we now turn to a concrete and widely-used example. Randomized Quicksort illustrates how randomness can be used to eliminate dependence on the input order and to guarantee good performance in expectation, even though individual executions may vary.

7.3.3 Randomized Quicksort

Recall that the deterministic version of QUICKSORT follows a classic divide-and-conquer strategy. Given an array A of n elements, it proceeds as follows:

1. Choose the rightmost element of A as the pivot x.
2. Call the auxiliary procedure PARTITION, which determines the correct position of x in the sorted array and rearranges the remaining elements into two subarrays: A_L, consisting of elements less than or equal to x, and A_R, consisting of elements strictly greater than x.
3. Recursively sort A_L and A_R.

The running time of QUICKSORT depends critically on the quality of the partition produced at each step.

Best case. PARTITION splits the array evenly, so $|A_L| = |A_R| = n/2$ at every level. The recurrence resolves to $T(n) = \mathcal{O}(n \lg n)$.

Average case. Assuming all permutations of the input are equally likely, the expected running time is $\Theta(n \lg n)$. This assumption, however, rarely holds in practice.

Worst case. When the input is already sorted (or reverse sorted), every partition is maximally unbalanced: one subarray has $n-1$ elements and the other has none. The running time degrades to $\mathcal{O}(n^2)$.

This $\mathcal{O}(n^2)$ worst-case behaviour stems directly from the fixed pivot selection rule: always choosing the rightmost element makes the algorithm vulnerable to adversarial or structured inputs. Two approaches address this vulnerability.

The first introduces a *preprocessing step* that randomly permutes the input before sorting begins. Since the permutation is chosen uniformly at random, no fixed input can consistently trigger the worst case, and the expected running time is $\Theta(n \lg n)$. This step takes $\Theta(n)$ time and adds no asymptotic overhead to the overall algorithm.

The second, simpler approach introduces randomness *directly into the algorithm* itself: rather than always selecting the rightmost element, the pivot is chosen uniformly at random at each recursive call. This eliminates the preprocessing step entirely and achieves the same effect. Crucially, the analysis no longer requires any assumption about the distribution of the input — the randomness is internal to the algorithm, so the bound holds for every input unconditionally. The result is the Las Vegas randomized algorithm RANDOMIZED-QUICKSORT, shown below, where p and r denote the indices of the first and last elements of the current subarray.

```
RANDOMIZED-QUICKSORT(A, p, r)
1  if p < r
2       q = RANDOMIZED-PARTITION(A, p, r)
3       RANDOMIZED-QUICKSORT(A, p, q − 1)
4       RANDOMIZED-QUICKSORT(A, q + 1, r)
```

The auxiliary procedure RANDOMIZED-PARTITION calls RANDOM(p,r), which returns a uniformly random index $t \in \{p,\ldots,r\}$. The element $A[t]$ is chosen as the pivot. The procedure then scans the subarray, and upon completion, places the pivot into its correct position via a final swap on line 8, returning the pivot's index.

```
RANDOMIZED-PARTITION(A, p, r)
1  t = RANDOM(p, r)
2  x = A[t]          // the pivot value
3  i = p − 1
4  for j = p to r − 1
5       if A[j] ≤ x
6            i = i + 1
7            exchange A[i] with A[j]
8  exchange A[i + 1] with A[r]
9  return i + 1
```

Before analyzing the running time of RANDOMIZED-QUICKSORT, we recall two essential probabilistic tools.

A Expected value.

For a discrete random variable X over a sample space $\mathcal{S}$, the ***expected value*** of X is:

$$\mathbb{E}[X] = \sum_{e \in \mathcal{S}} \Pr(e) X(e) \tag{7.3.1}$$

In other words, the expectation of a random variable X is just its average value over $\mathcal{S}$, where each event e in the sample space is weighted according to its probability $\Pr(e)$.

B Linearity of expectation.

For any two random variables X and Y (regardless of whether they are independent),

$$\mathbb{E}[X+Y] = \mathbb{E}[X] + \mathbb{E}[Y] \tag{7.3.2}$$

This property extends to any finite sum of random variables and will be the key tool in the analysis that follows.

7.3.3.1 Running Time Analysis of Randomized Quicksort

As is standard for sorting algorithms, we measure the running time of RANDOMIZED-QUICKSORT in terms of the number of *element comparisons* it performs, since comparisons constitute the dominant cost in any reasonable implementation. Our goal is to bound the *expected* number of comparisons over a single execution. Observe that all comparisons occur inside RANDOMIZED-PARTITION, where each non-pivot element is compared against the chosen pivot exactly once.

A Setup

For $1 \le i \le n$, let $A_{(i)}$ denote the element of *rank* i in A, i.e., $A_{(i)}$ is the i-th smallest element. Thus $A_{(1)}$ is the minimum and $A_{(n)}$ the maximum. Define the indicator random variable X_{ij} for each pair $1 \le i < j \le n$:

$$X_{ij} = \begin{cases} 1 & \text{if } A_{(i)} \text{ and } A_{(j)} \text{ are compared during the execution,} \\ 0 & \text{otherwise,} \end{cases}$$

Since no pair of elements is ever compared more than once, the total number of comparisons is exactly

$$X = \sum_{i=1}^{n} \sum_{j=i+1}^{n} X_{ij}$$

Taking expectations and applying linearity (7.3.2),

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{i=1}^{n} \sum_{j=i+1}^{n} X_{ij}\right] = \sum_{i=1}^{n} \sum_{j=i+1}^{n} \mathbb{E}[X_{ij}] \tag{7.3.3}$$

B Computing $\mathbb{E}[X_{ij}]$

Let $p_{ij} = \Pr(X_{ij} = 1)$ denote the probability that $A_{(i)}$ and $A_{(j)}$ are compared. Since X_{ij} is an indicator variable, (7.3.1) gives

$$\mathbb{E}[X_{ij}] = p_{ij} \cdot 1 + (1 - p_{ij}) \cdot 0 = p_{ij} \tag{7.3.4}$$

C Computing p_{ij}

To compute p_{ij}, consider the contiguous block of ranked elements $A_{(i)}, A_{(i+1)}, \ldots, A_{(j)}$, which contains $j - i + 1$ elements. The value of X_{ij} is determined entirely by which element from this block is first selected as a pivot during the execution.

Observation 1. If this element is some interior $A_{(k)}$ with $i < k < j$, then $A_{(i)}$ and $A_{(j)}$ are separated into different subarrays and will never be compared. Hence $X_{ij} = 0$.

Observation 2. If this element is either $A_{(i)}$ or $A_{(j)}$ itself, then the two are compared directly before any separation can occur. Hence $X_{ij} = 1$.

These two observations are exhaustive and mutually exclusive, so p_{ij} is simply the probability that $A_{(i)}$ or $A_{(j)}$ is chosen before any interior element of the block. Since each element is equally likely to be selected first, we have

$$p_{ij} = \Pr(X_{ij} = 1) = \frac{2}{j - i + 1} \tag{7.3.5}$$

D Bounding $\mathbb{E}[X]$

Combining (7.3.3), (7.3.4), and (7.3.5), and substituting $k = j - i$ to re-index the inner sum,

$$\begin{aligned}
\mathbb{E}[X] &= \sum_{i=1}^{n} \sum_{j=i+1}^{n} \frac{2}{j-i+1} \\
&\leq \sum_{i=1}^{n} \sum_{k=1}^{n-i+1} \frac{2}{k} \qquad (k = j-i) \\
&\leq 2 \sum_{i=1}^{n} \sum_{k=1}^{n} \frac{1}{k} \\
&= 2 \sum_{i=1}^{n} H_n \\
&= 2nH_n
\end{aligned}$$

where $H_n = \sum_{k=1}^{n} \frac{1}{k}$ is the n-th harmonic number. Thus $\mathbb{E}[X] \leq 2nH_n$.

Using the well-known asymptotic $H_n = \ln n + \mathcal{O}(1)$, we obtain the following result.

Theorem 7.3. *The expected number of comparisons performed by* RANDOMIZED-QUICKSORT *on any input of size n is at most $2nH_n$. Consequently, its worst-case expected running time is $\mathcal{O}(n \lg n)$.*

Note that this bound is unconditional: unlike the $\Theta(n \lg n)$ average-case bound for deterministic QUICKSORT, it requires no assumption about the distribution of the input. The $\mathcal{O}(n \lg n)$ guarantee holds for every input, with the randomness coming solely from the pivot choices made internally by the algorithm.

7.3.4 Does Randomization Help for $\mathcal{NP}$-*complete* Problems?

A natural question arises: can randomization help in solving $\mathcal{NP}$-complete problems? The answer depends on what guarantees one demands from the algorithm.

If the requirement is that the algorithm *always* produces a correct solution, then the answer is no: randomization offers no relief from the hardness of $\mathcal{NP}$-completeness in this strict sense. If, however, one is willing to tolerate occasional errors or accept that the algorithm may sometimes fail to return an answer, then the answer is yes. In such settings, repeatedly running the algorithm with independent random choices drives the probability of obtaining a correct solution arbitrarily close to 1.

More broadly, randomization is a powerful tool for developing *faster* algorithms for $\mathcal{NP}$-complete problems in practice, even if it cannot reduce their theoretical hardness. Randomized algorithms do exist for problems such as 3-SAT, though a detailed treatment is beyond the scope of this discussion.